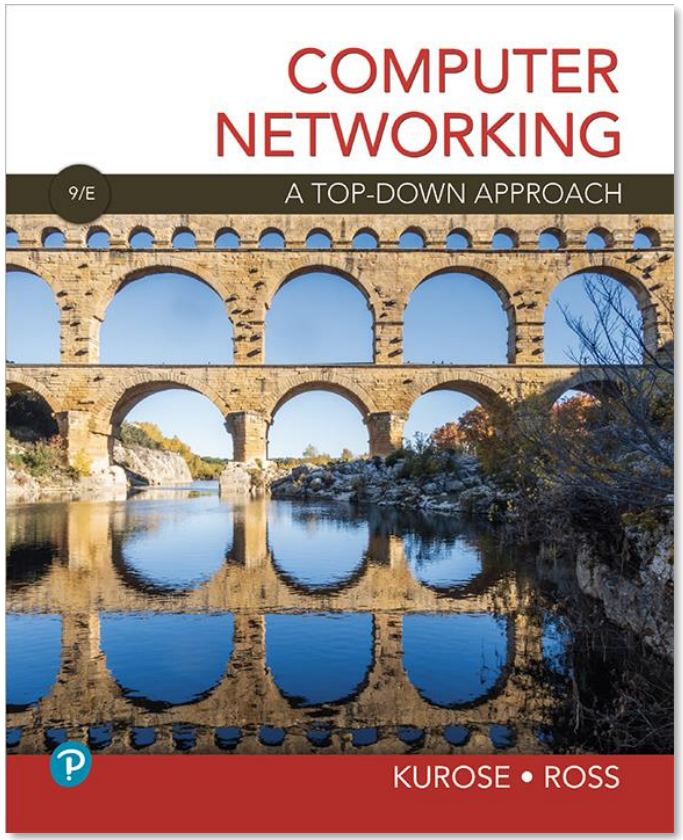
Chapter 3
Transport Layer

第00讲-课程简介_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1Ut411d7RE/?p=1&vd_source=5fbc4920b6c246ce2d9126818c78b17b







Computer Networking: A Top-Down Approach

9th edition

Jim Kurose, Keith Ross

Pearson, 2025

Transport layer: overview

Our goal:

- understand principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- learn about Internet transport layer protocols:
  - UDP: connectionless transport
  - TCP: connection-oriented reliable transport
  - TCP congestion control



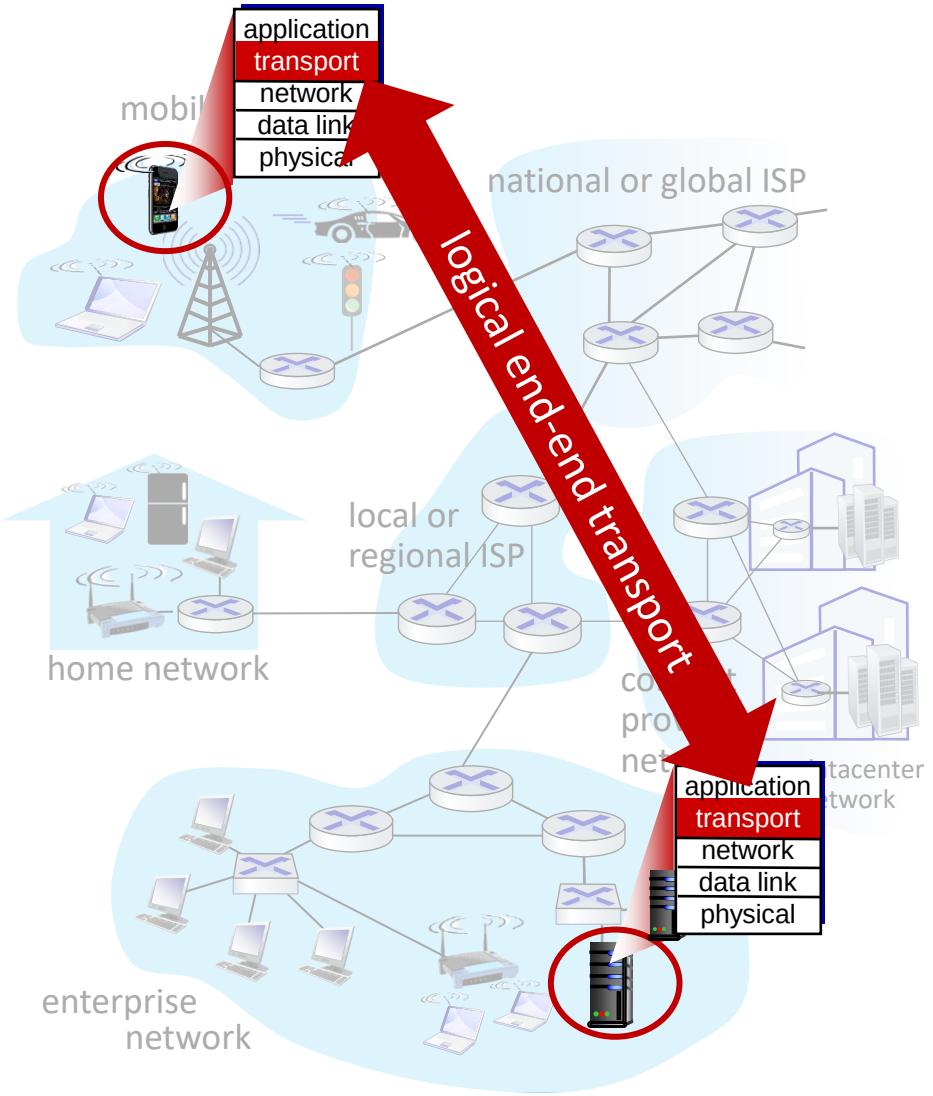
Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



Transport services and protocols

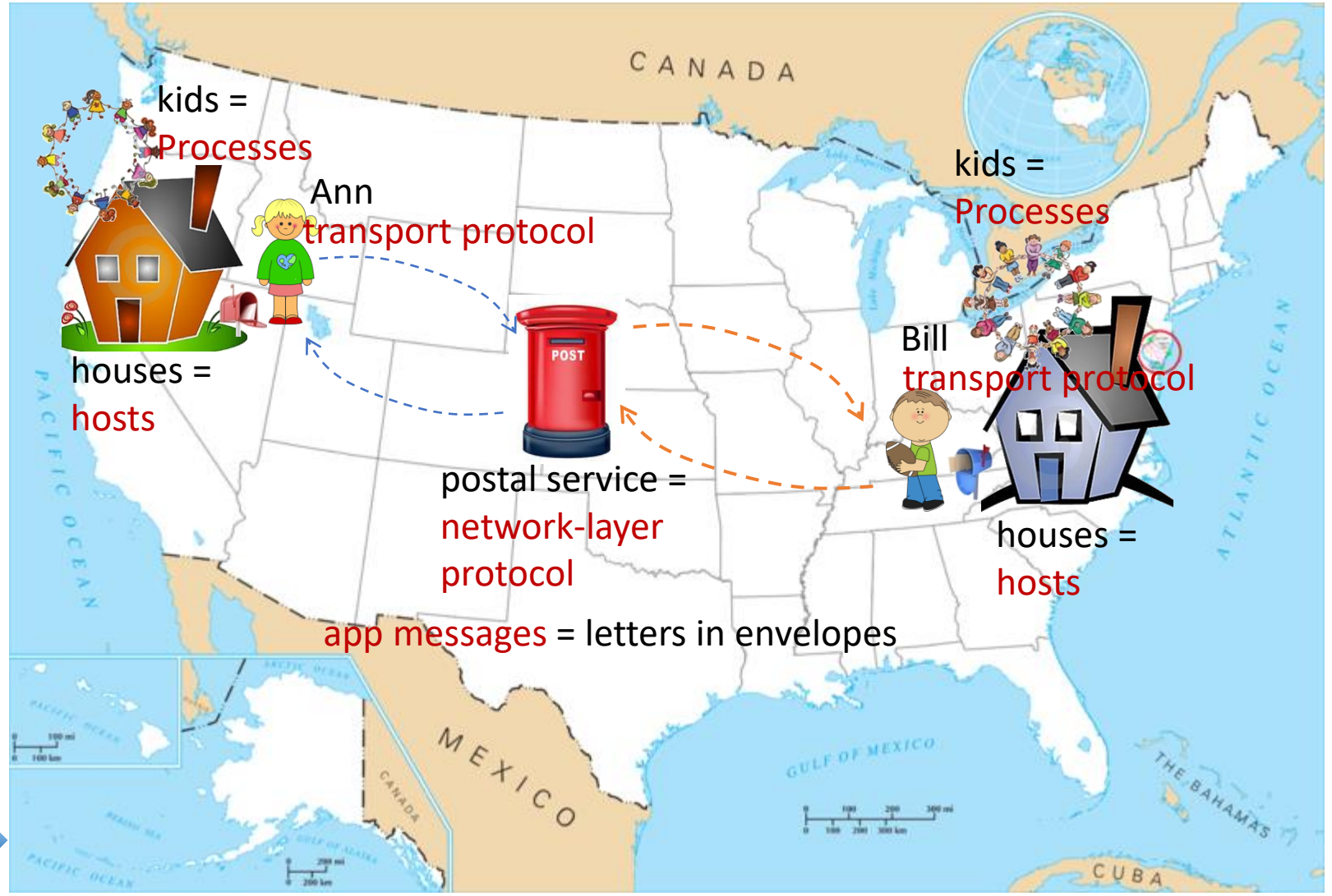
- provide *logical communication* between application processes running on different hosts
- transport protocols actions in end systems:
  - sender: breaks application messages into *segments*, passes to network layer
  - receiver: reassembles segments into messages, passes to application layer
- two transport protocols available to Internet applications
  - TCP, UDP



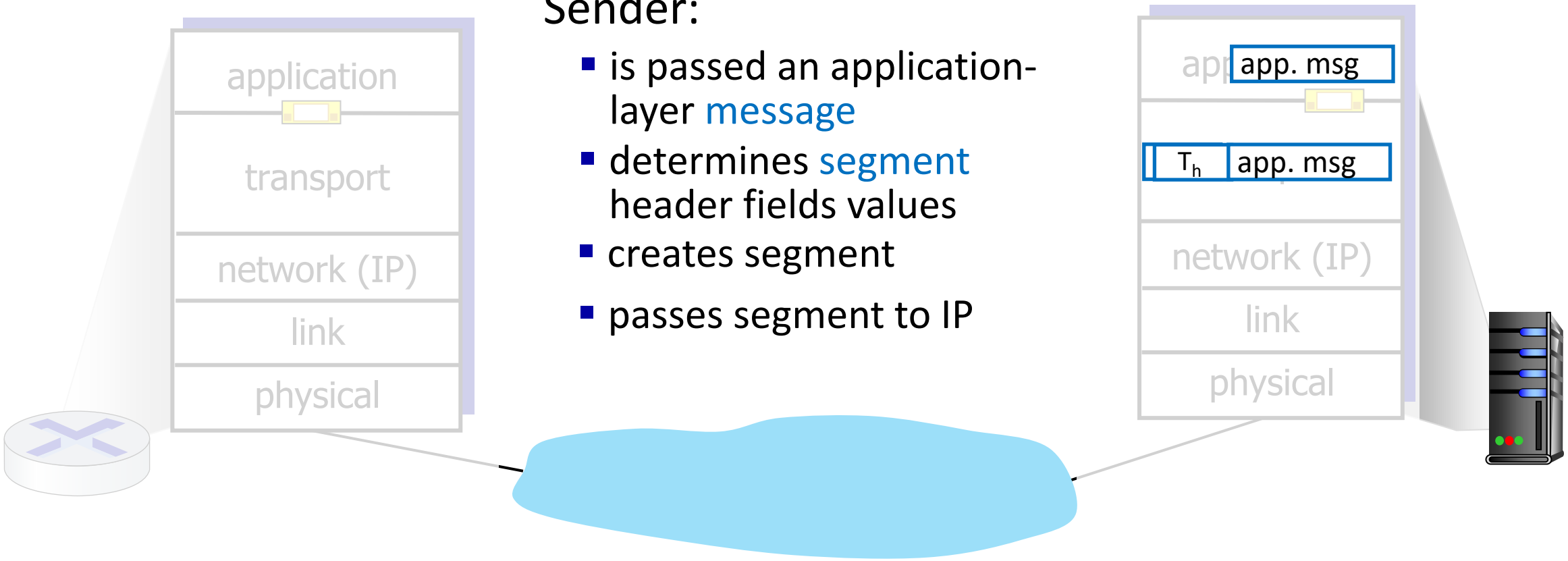
Transport vs. network layer services and protocols

- network layer: logical communication between *hosts*
- transport layer: logical communication between *processes*
  - relies on, enhances, network layer services

Household analogy



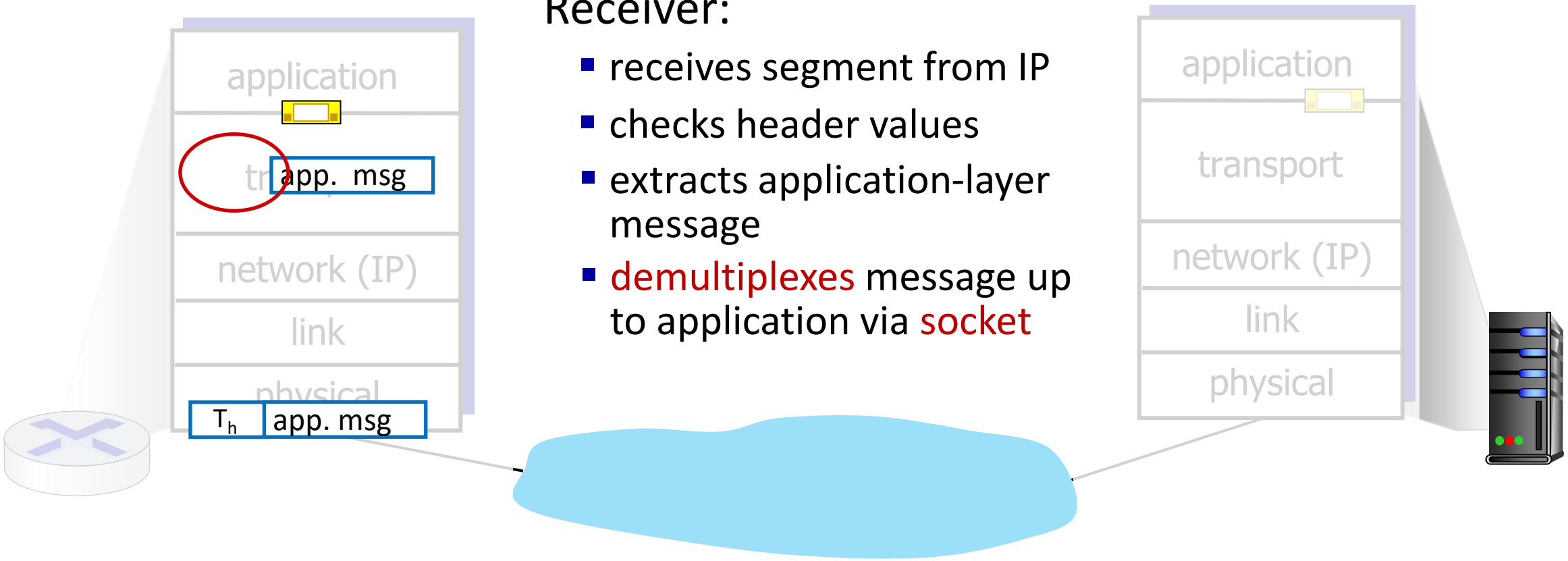
Transport Layer Actions



Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

Transport Layer Actions

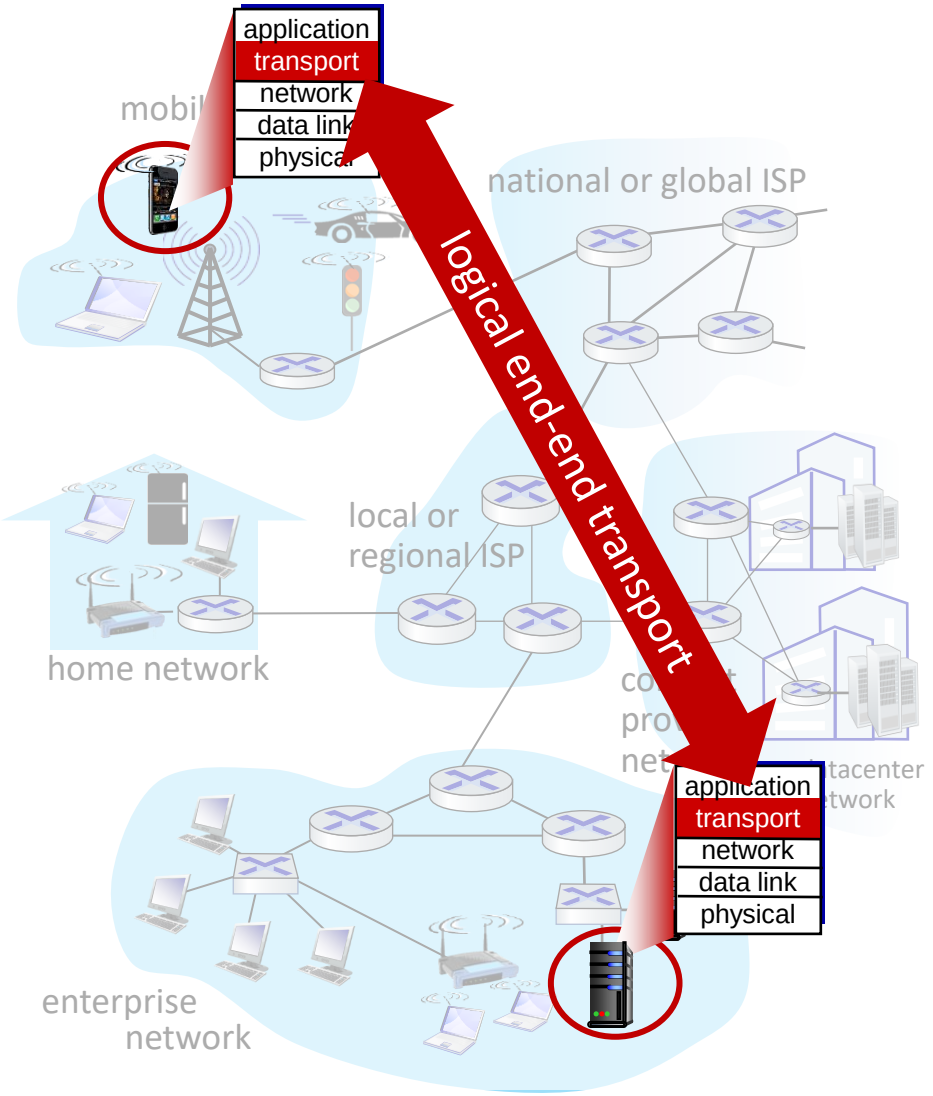


Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

Two principal Internet transport protocols

- **TCP:** Transmission Control Protocol
  - reliable, in-order delivery
  - congestion control
  - flow control
  - connection setup
- **UDP:** User Datagram Protocol
  - unreliable, unordered delivery
  - no-frills extension of “best-effort” IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



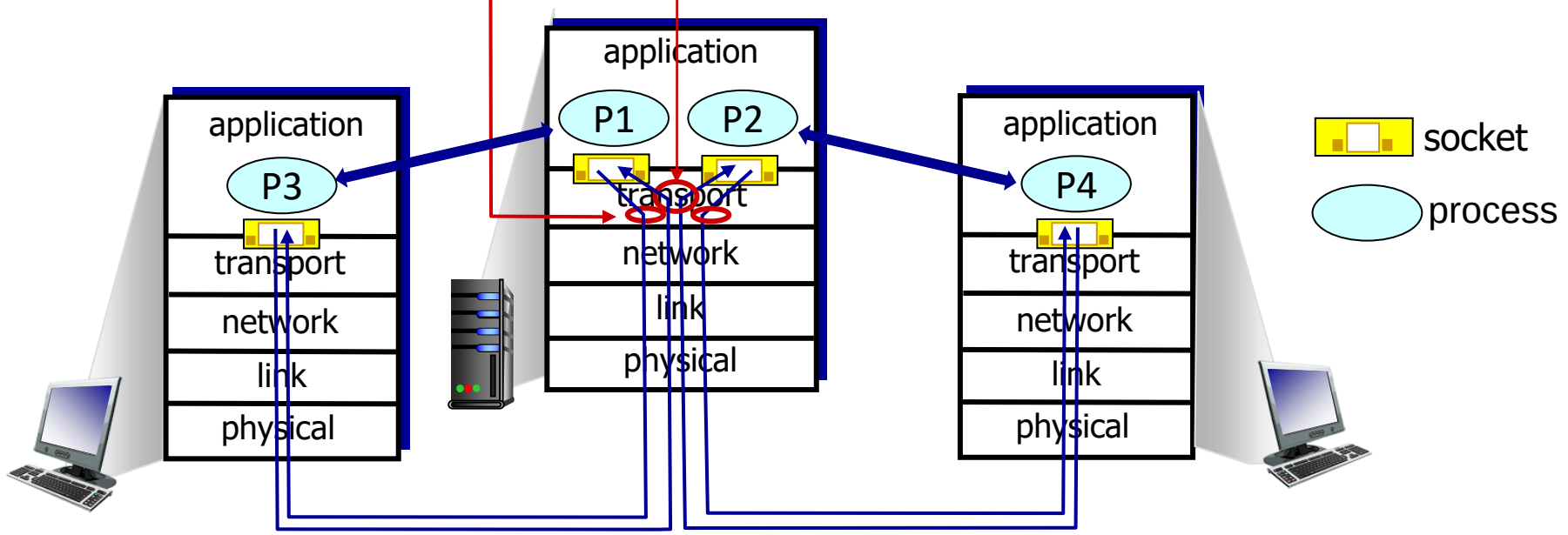
Transport-layer Multiplexing/demultiplexing

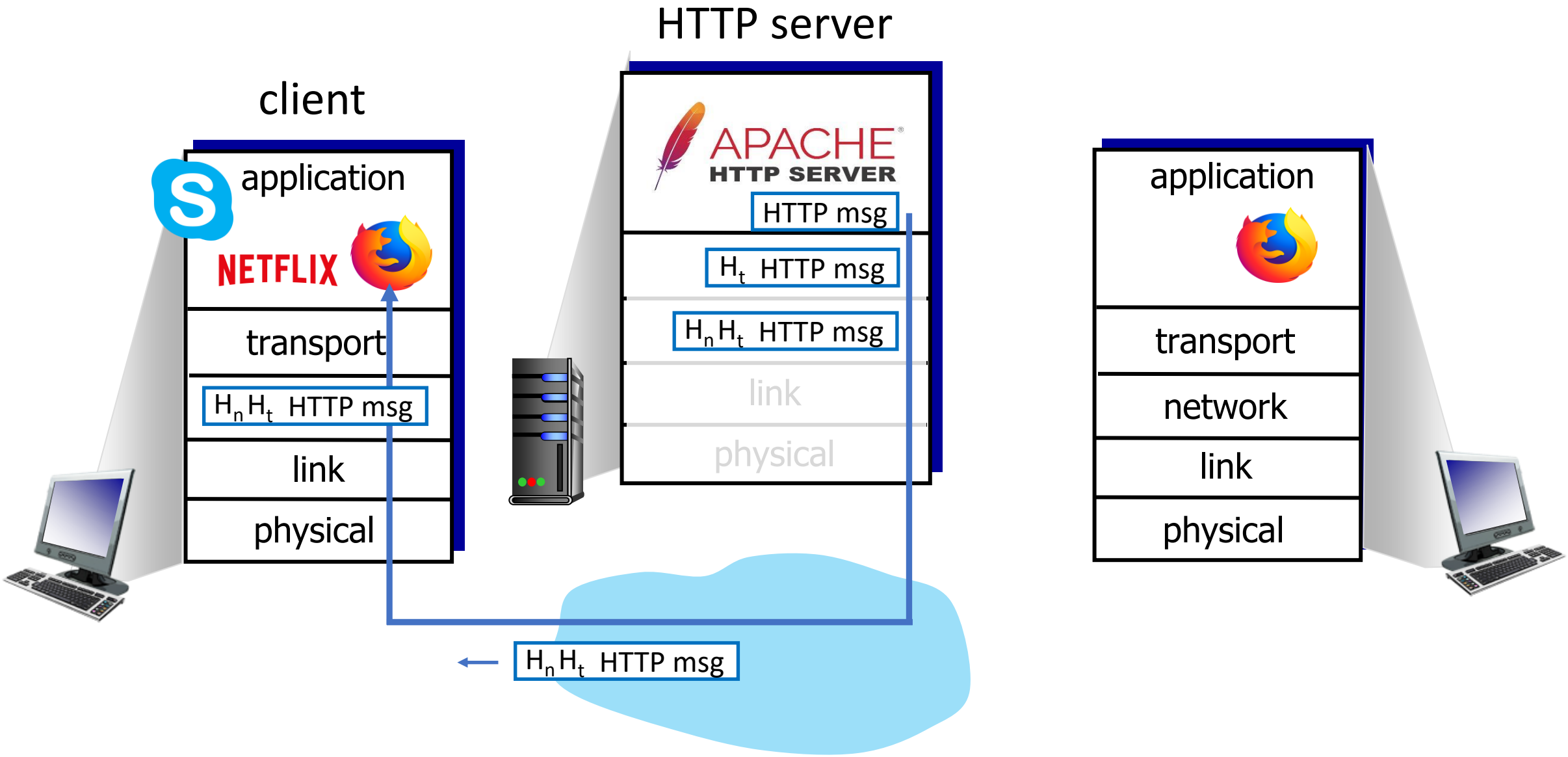
multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

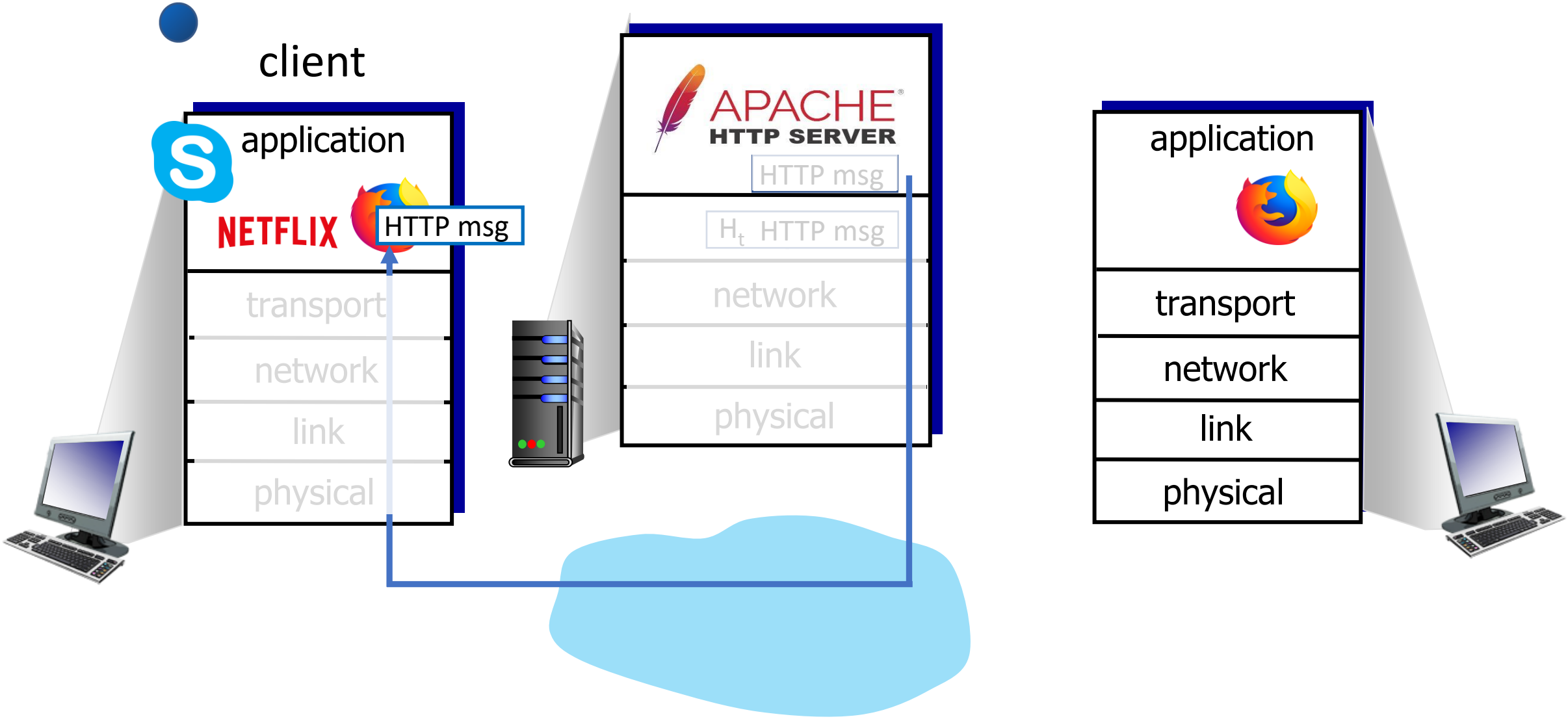
demultiplexing at receiver:

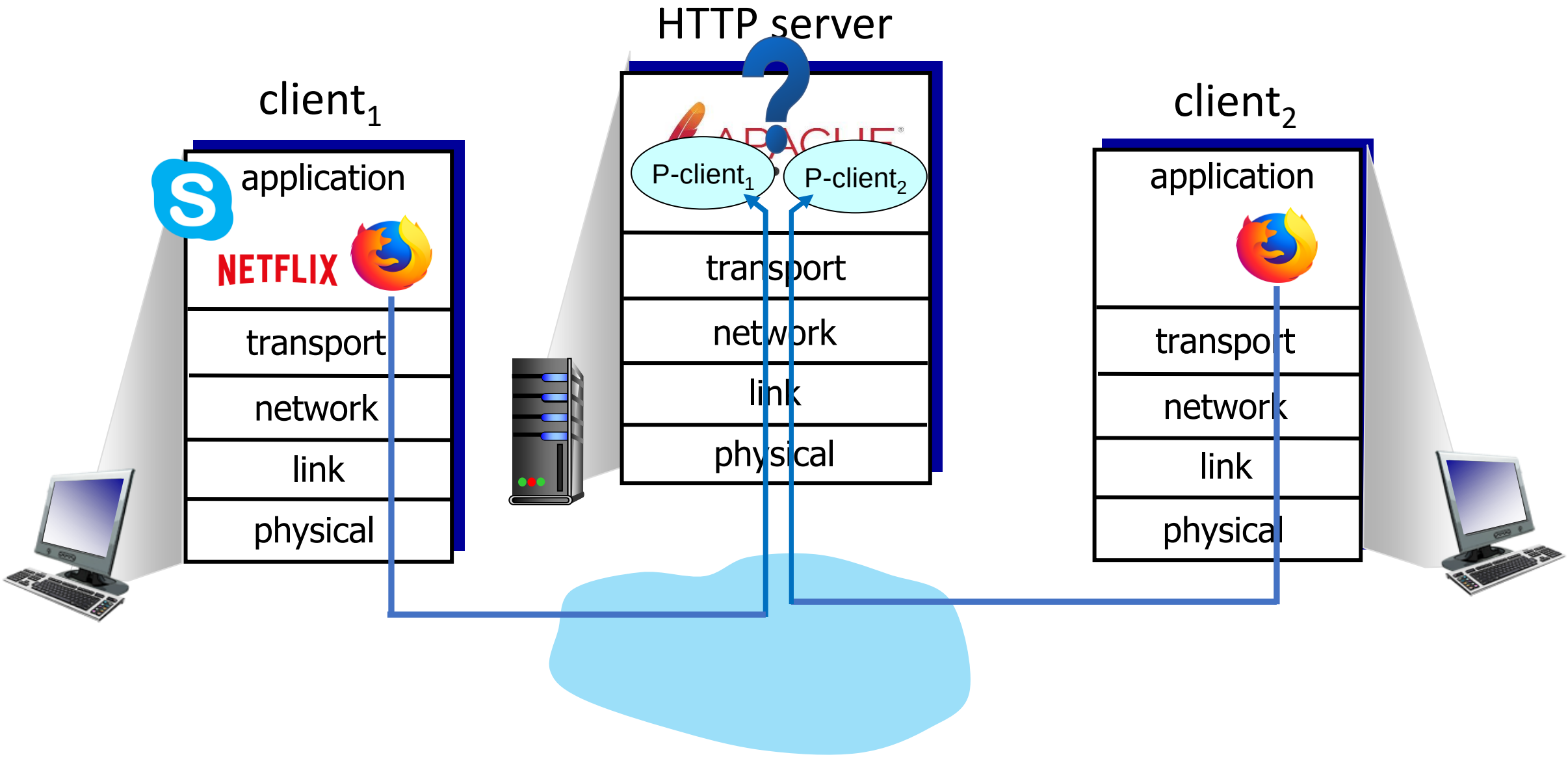
use header info to deliver received segments to correct socket



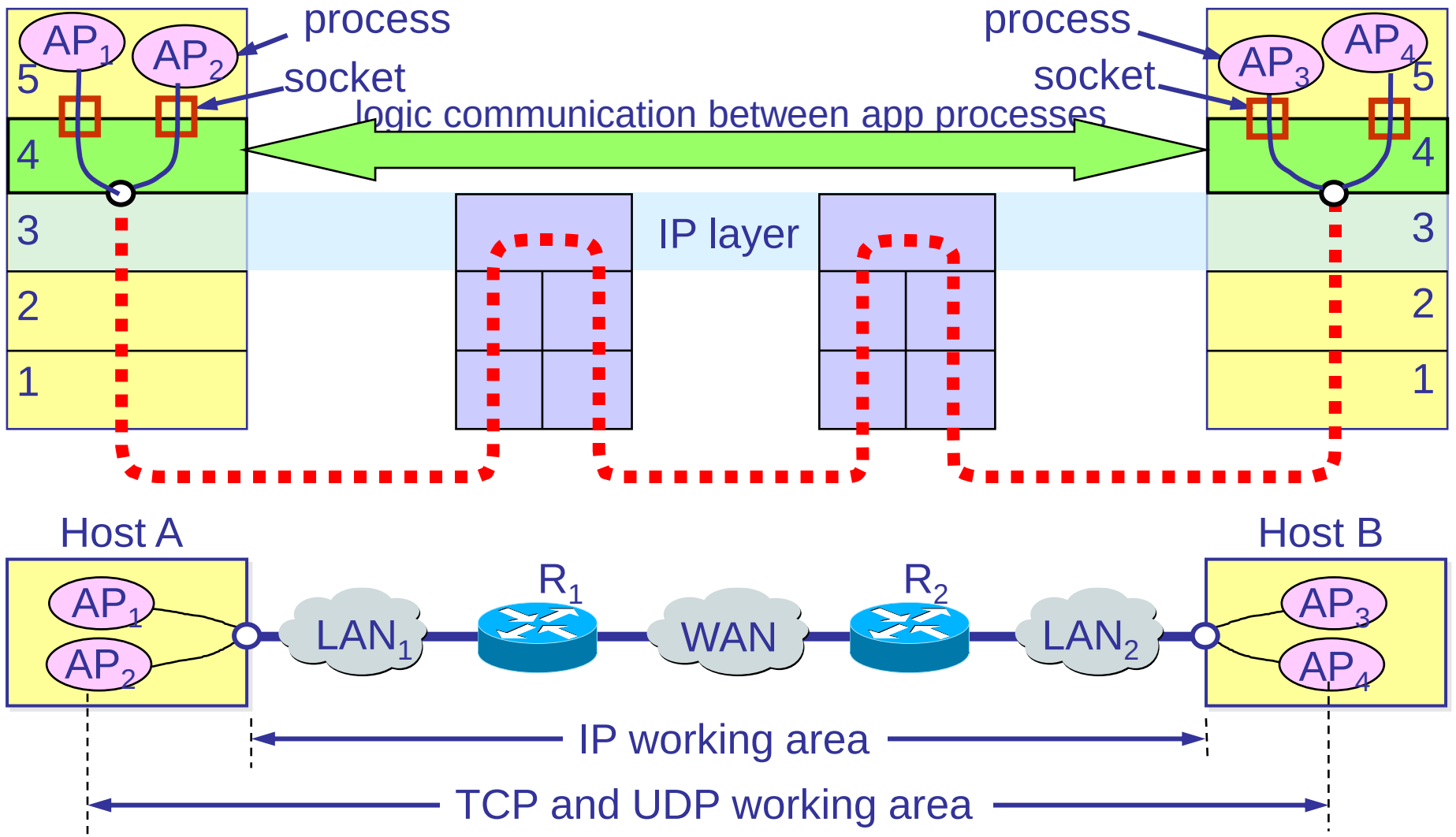

client
application
NETFLIX
transport
$H_n H_t$ HTTP msg
link
physical
HTTP server
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
$H_n H_t$ HTTP msg
link
physical
application
transport
network
link
physical
$H_n H_t$ HTTP msg

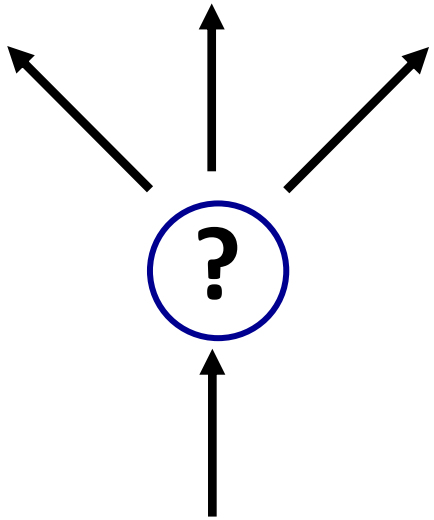
Q: how did transport layer know to deliver message to Firefox browser process rather then Netflix process?



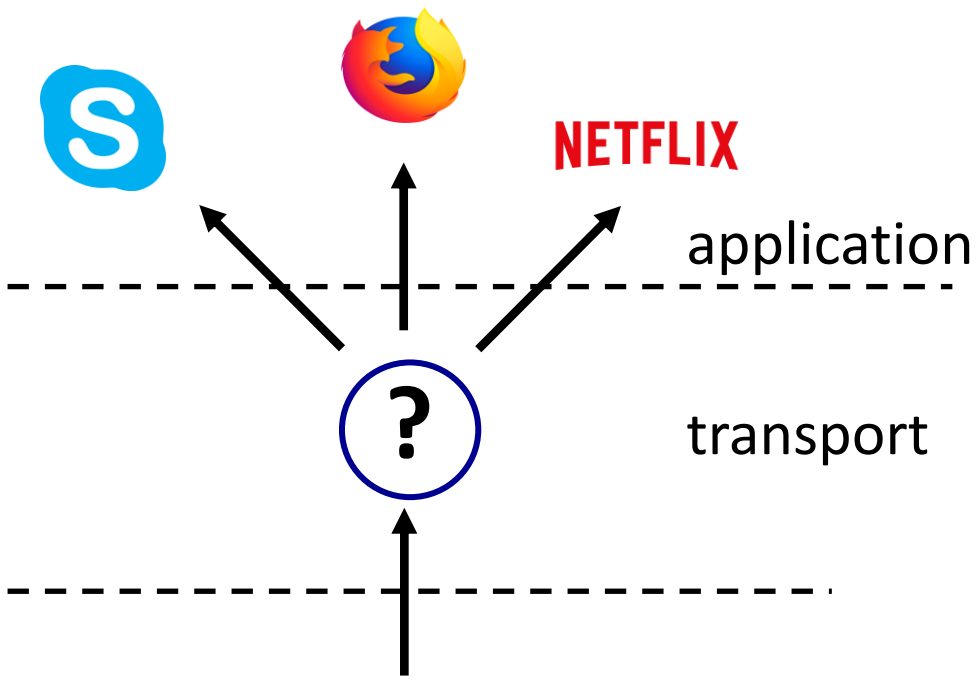

HTTP server
client1
application
NETFLIX
transport
network
link
physical
P-client1
P-client2
transport
network
link
physical
client2
application
transport
network
link
physical

Multiplexing/demultiplexing (2)





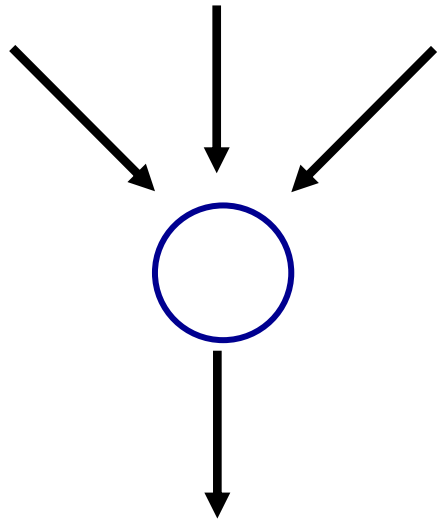
de-multiplexing



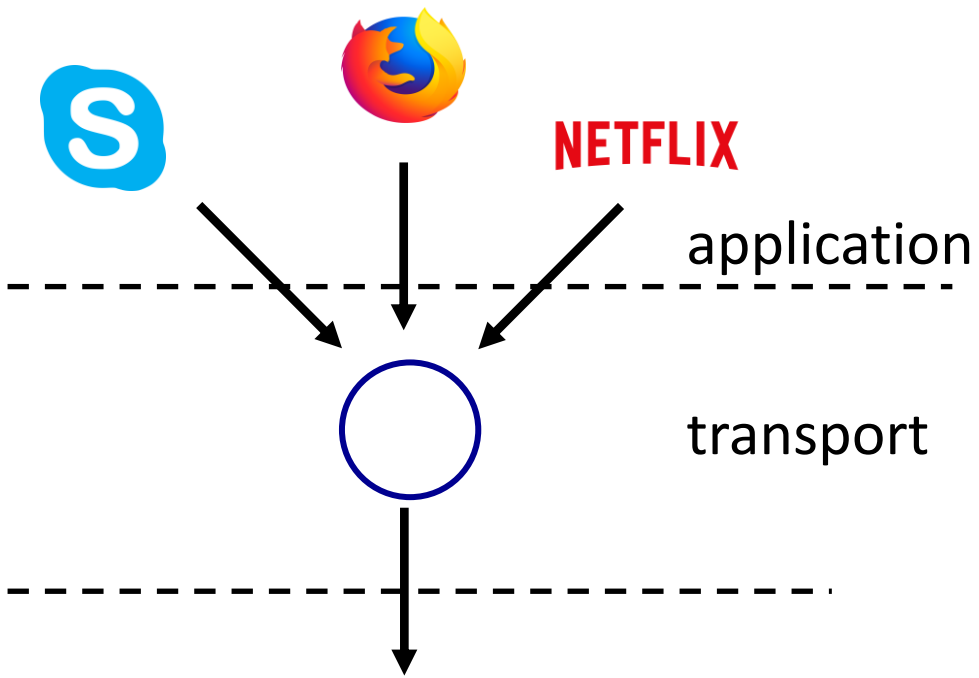
de-multiplexing



Demultiplexing



multiplexing



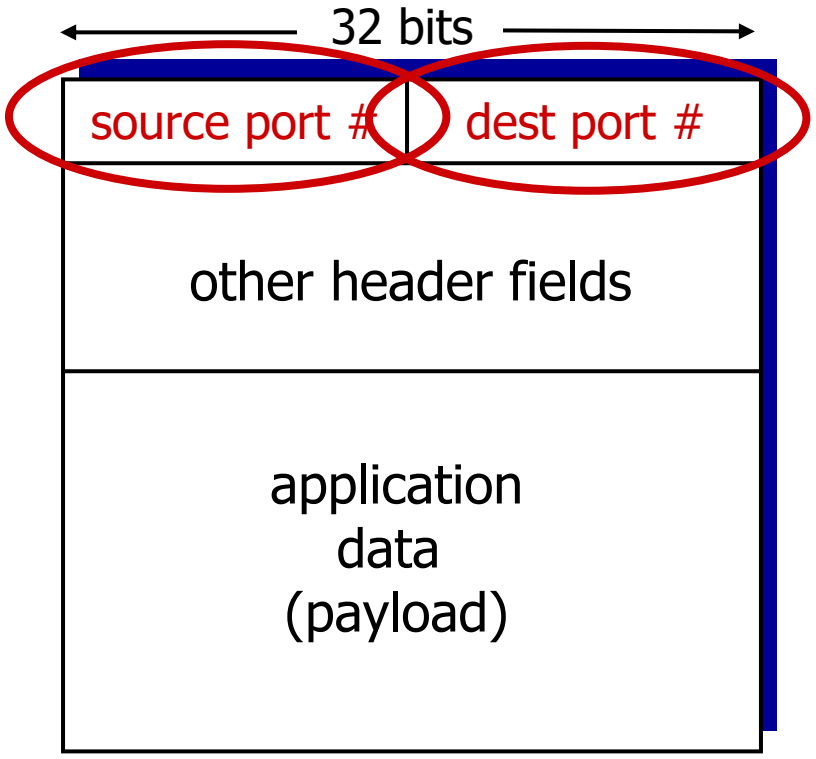
multiplexing


Multiplexing

How demultiplexing works

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket
- The port numbers ranging from 0 to 1023 are called *well-known port numbers*
  - HTTP → 80
  - FTP → 21

Each port number is a 16-bit number, ranging from 0 to 65535.



TCP/UDP segment format

Connectionless demultiplexing

Recall:

- when creating socket, must specify *host-local* port #:

```
DatagramSocket mySocket1
 = new DatagramSocket(12534);
```

- when creating datagram to send into UDP socket, must specify (2-tuple)
  - destination IP address
  - destination port #

Q:What is the purpose of the source port number in UDP?

when receiving host receives *UDP* segment:

- checks destination port # in segment
- directs UDP segment to socket with that port #



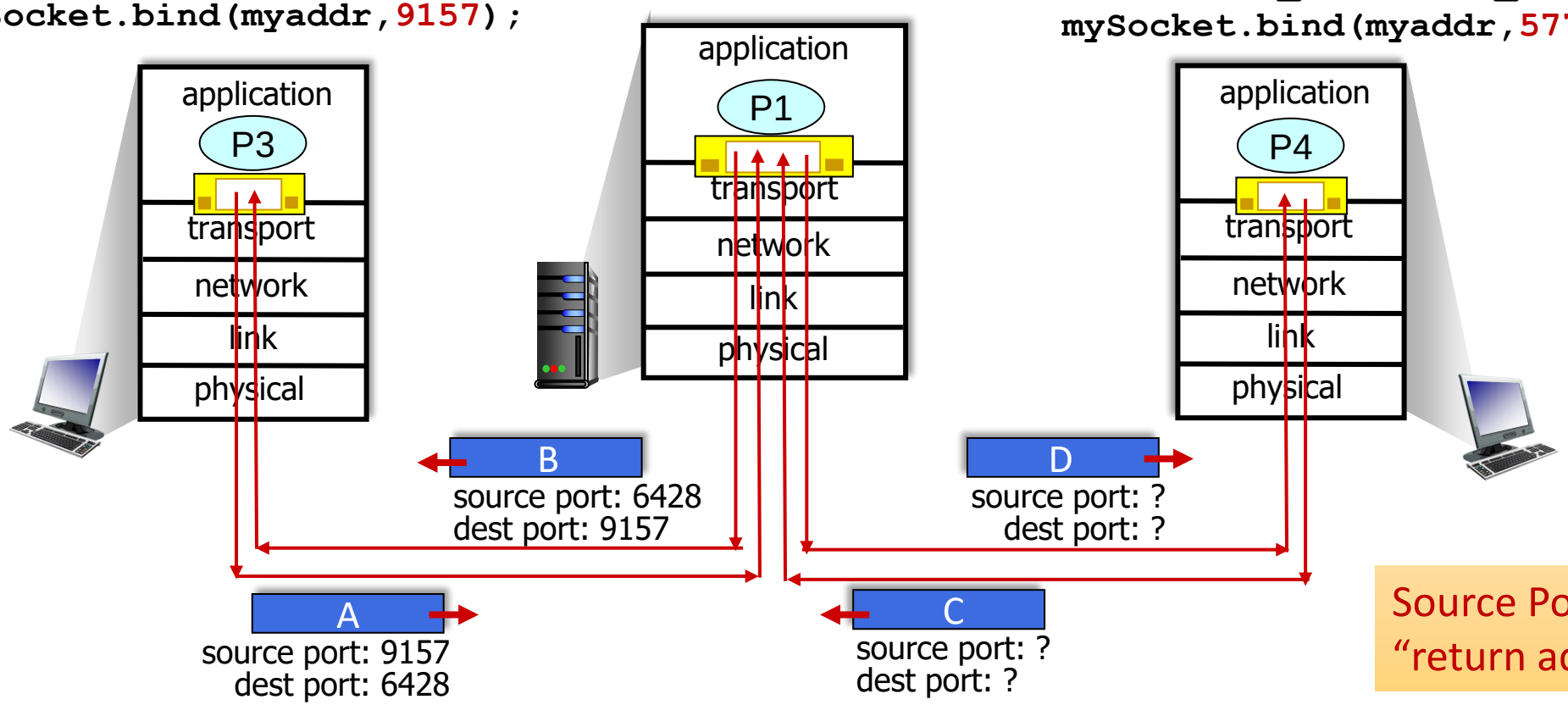
IP/UDP datagrams with *same dest. IP addr. AND dest. port #,* but different source IP addresses and/or source port # will be directed to *same socket* at receiving host

Connectionless demultiplexing: an example

```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,6428);
```

```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,9157);
```

```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,5775);
```

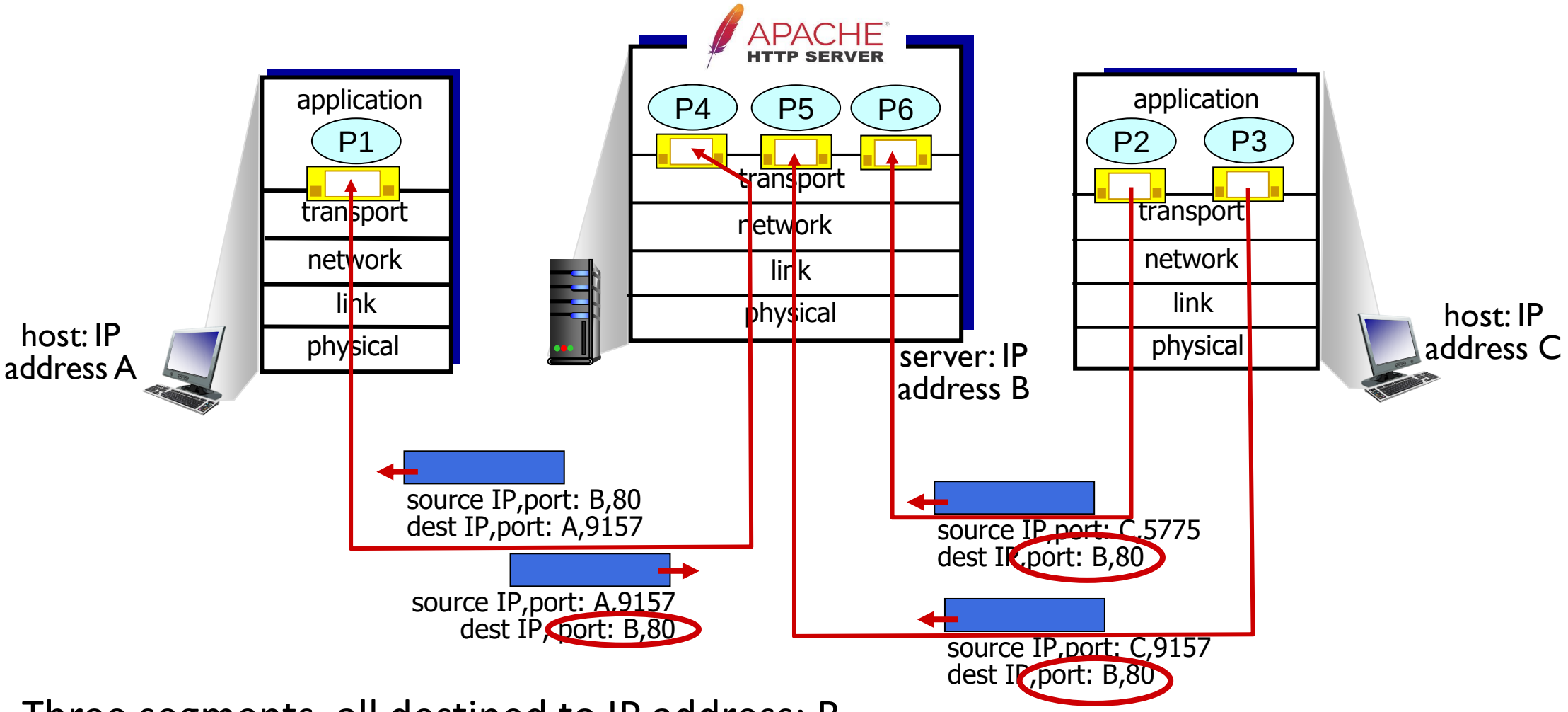


Source Port provides "return address"

Connection-oriented demultiplexing

- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest. IP address
  - dest. port number
- demux: receiver uses *all four values (4-tuple)* to direct segment
  - Two arriving TCP segments with different source IP addresses or source port numbers will be directed to two different sockets.
- server may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
  - each socket associated with a different connecting client
- Web servers have different sockets for each connecting client
  - non-persistent HTTP will have different socket for each request

Connection-oriented demultiplexing: example



Three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Section Summary

- Multiplexing, demultiplexing: based on segment, datagram header field values
- **UDP:** demultiplexing using 2-tuple: destination IP address and port number (only)
- **TCP:** demultiplexing using 4-tuple: source and destination IP addresses, and port numbers
- Multiplexing/demultiplexing happen at *all* layers
  - whenever a single protocol at one layer (at the transport layer or elsewhere) is used by multiple protocols at the next higher layer.

Chapter 3: roadmap

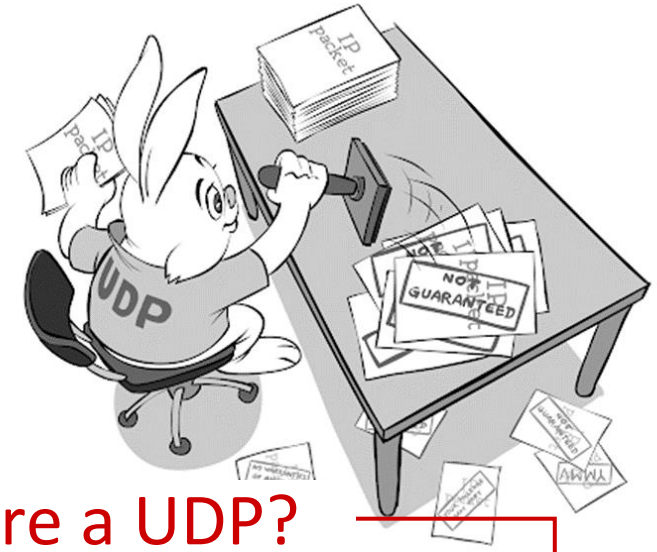
- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Secure remote terminal access	SSH	TCP
Web	HTTP, HTTP/3	TCP (for HTTP), UDP (for HTTP/3)
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	DASH	TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

Figure 3.6 ♦ Popular Internet applications and their underlying transport protocols

UDP: User Datagram Protocol



- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- *connectionless:*
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others

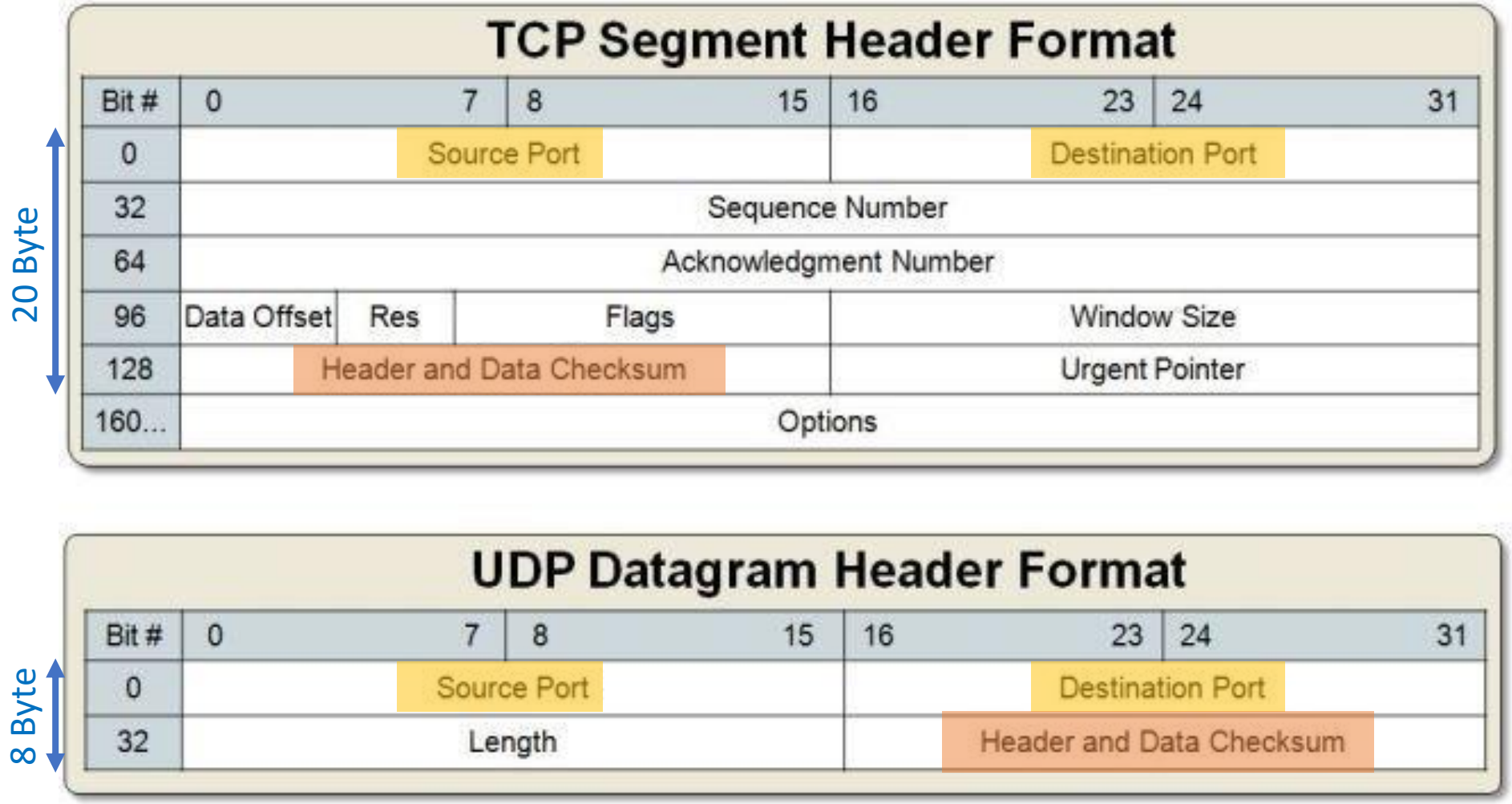
Why is there a UDP?

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion



TCP and UDP Header

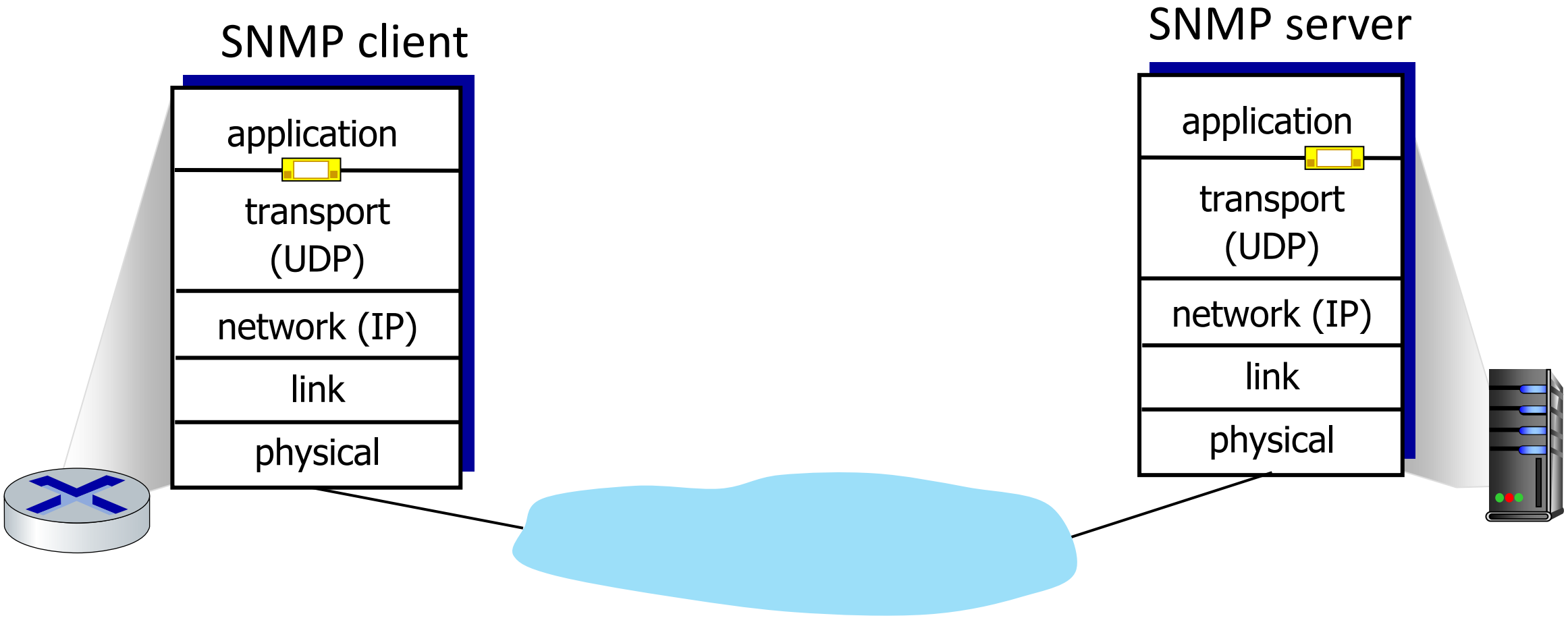
The two minimal transport-layer services—**process-to-process data delivery and error checking**—are the only two services that UDP provides!



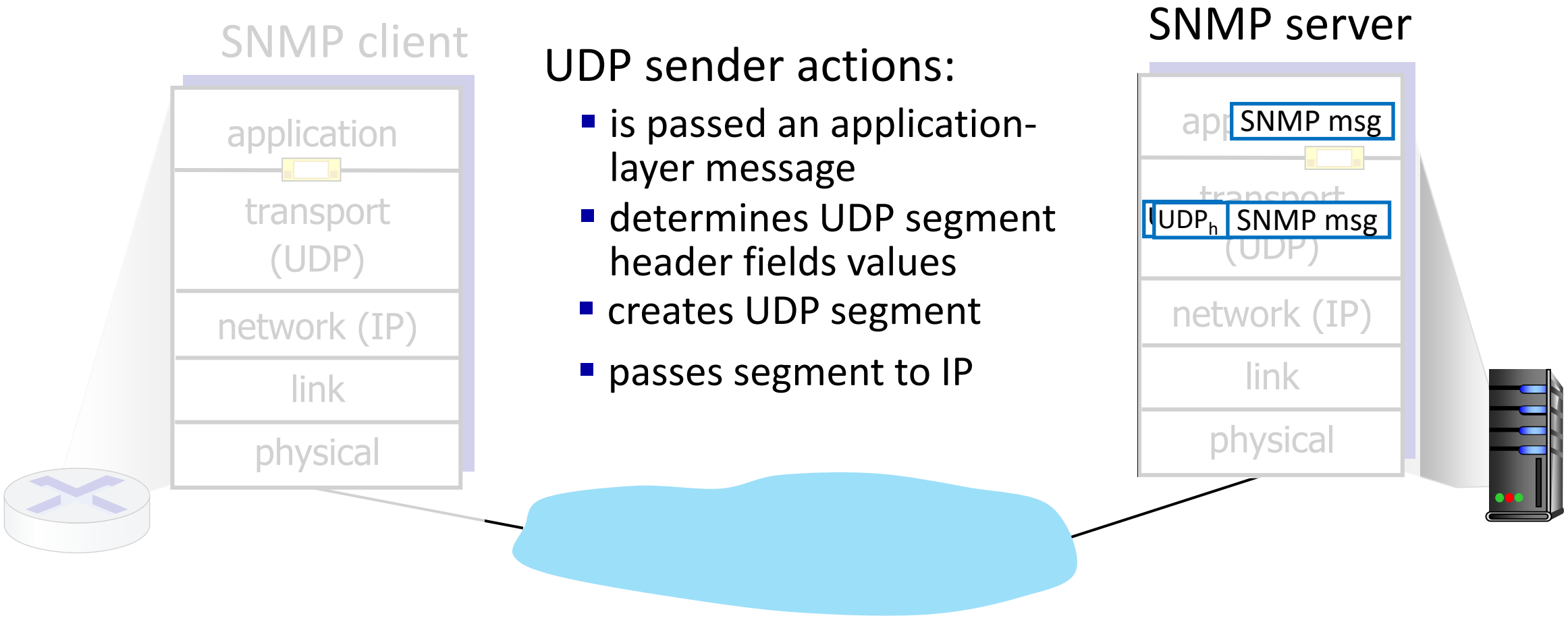
UDP: User Datagram Protocol

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
  - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
  - add needed reliability at application layer
  - add congestion control at application layer

UDP: Transport Layer Actions



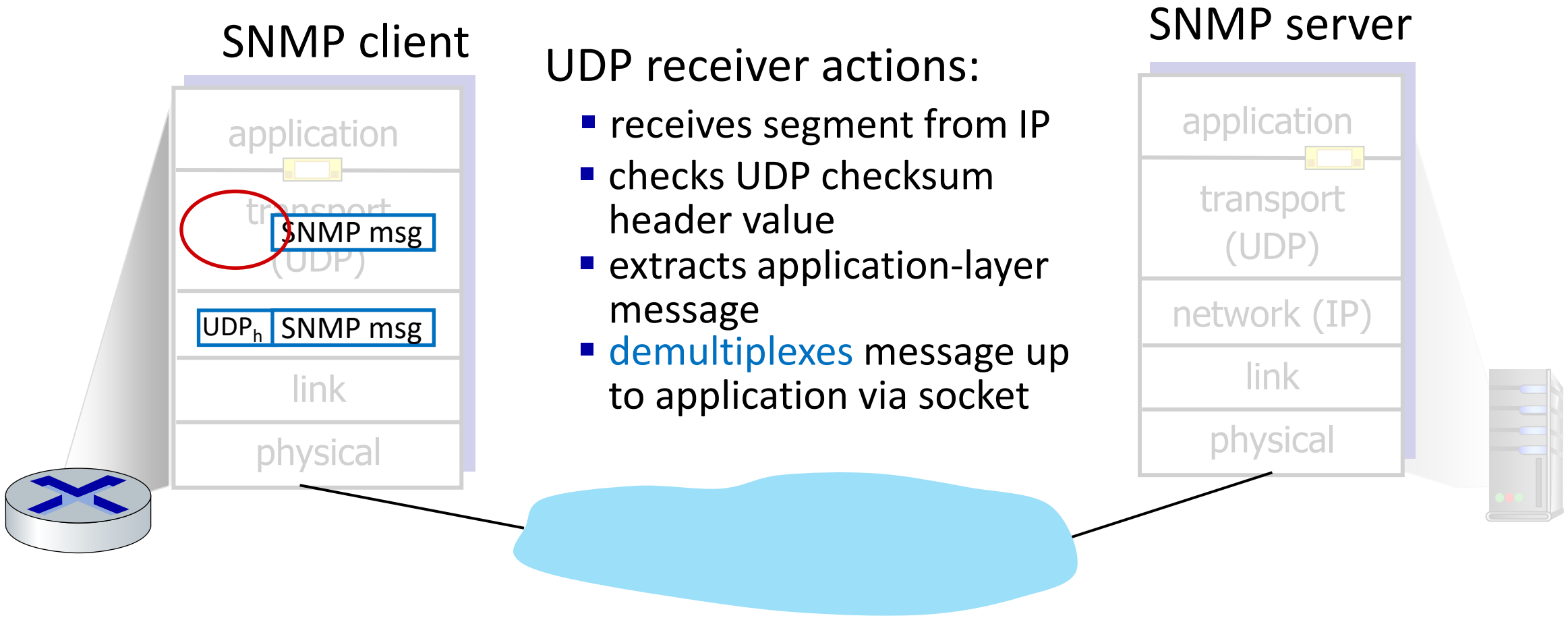
UDP: Transport Layer Actions



UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

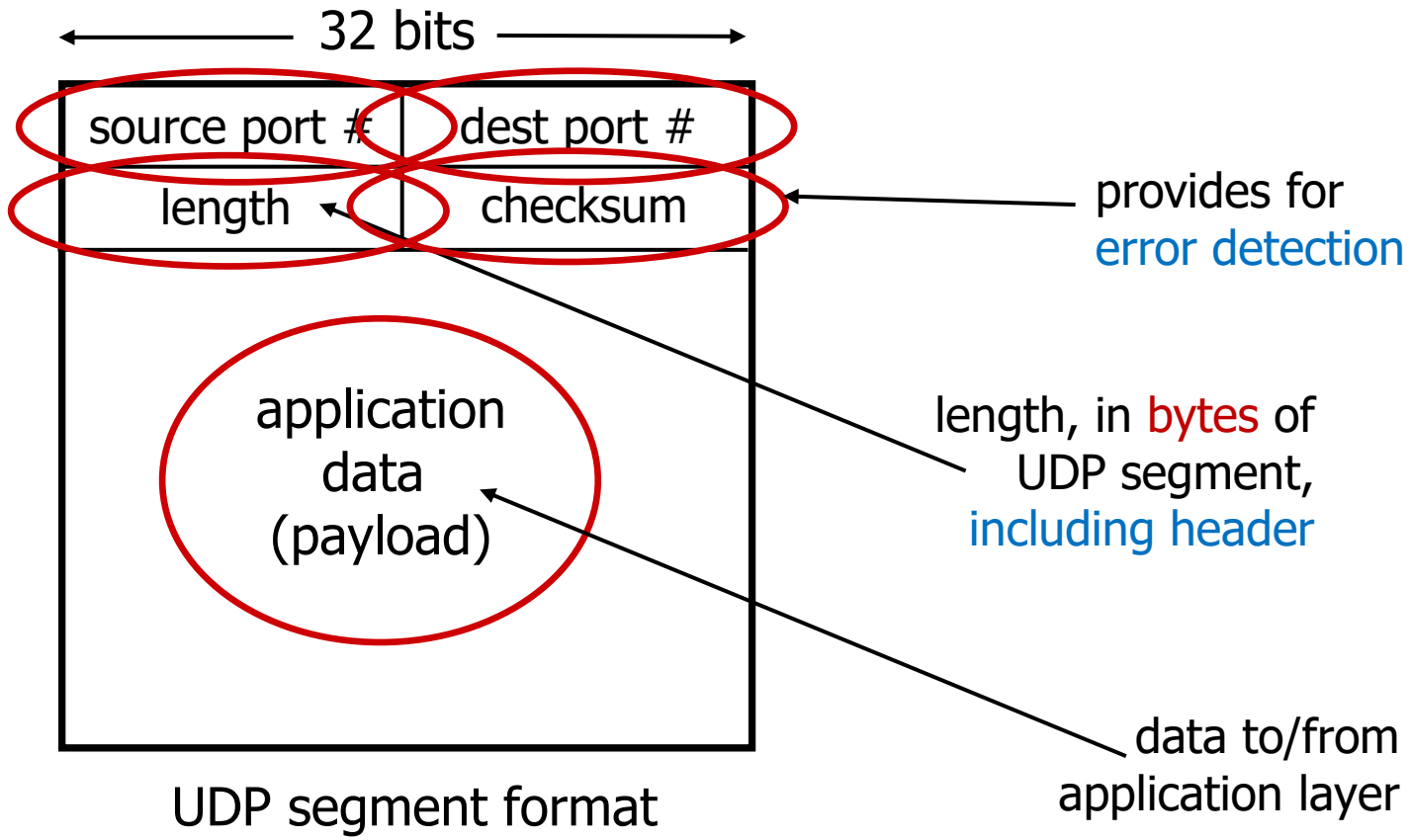
UDP: Transport Layer Actions



UDP receiver actions:

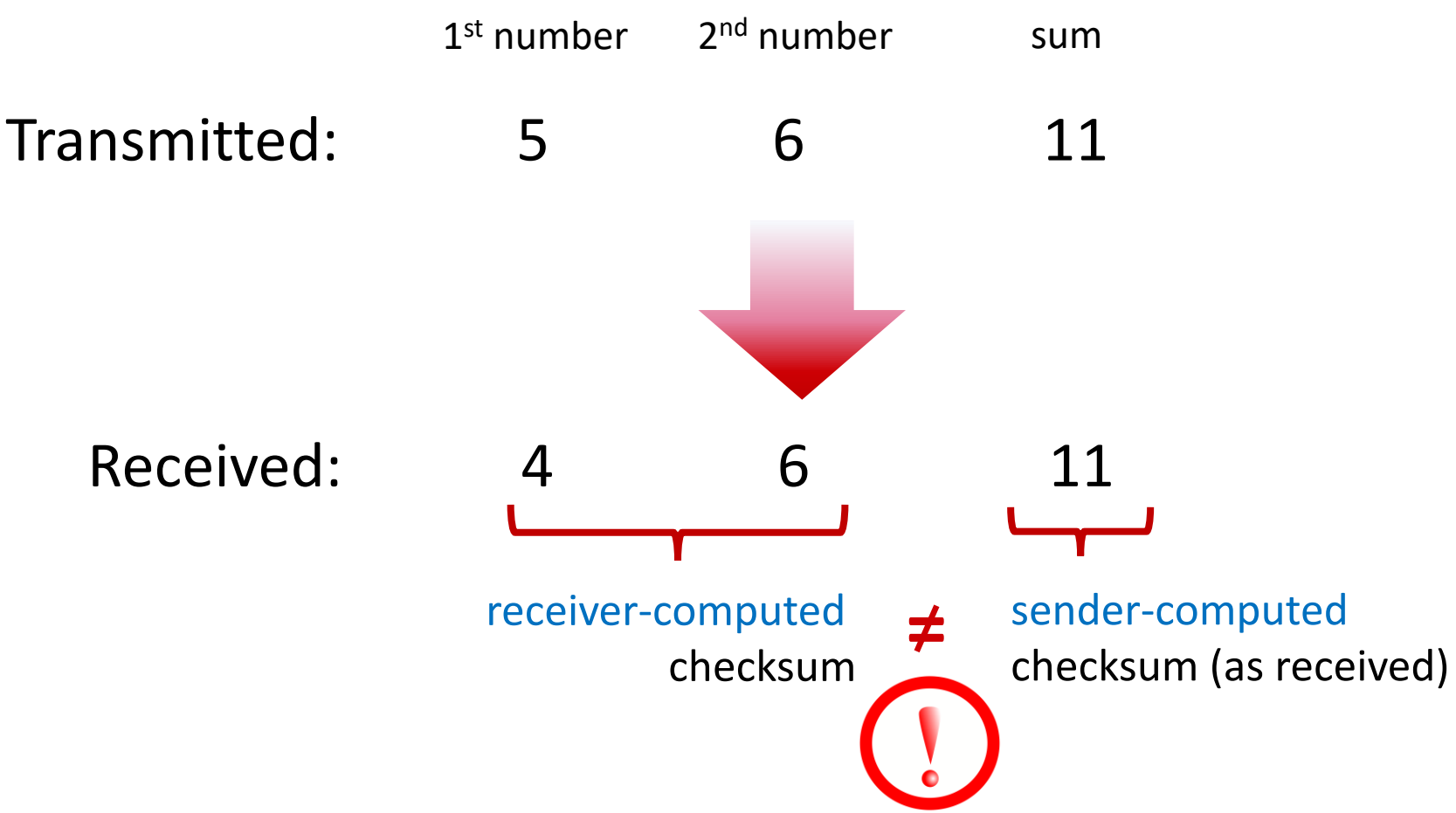
- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

UDP segment header



UDP checksum

Goal: detect errors (*i.e.,* flipped bits) in transmitted segment



UDP checksum

Goal: detect errors (*i.e.,* flipped bits) in transmitted segment

sender:

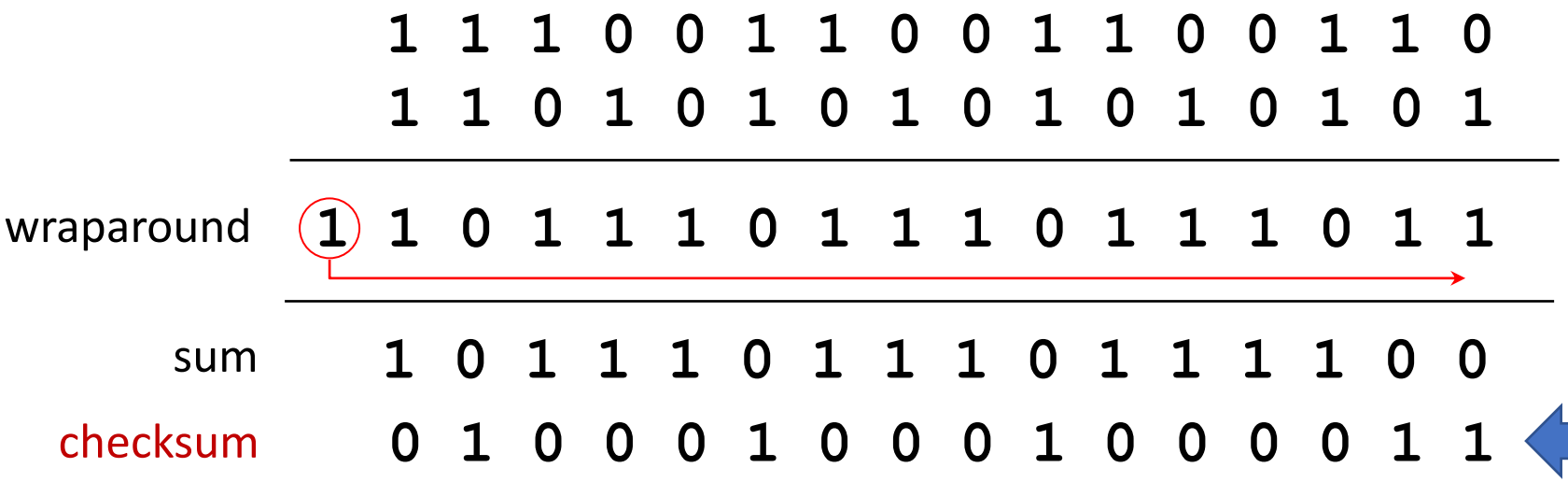
- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - Not equal - error detected
  - Equal - no error detected. *But maybe errors nonetheless?* More later ….

Internet checksum: an example

example: add two 16-bit integers



Note:

when adding numbers, a carryout from the most significant bit (MSB) needs to be added to the result.

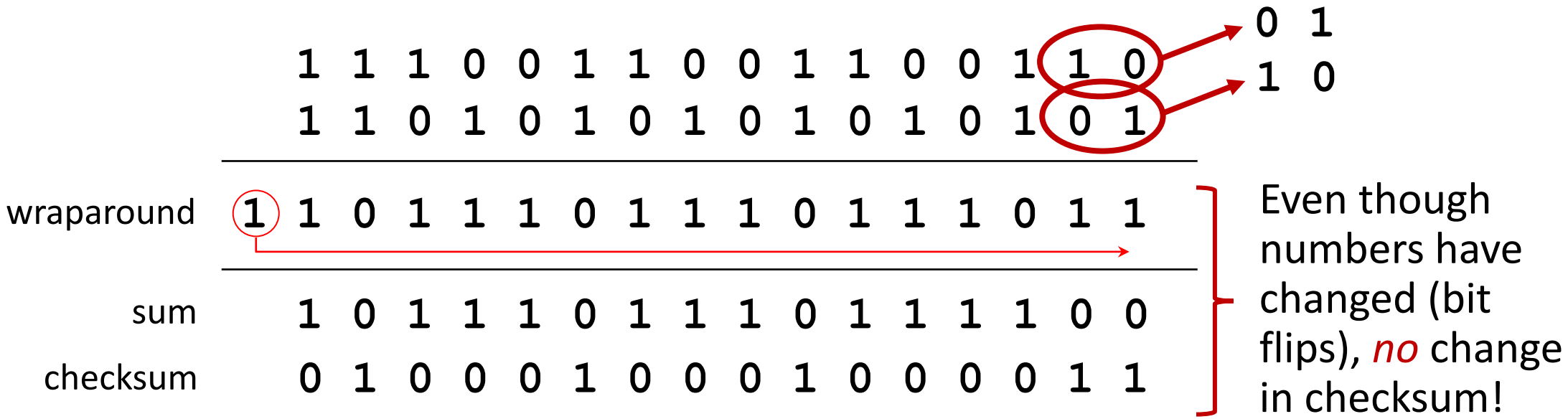
UDP at the sender side performs the 1's complement of the sum

At the receiver, all 16-bit words are added, including the checksum.

- If no errors, sum at the receiver will be 1111111111111111 (16-bits all 1s).
- If one of the bits is a 0, then we know that errors have been introduced into the packet.

Internet checksum: weak protection!

example: add two 16-bit integers



Section Summary: UDP

- "no frills" protocol:
  - segments may be lost, delivered out of order
  - best effort service: "send and hope for the best"
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised
  - helps with reliability (checksum)



- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



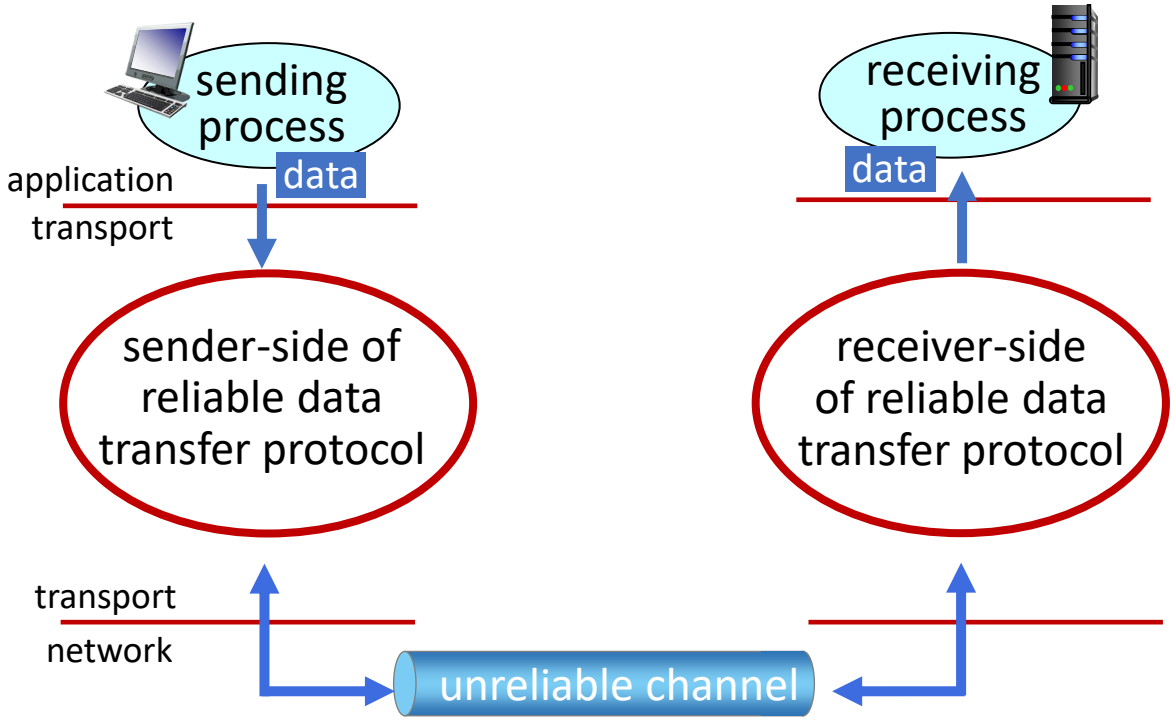
Principles of reliable data transfer



reliable service *abstraction*

Principles of reliable data transfer

reliable service *abstraction*

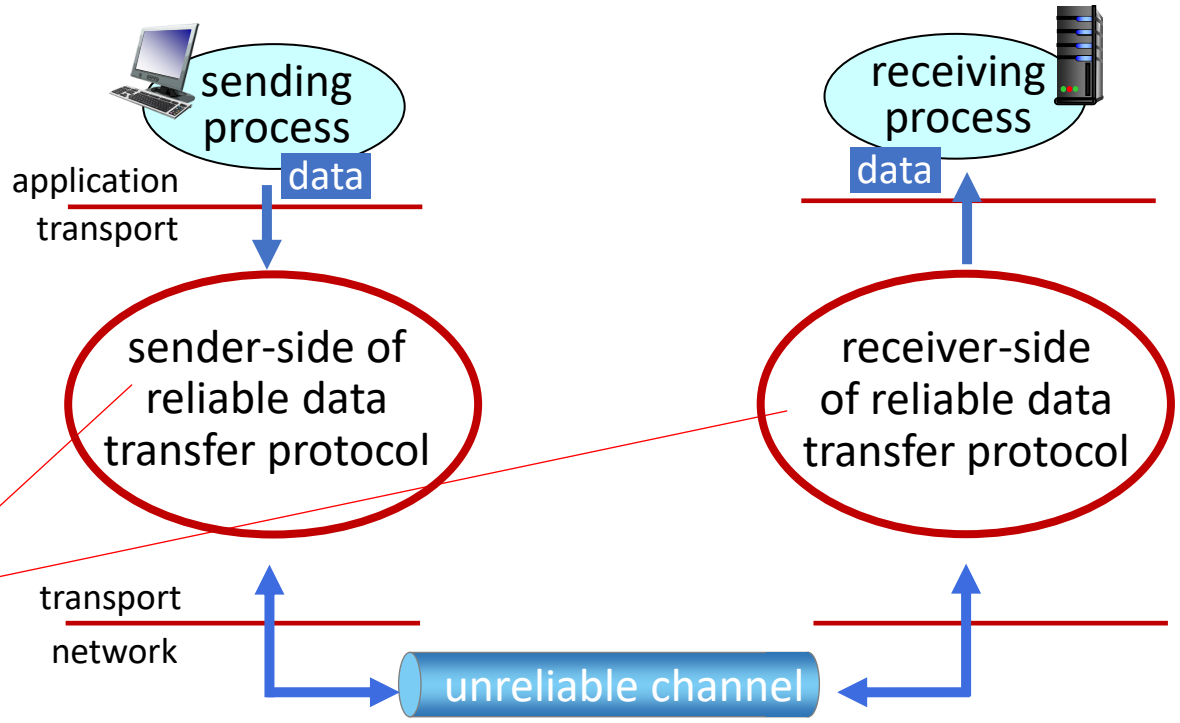


reliable service *implementation*

Principles of reliable data transfer

How much work they'll have to do depends on the IMPAIRMENTS introduced by channel
– if the channel is perfect – no problem!

Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel (lose, corrupt, reorder data?)

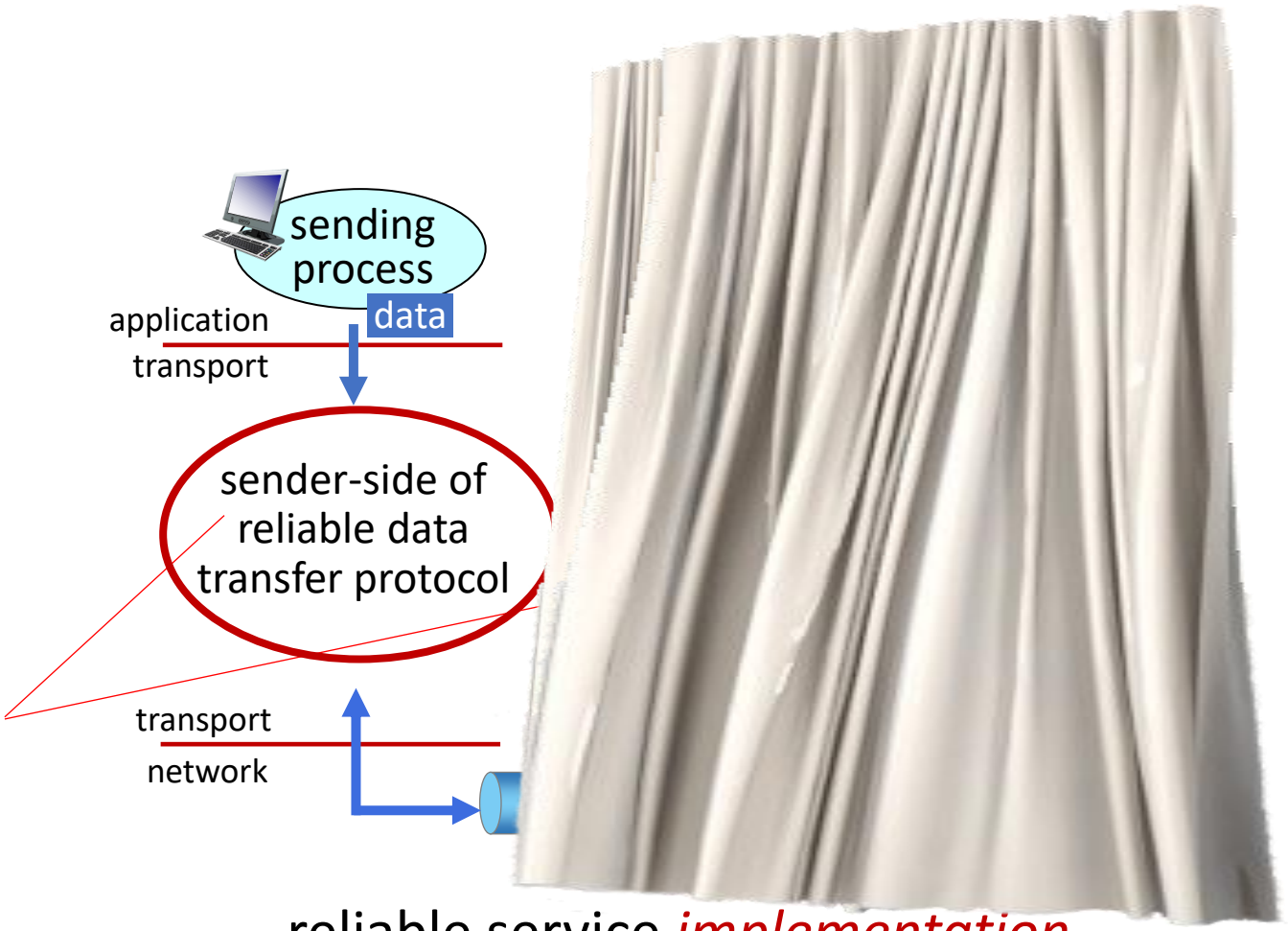


reliable service *implementation*

Principles of reliable data transfer

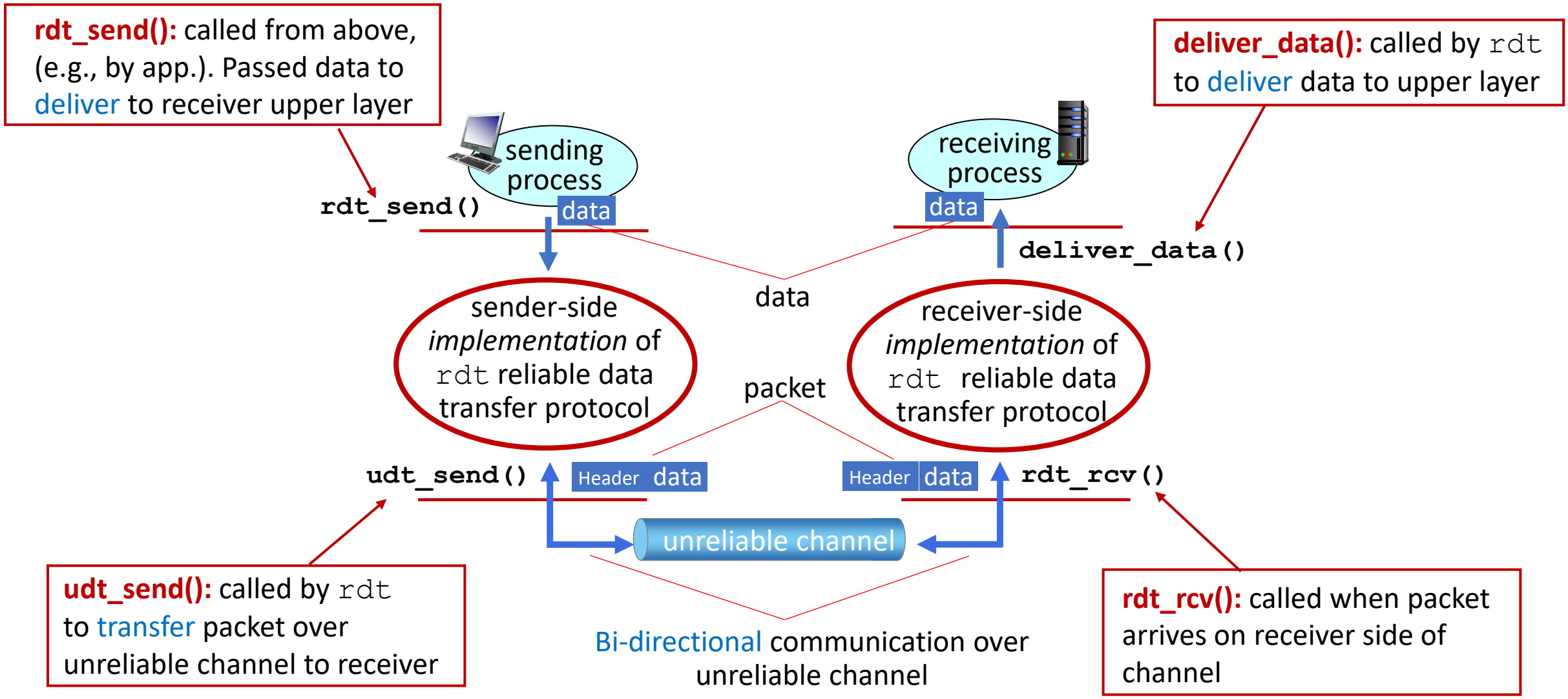
Sender, receiver do *not* know the "state" of each other, e.g., was a message received?

- unless communicated via a message



reliable service *implementation*

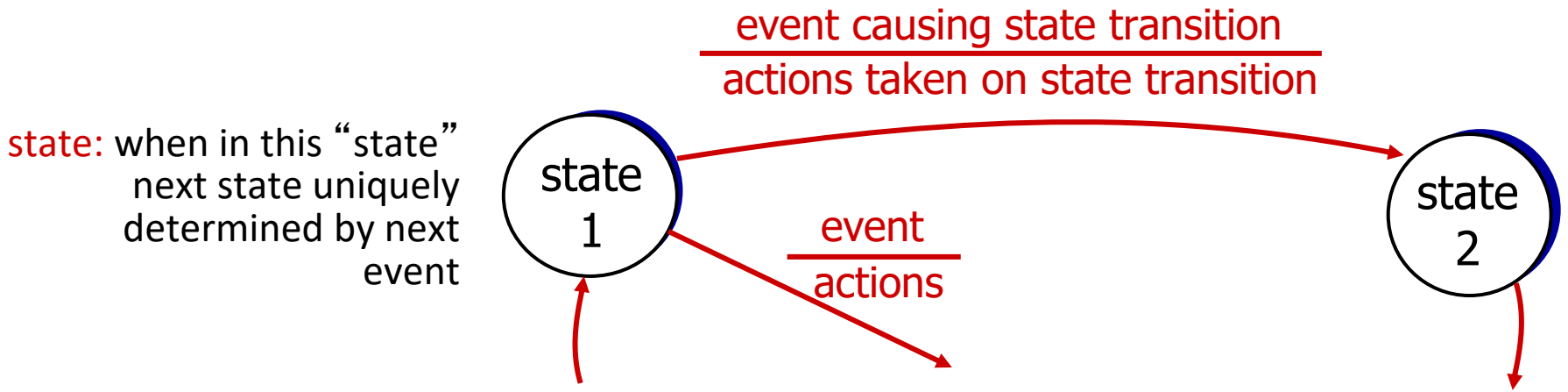
Reliable data transfer protocol (rdt): interfaces



Reliable data transfer: getting started

We will:

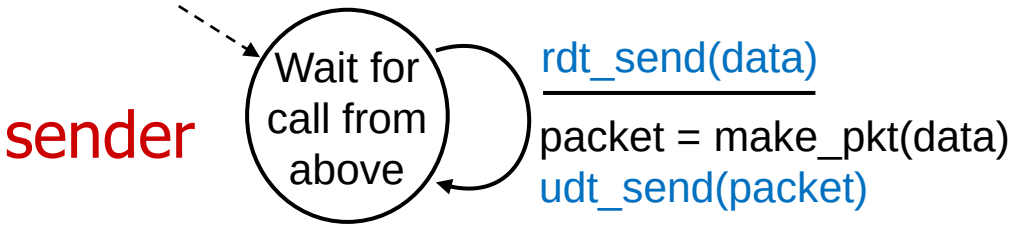
- incrementally develop sender, receiver sides of reliable data transfer protocol (`rdt`)
- consider only unidirectional data transfer
  - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver

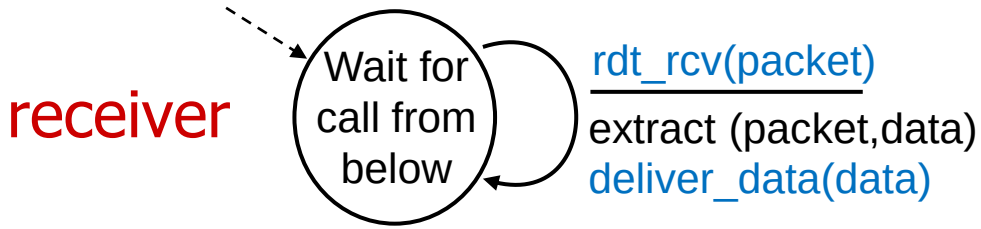


rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- *separate* FSMs for sender, receiver:
  - sender sends data into underlying channel
  - receiver reads data from underlying channel







rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum (e.g., Internet checksum) to detect bit errors
- *the* question: how to recover from errors?

How do humans recover from "errors" during conversation?

rdt2.0: channel with bit errors

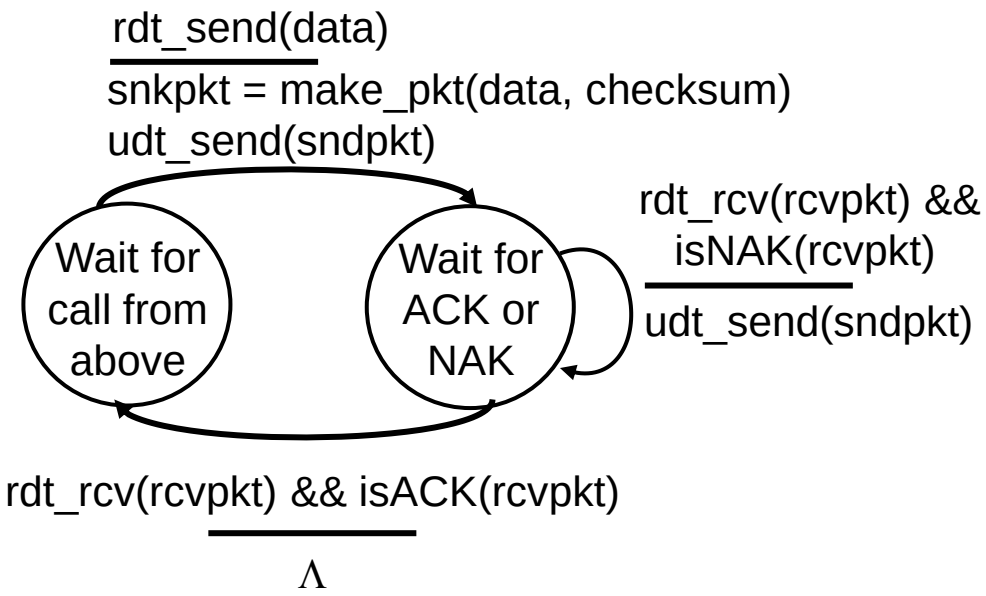
- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors?
  - *acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - sender *retransmits* pkt on receipt of NAK

stop and wait

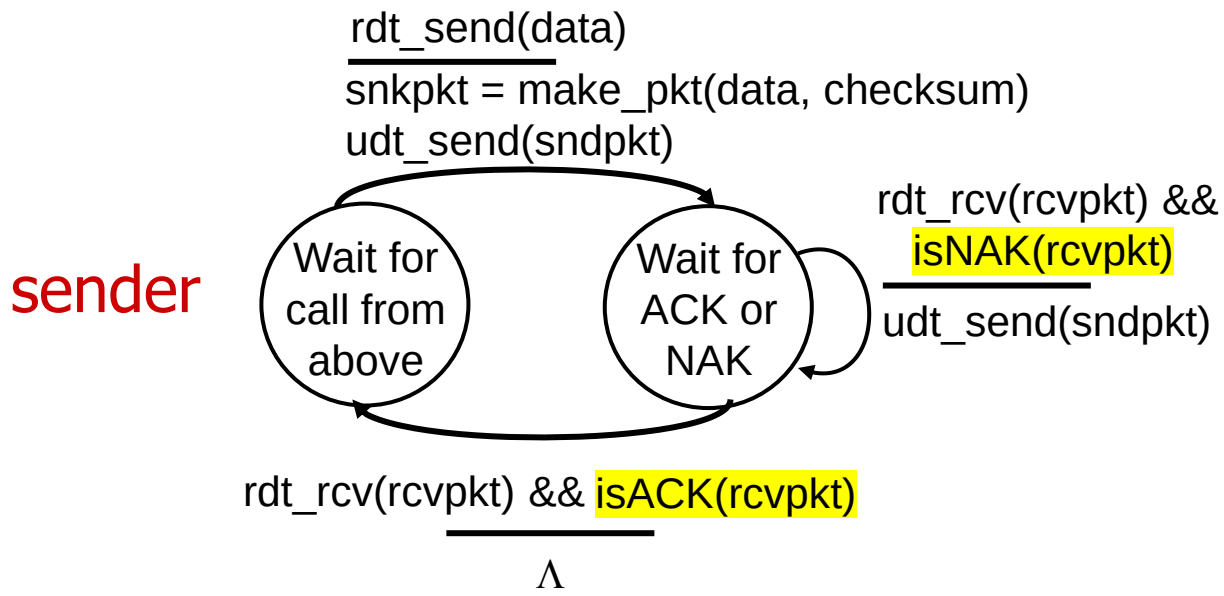
sender sends one packet, then waits for receiver response

rdt2.0: FSM specifications

sender



rdt2.0: FSM specification

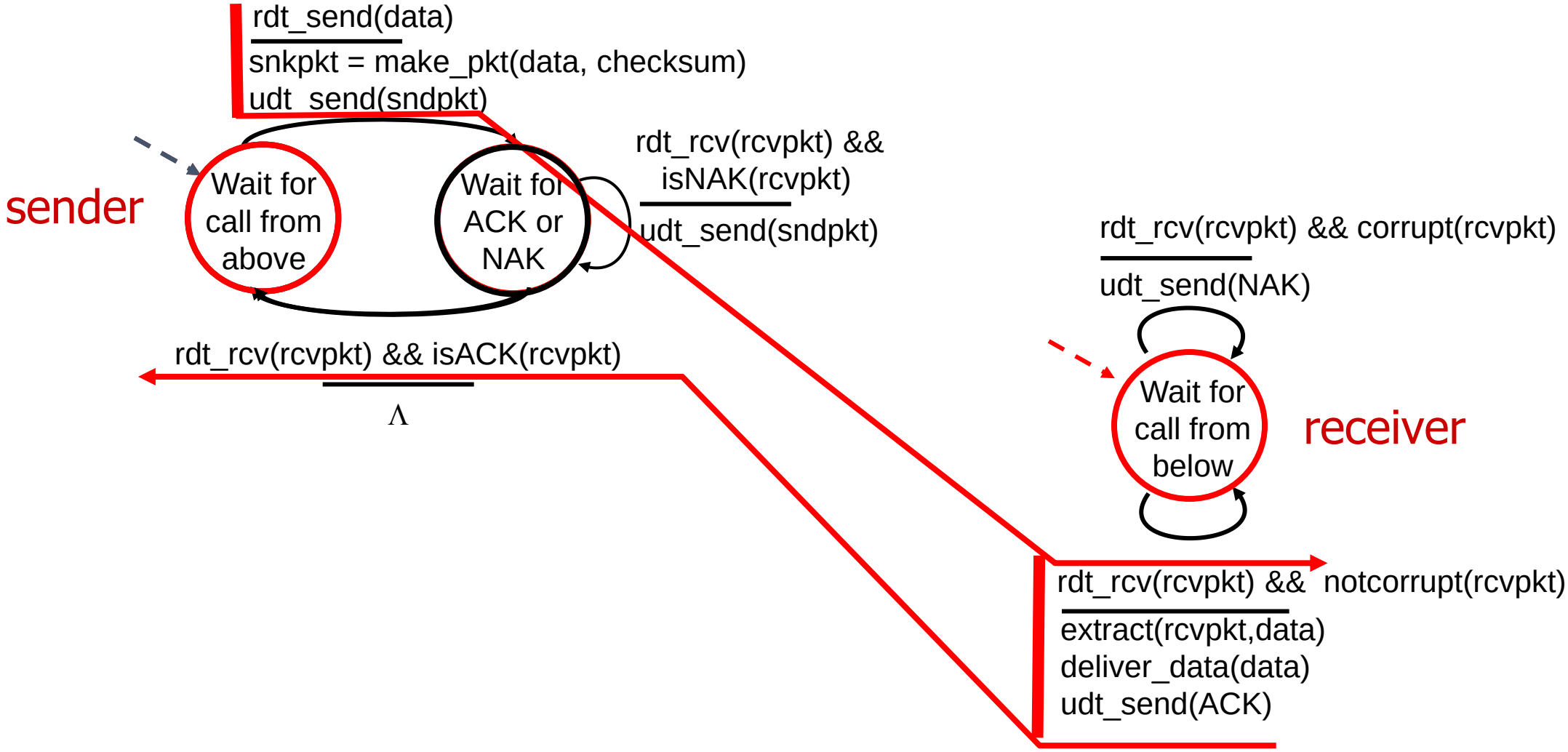


Note: "state" of receiver (did the receiver get my message correctly?) isn't known to sender unless somehow communicated from receiver to sender

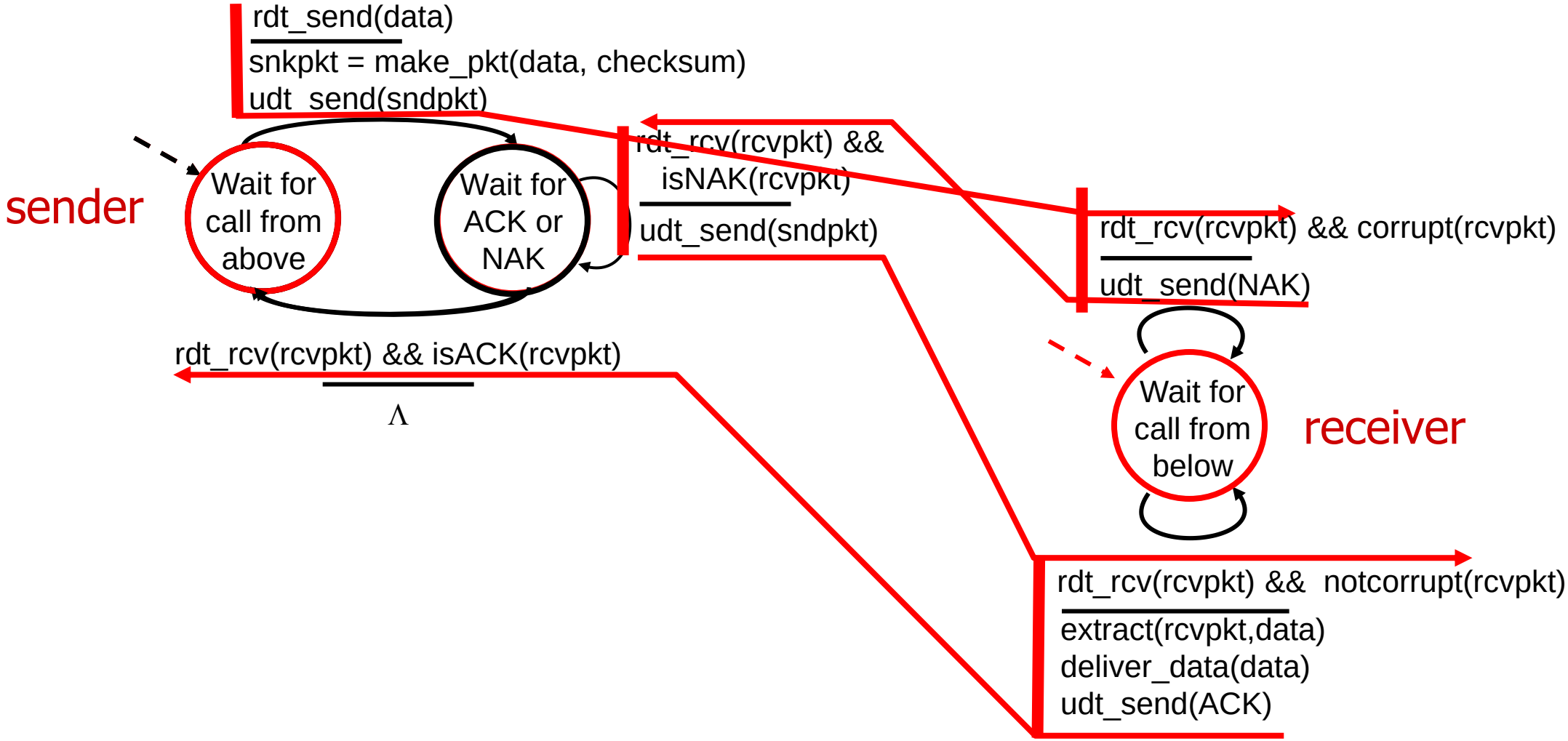
- that's why we need a protocol!



rdt2.0: operation with no errors



rdt2.0: corrupted packet scenario



rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

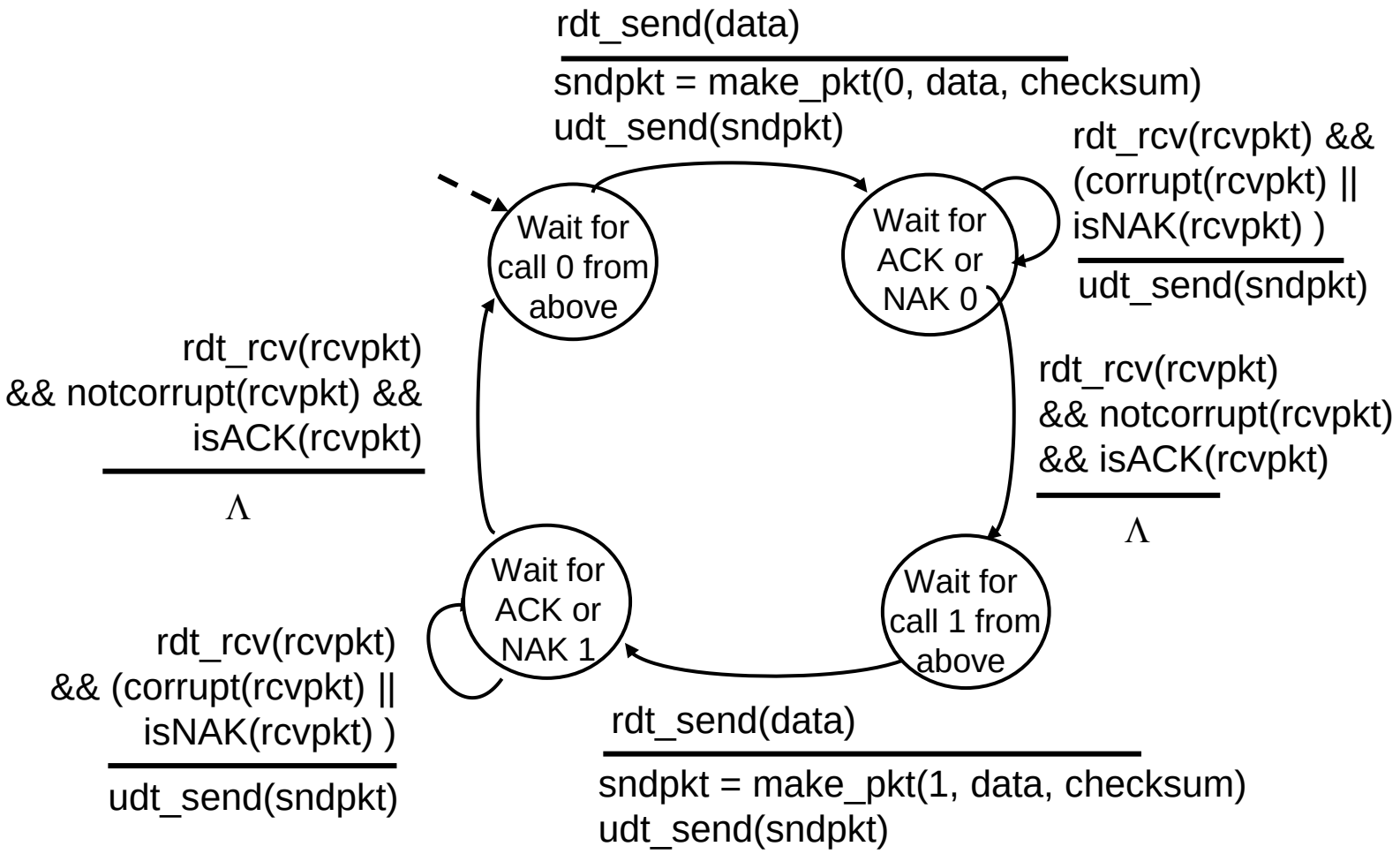
handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

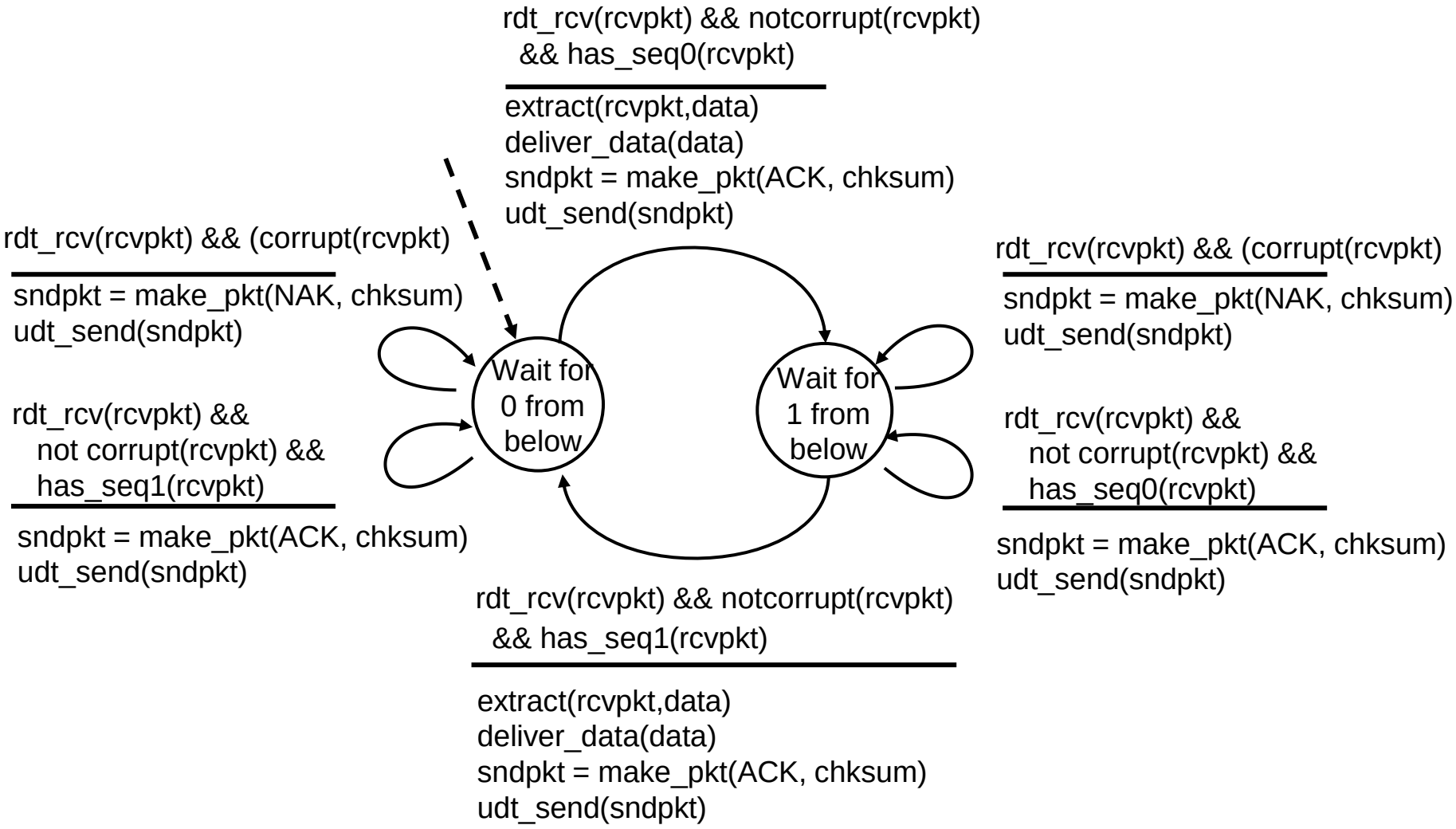
stop and wait

sender sends one packet, then waits for receiver's response

rdt2.1: sender, handling garbled ACK/NAKs



rdt2.1: receiver, handling garbled ACK/NAKs



rdt2.1: discussion

sender:

- seq # added to pkt
- two seq. #s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states
  - state must "remember" whether "expected" pkt should have seq # of 0 or 1

receiver:

- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

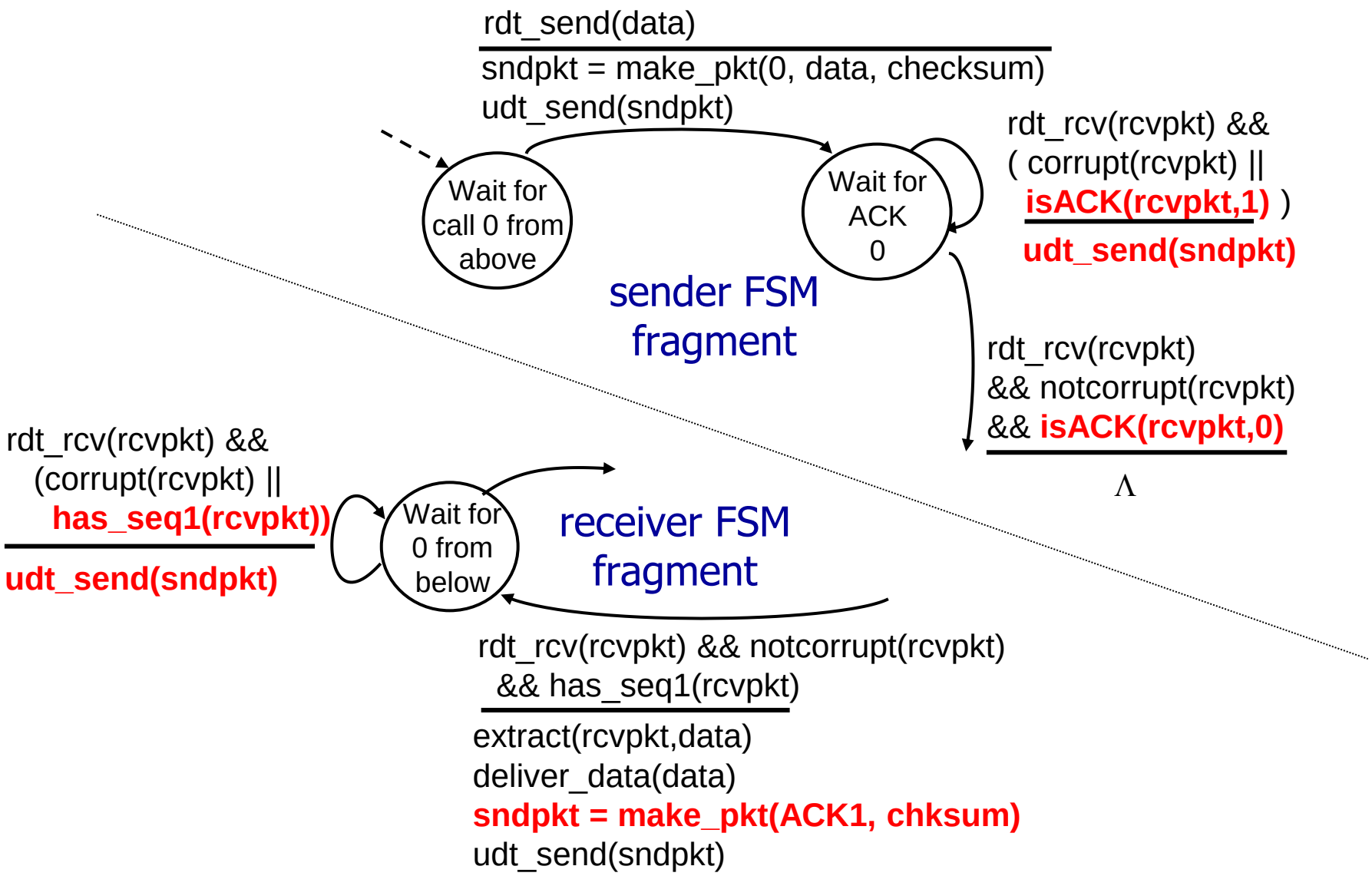
rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

NAK = duplicate ACK

As we will see, TCP uses this approach to be NAK-free

rdt2.2: sender, receiver fragments



rdt3.0: channels with errors *and* loss

New channel assumption: underlying channel can also *lose* packets (data, ACKs)

- checksum, sequence #s, ACKs, retransmissions will be of help … but not quite enough

Q: How do *humans* handle lost sender-to-receiver words in conversation?





rdt3.0: channels with errors *and* loss

Approach: sender waits "reasonable" amount of time for ACK

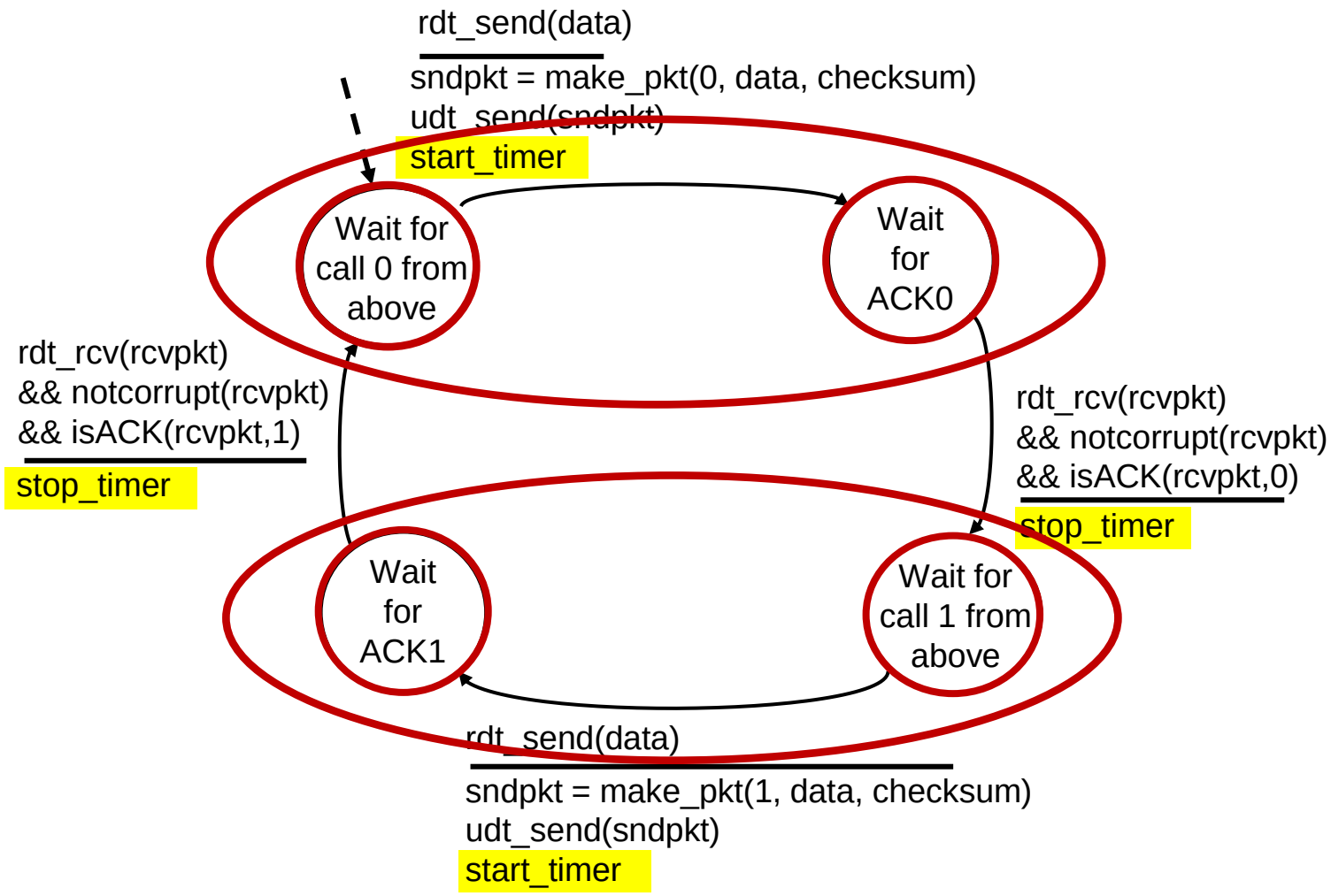
- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq #s already handles this!
  - receiver must specify seq # of packet being ACKed
- use countdown timer to interrupt after "reasonable" amount of time



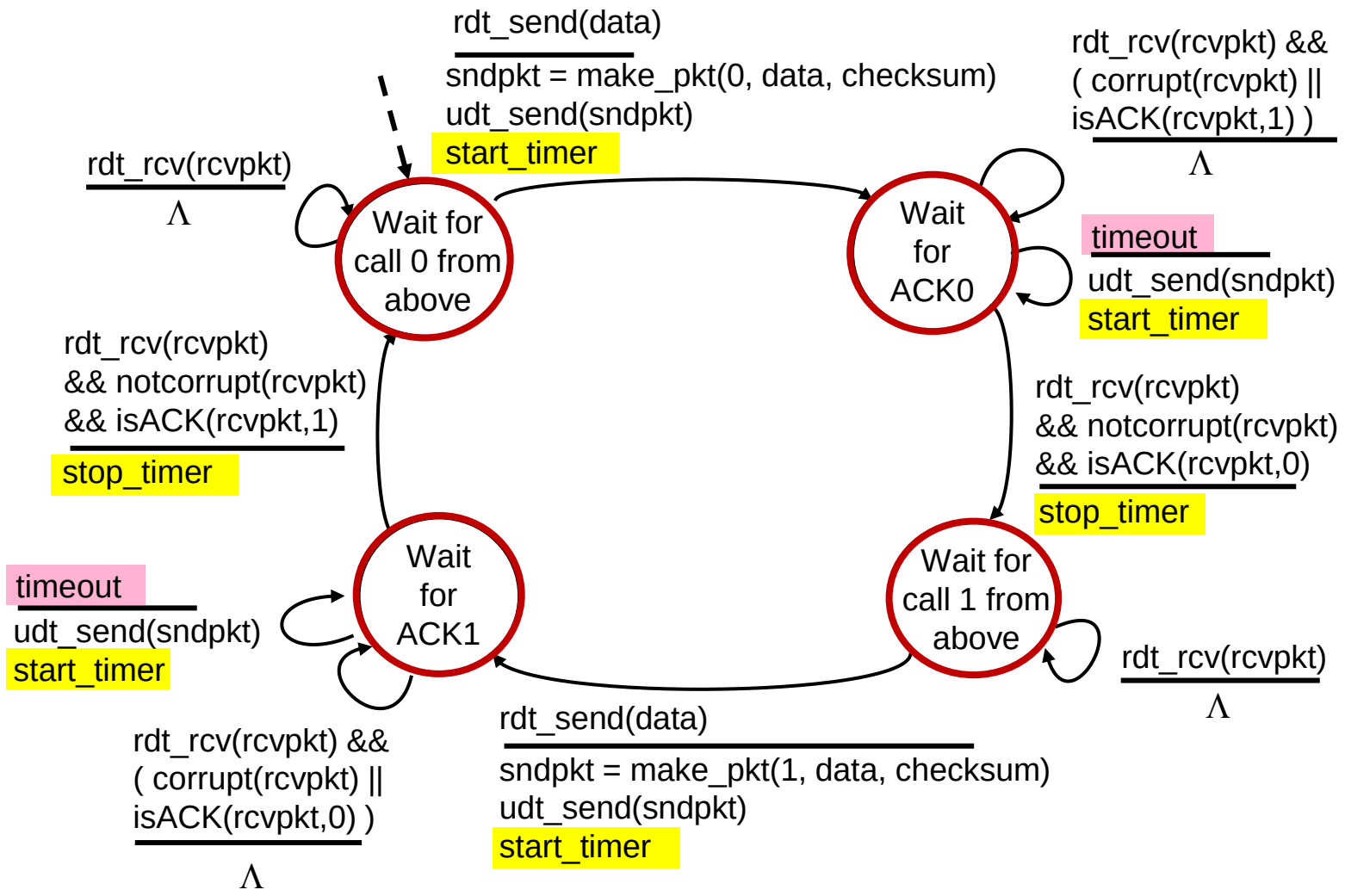
timeout

Q: If RTT is constant, would a timer still be necessary?

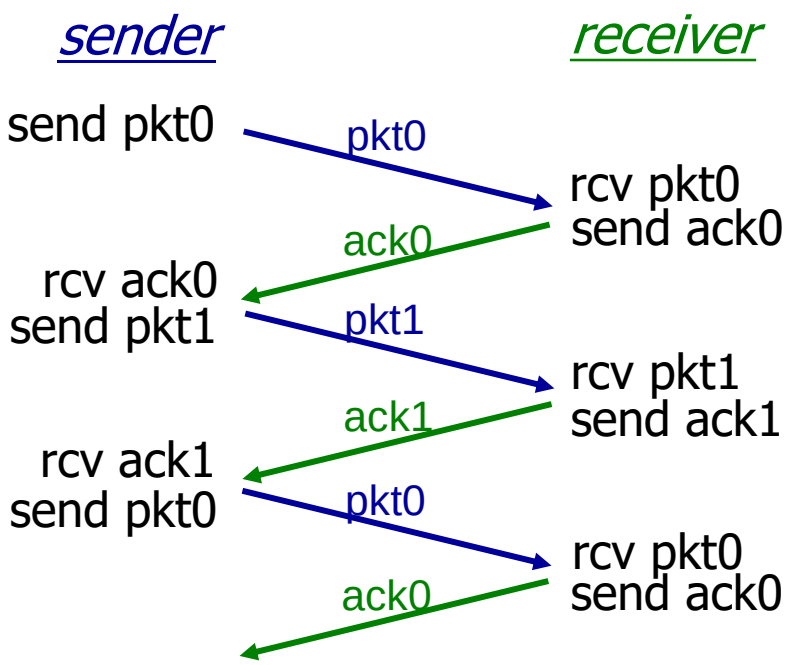
rdt3.0 sender



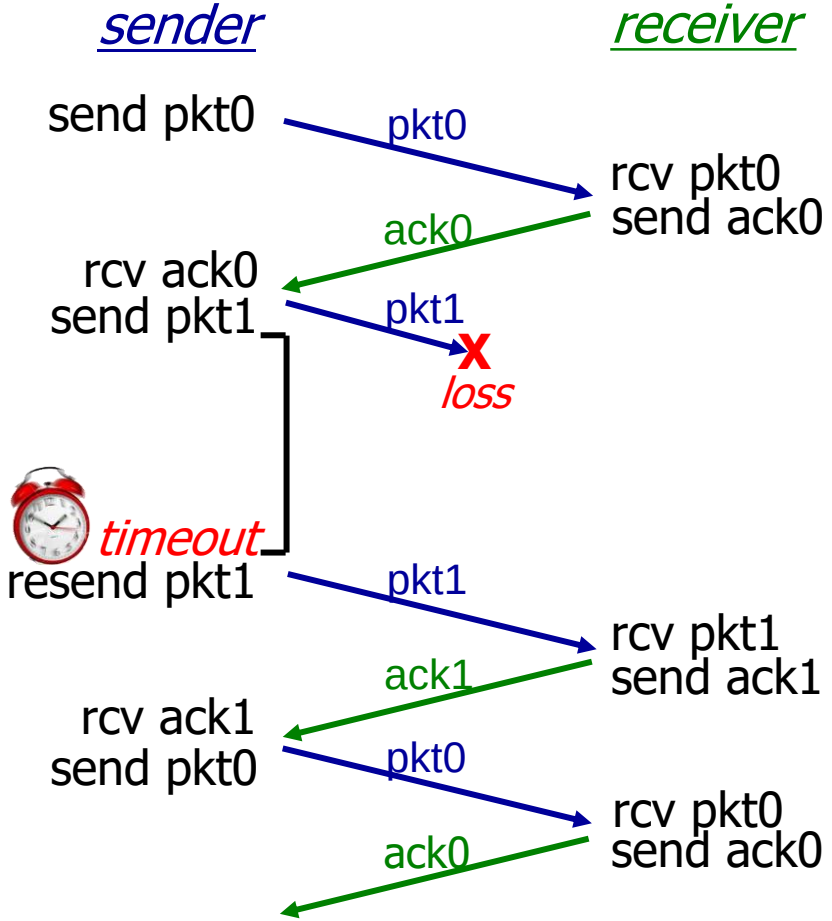
rdt3.0 sender



rdt3.0 in action

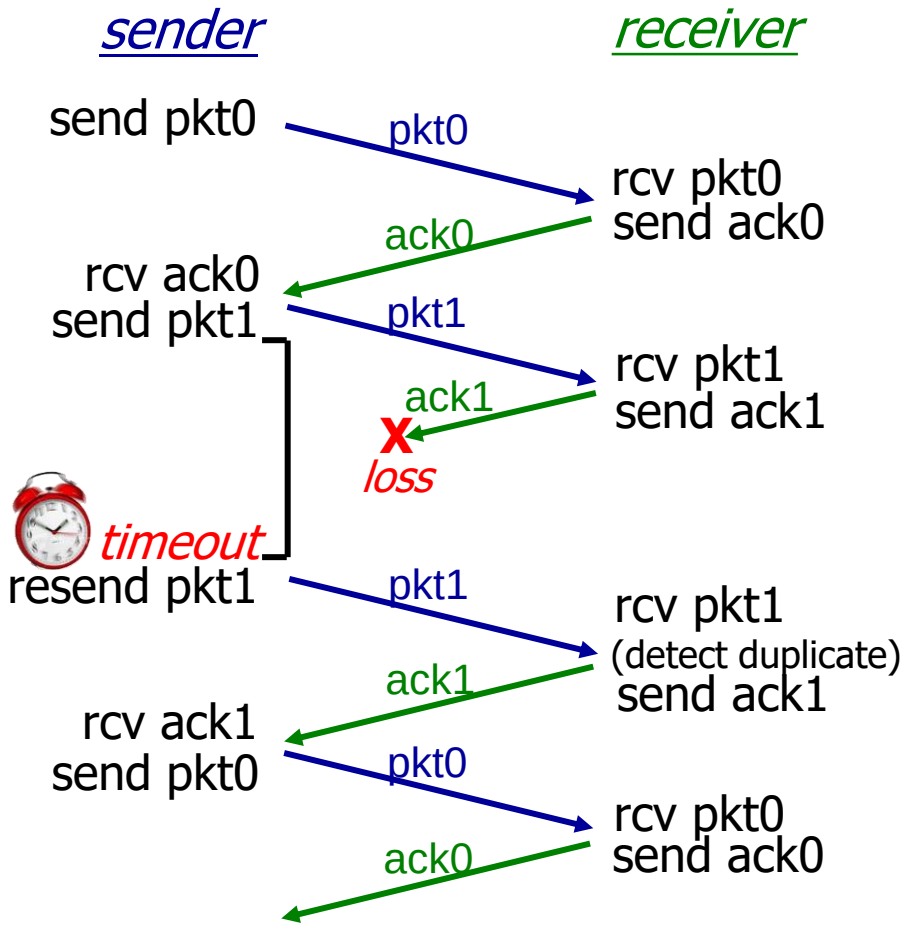


(a) no loss

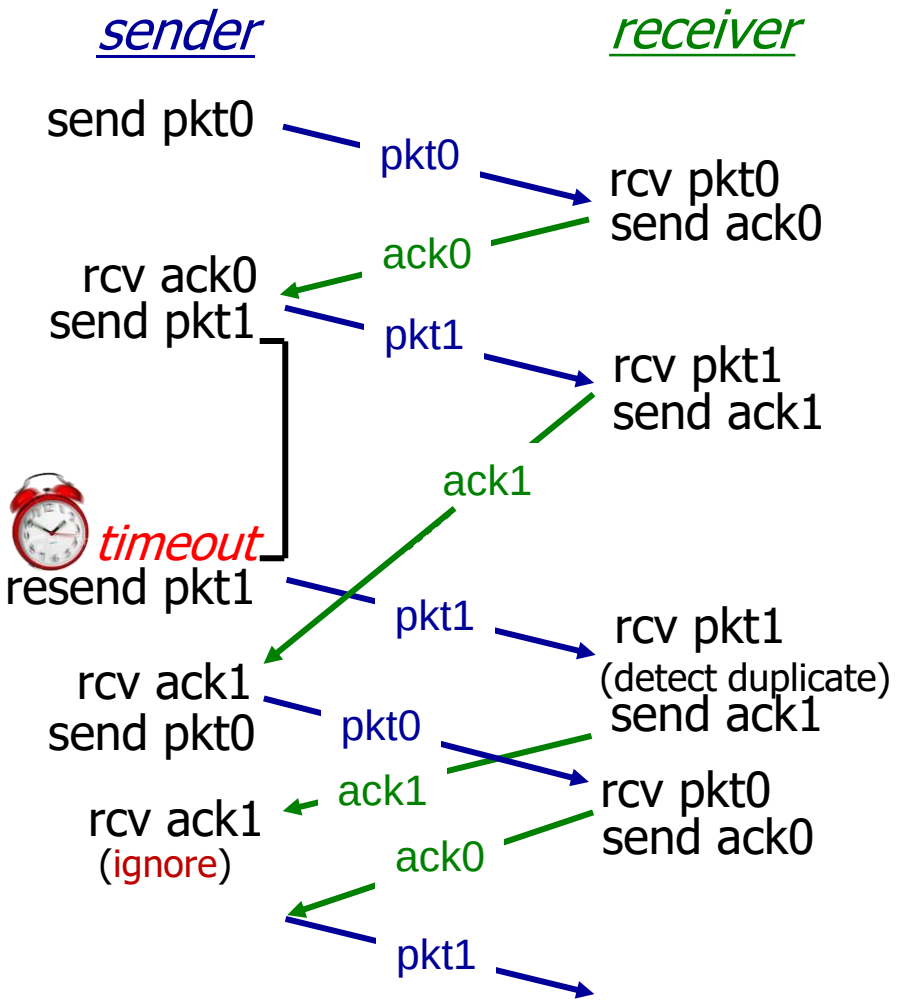


(b) packet loss

rdt3.0 in action



(c) ACK loss



(d) premature timeout / delayed ACK

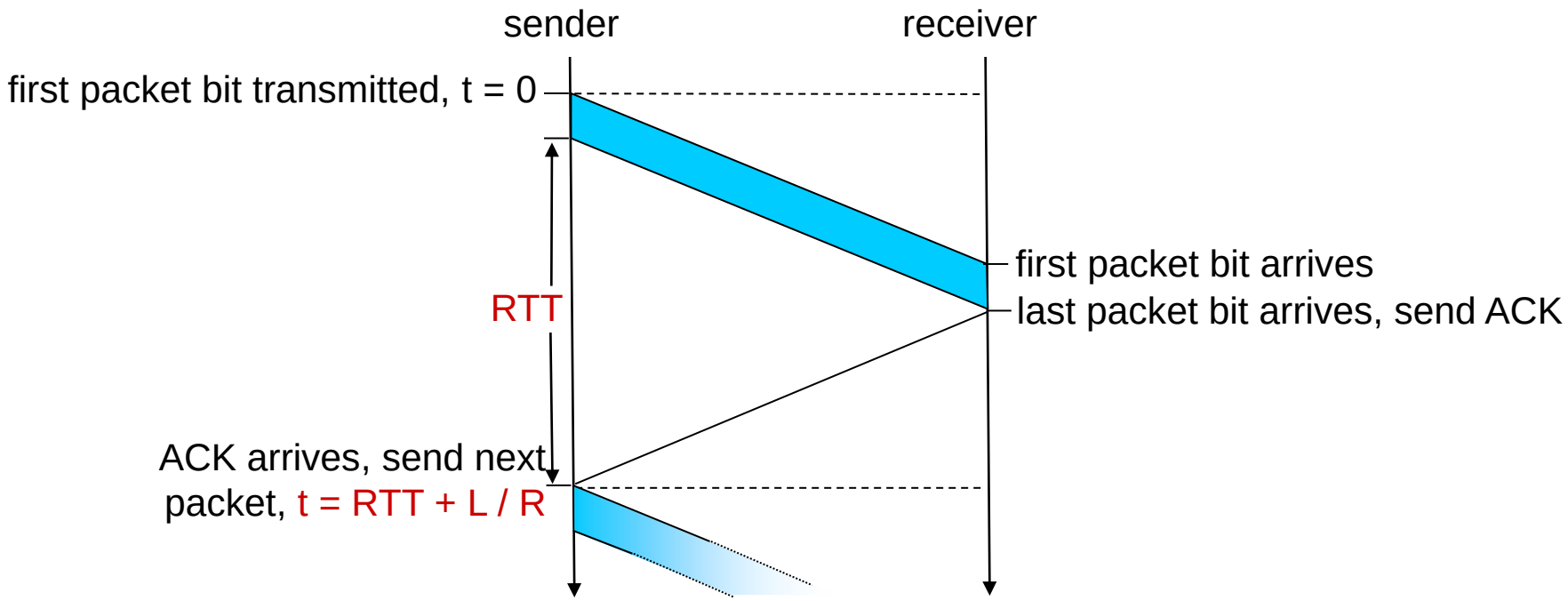
Performance of rdt3.0 (stop-and-wait)

- U_{sender}: *utilization* – fraction of time sender busy sending

- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet
  - time to transmit packet into channel:

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs} = 8\ \mu s$$

rdt3.0: stop-and-wait operation

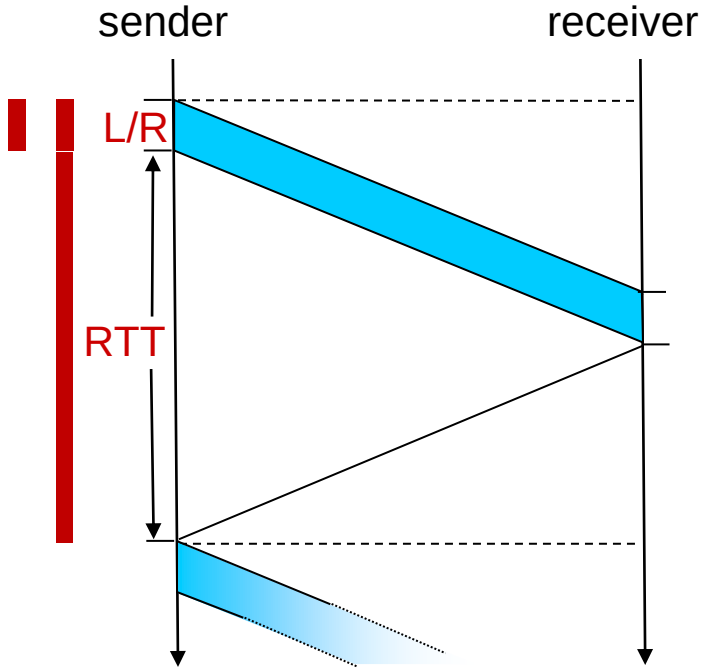


rdt3.0: stop-and-wait operation

$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{1}{\frac{RTT \cdot R}{L} + 1}$$

$$= \frac{.008}{30.008}$$

$$= 0.00027$$



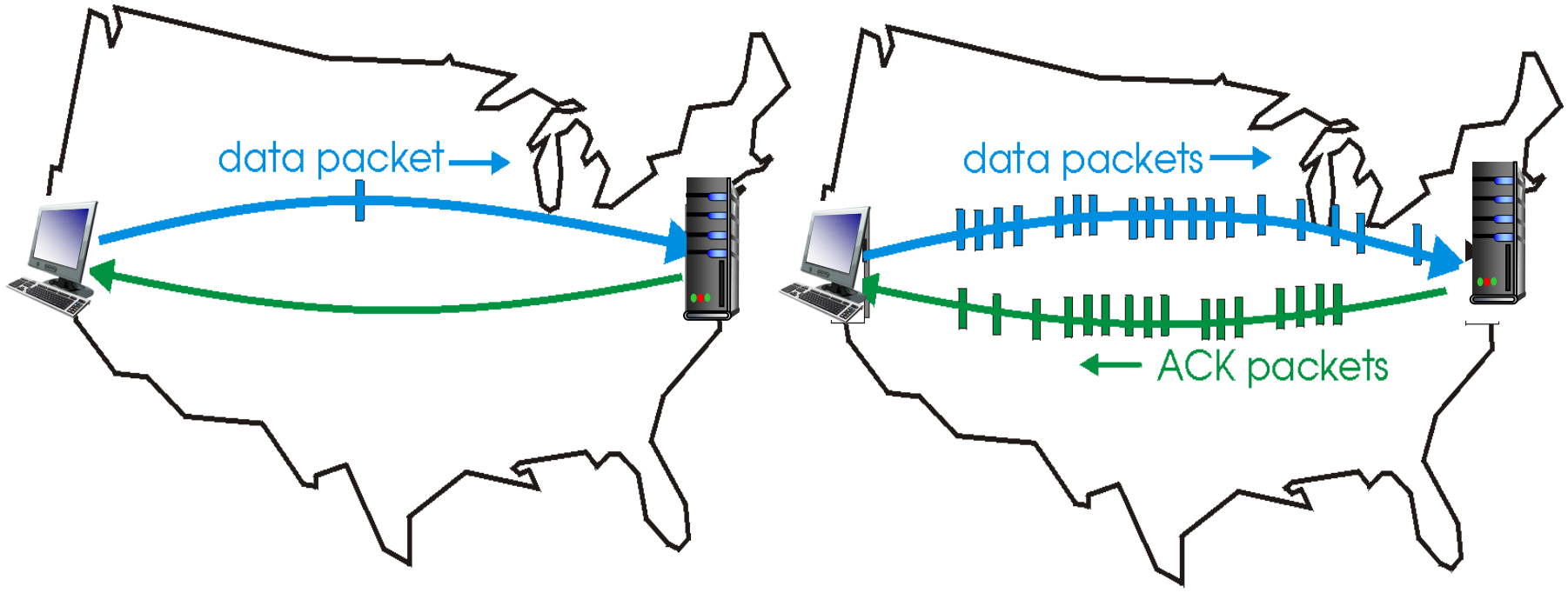
- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

rdt3.0: pipelined protocols operation

pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged packets

- range of sequence numbers must be increased, each in-transit packet (not counting retransmissions) must have a unique sequence number
- buffering at sender and/or receiver

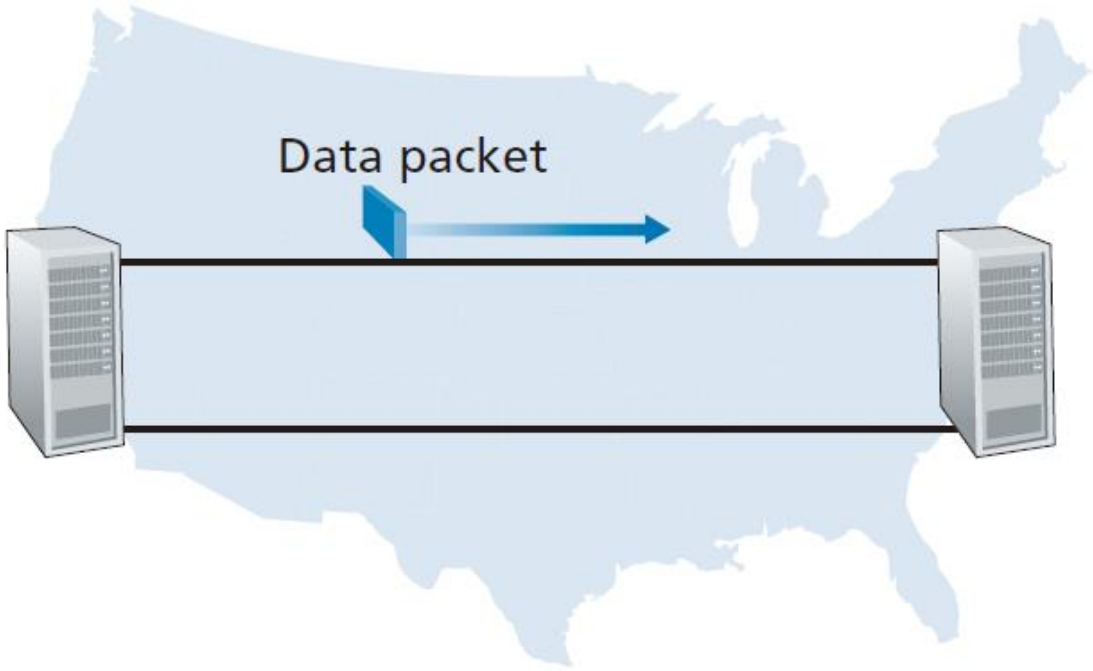
Two basic approaches toward pipelined error recovery can be identified: **Go-Back-N** and **selective repeat**.



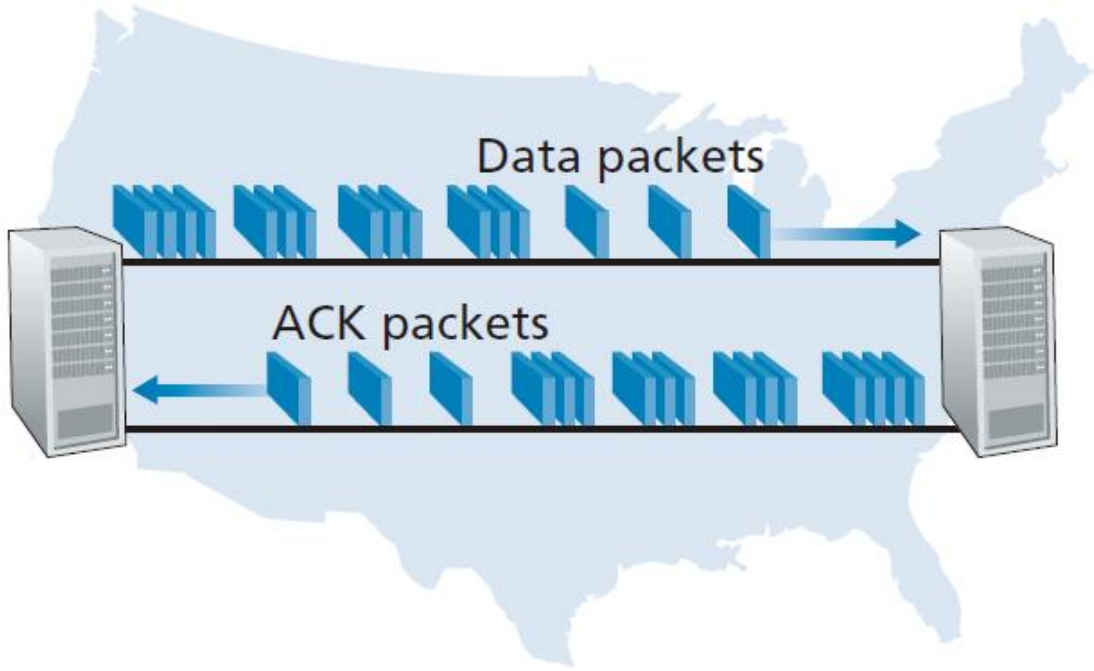
(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

Stop-and-wait versus pipelined protocol

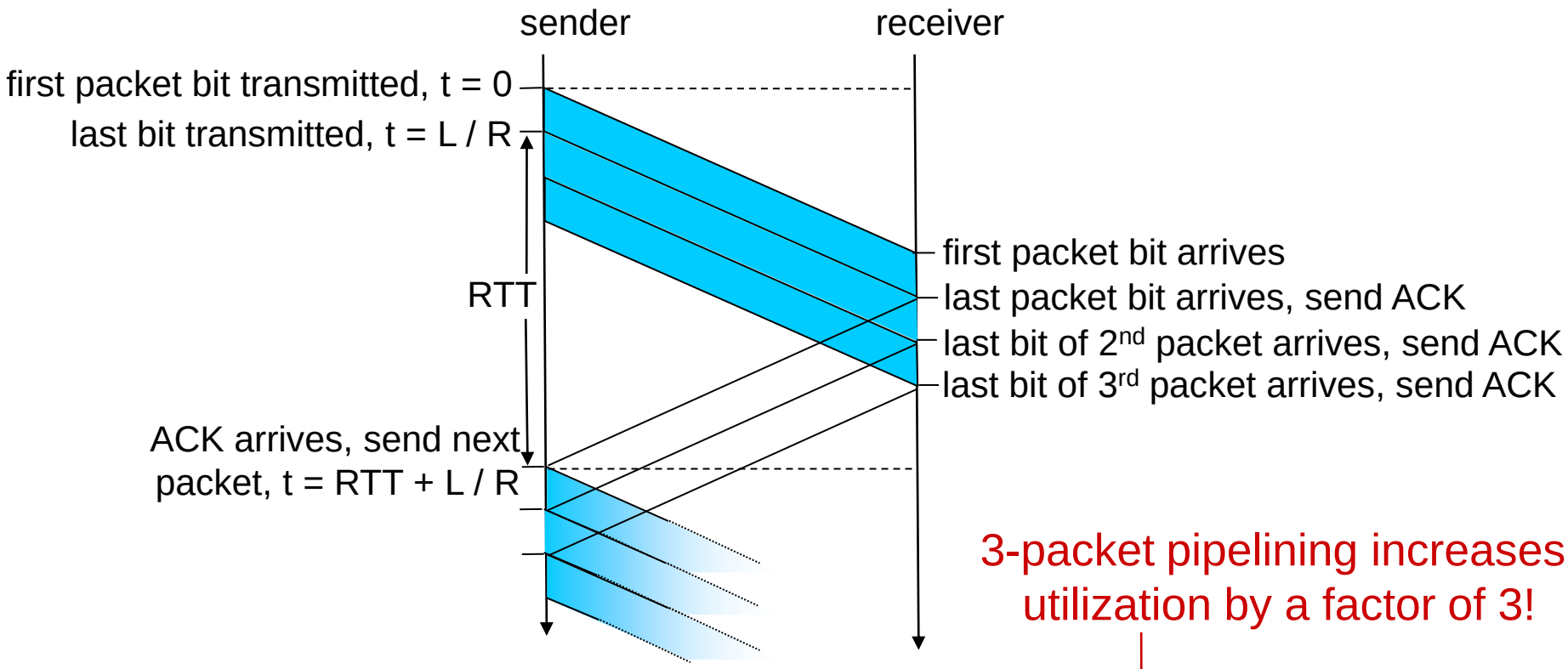


a. A stop-and-wait protocol in operation



b. A pipelined protocol in operation

Pipelining: increased utilization

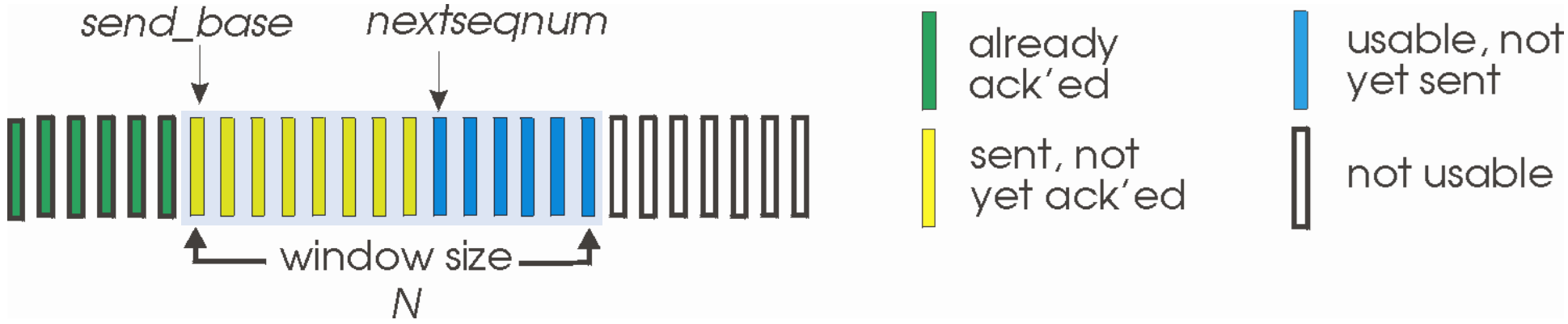


3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Go-Back-N: sender

- sender: "window" of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header

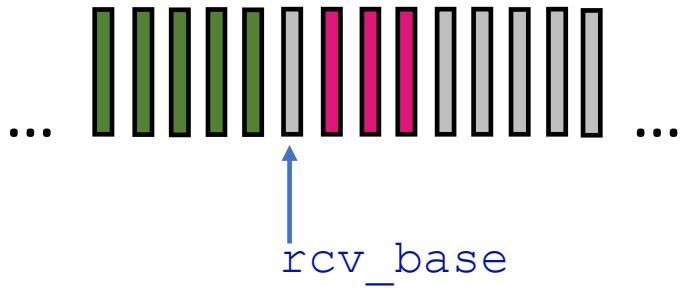


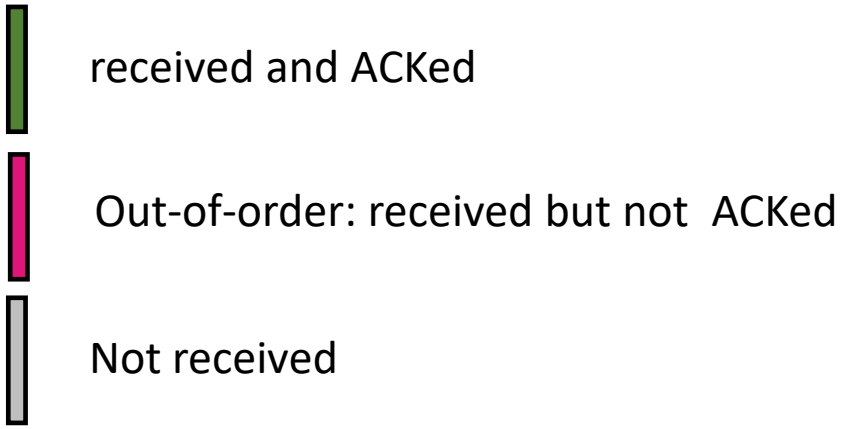
- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet *n* and all higher seq # packets in window

Go-Back-N: receiver

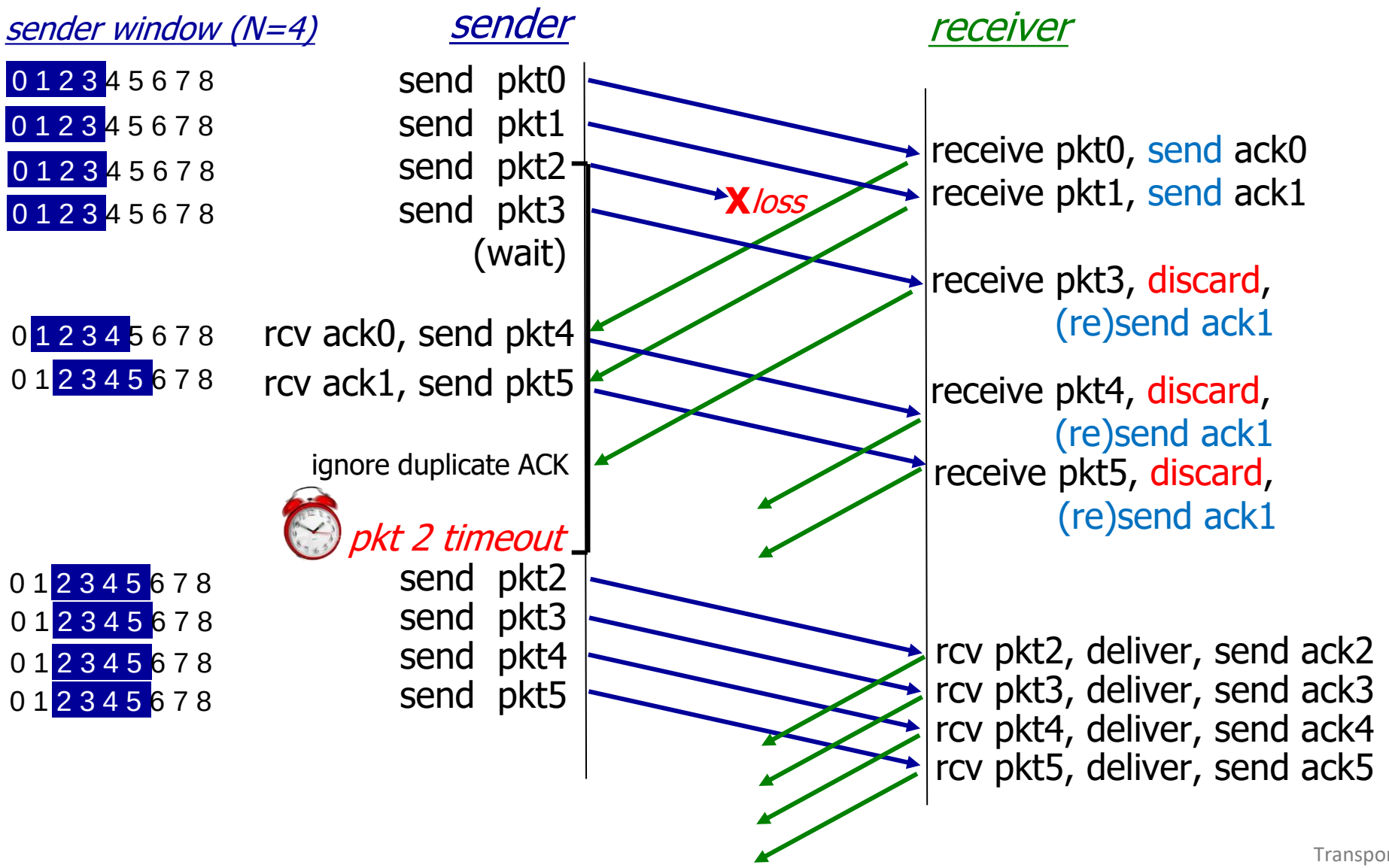
- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember **`rcv_base`**
- on receipt of **out-of-order** packet:
  - can discard (don't buffer) or buffer: an implementation decision
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:





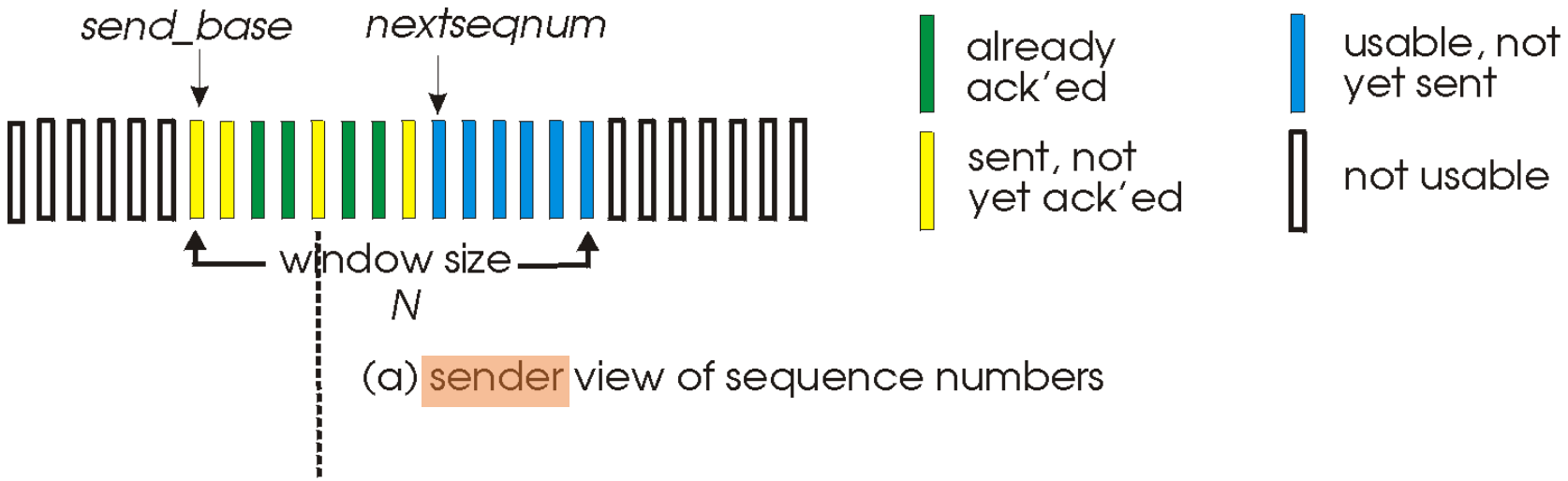
Go-Back-N in action



Selective repeat: the approach

- *pipelining*: *multiple* packets in flight
- *receiver individually ACKs* all correctly received packets
  - buffers packets, as needed, for in-order delivery to upper layer
- sender:
  - maintains (conceptually) a timer for each unACKed pkt
    - *timeout:* retransmits single unACKed packet associated with timeout
  - maintains (conceptually) "window" over *N* consecutive seq #s
    - limits pipelined, "in flight" packets to be within this window

Selective repeat (SR): sender, receiver windows



Selective repeat (SR): sender and receiver

sender

data from above:

- if next available seq # in window, send packet

timeout(*n*):

- resend packet *n*, restart timer

ACK(*n*) in [sendbase, sendbase+N]:

- mark packet *n* as received
- if *n* smallest unACKed packet, advance window base to next unACKed seq #

receiver

packet *n* in [rcvbase, rcvbase+N-1]

- send ACK(*n*)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

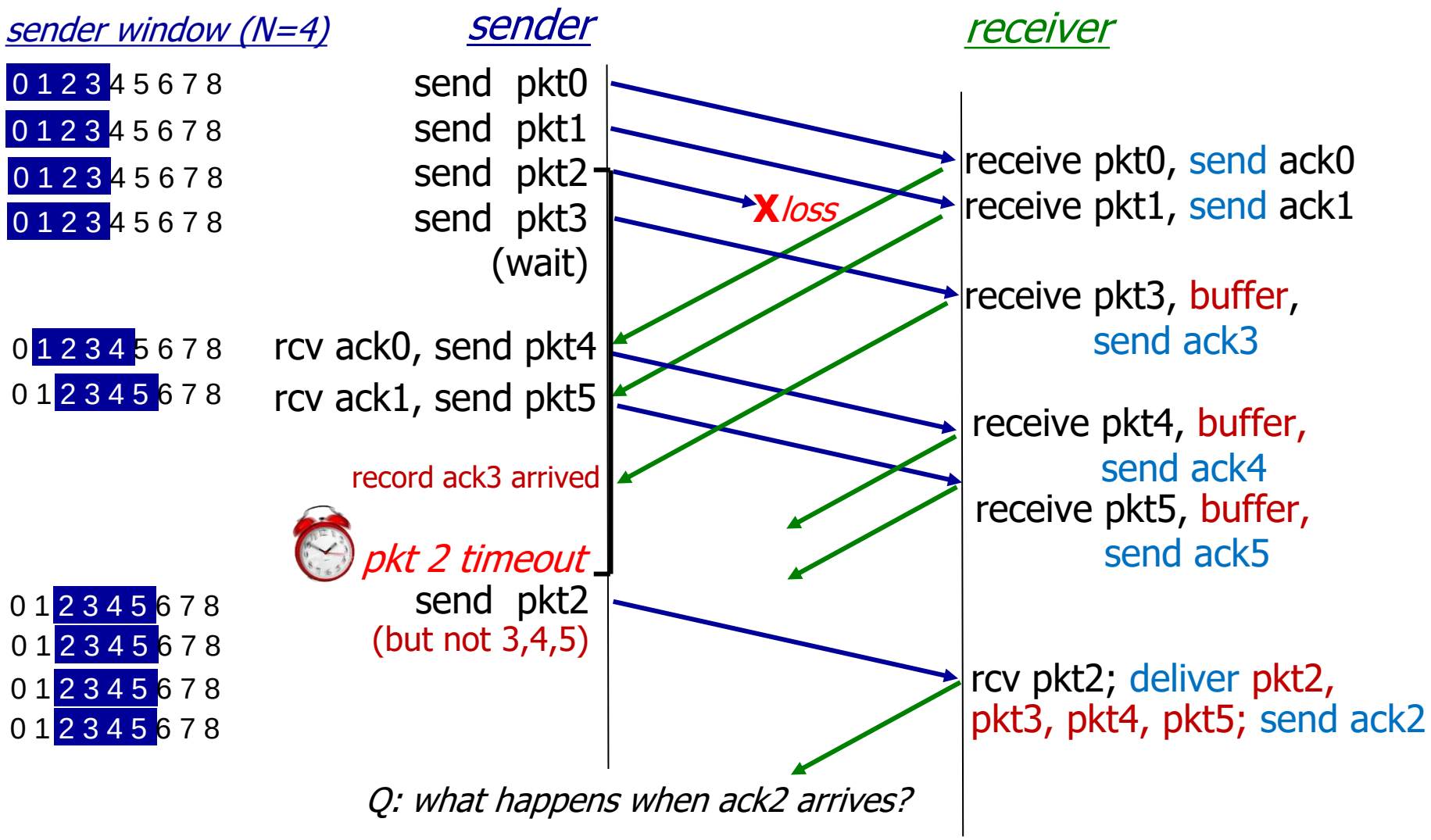
packet *n* in [rcvbase-N,rcvbase-1]

- ACK(*n*)

otherwise:

- ignore

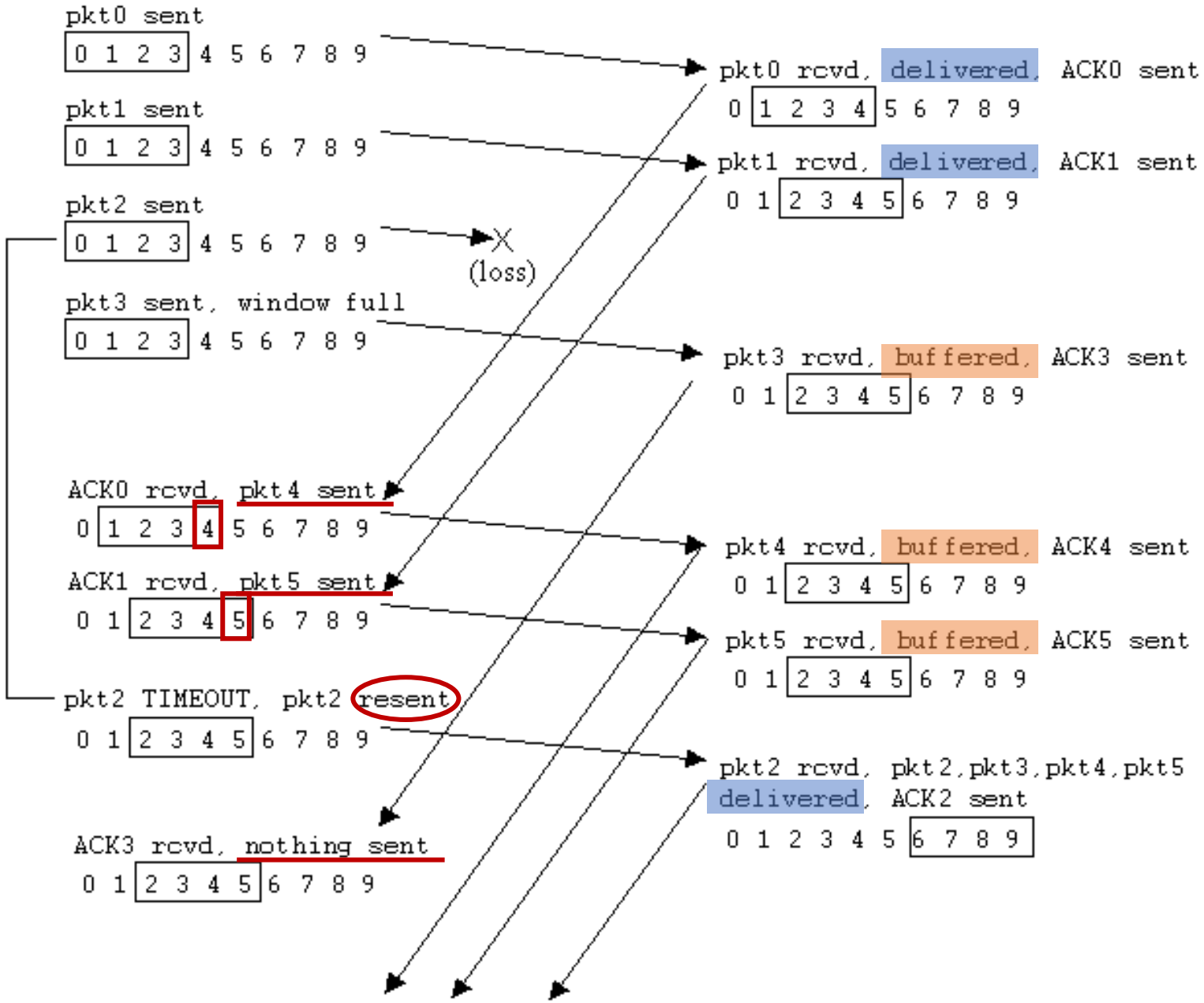
Selective Repeat (SR) in action



Selective repeat in action

If the receiver were not to acknowledge this packet, the sender's window would never move forward!

The **sender** and **receiver** will not always have an identical view of what has been received correctly and what has not.

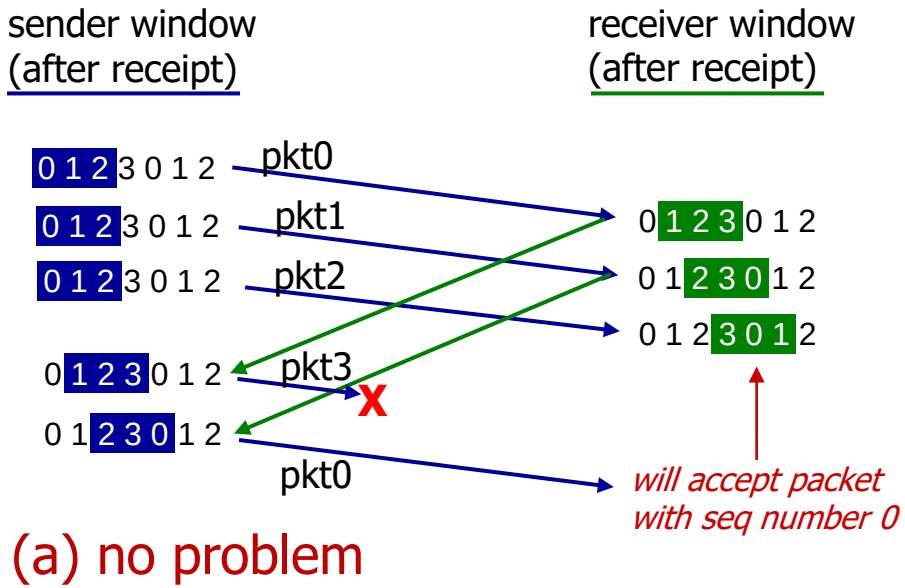


SR receiver dilemma

with too-large windows: A new packet or a retransmission?

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

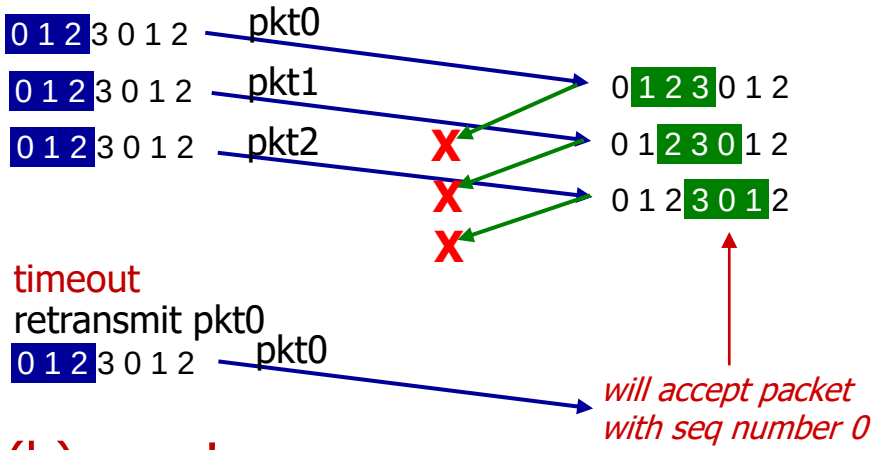


(a) no problem

The two scenarios in the right Figure are

identical.

There is no way of distinguishing the retransmission of the first packet from an original transmission of the fifth packet.



(b) oops!

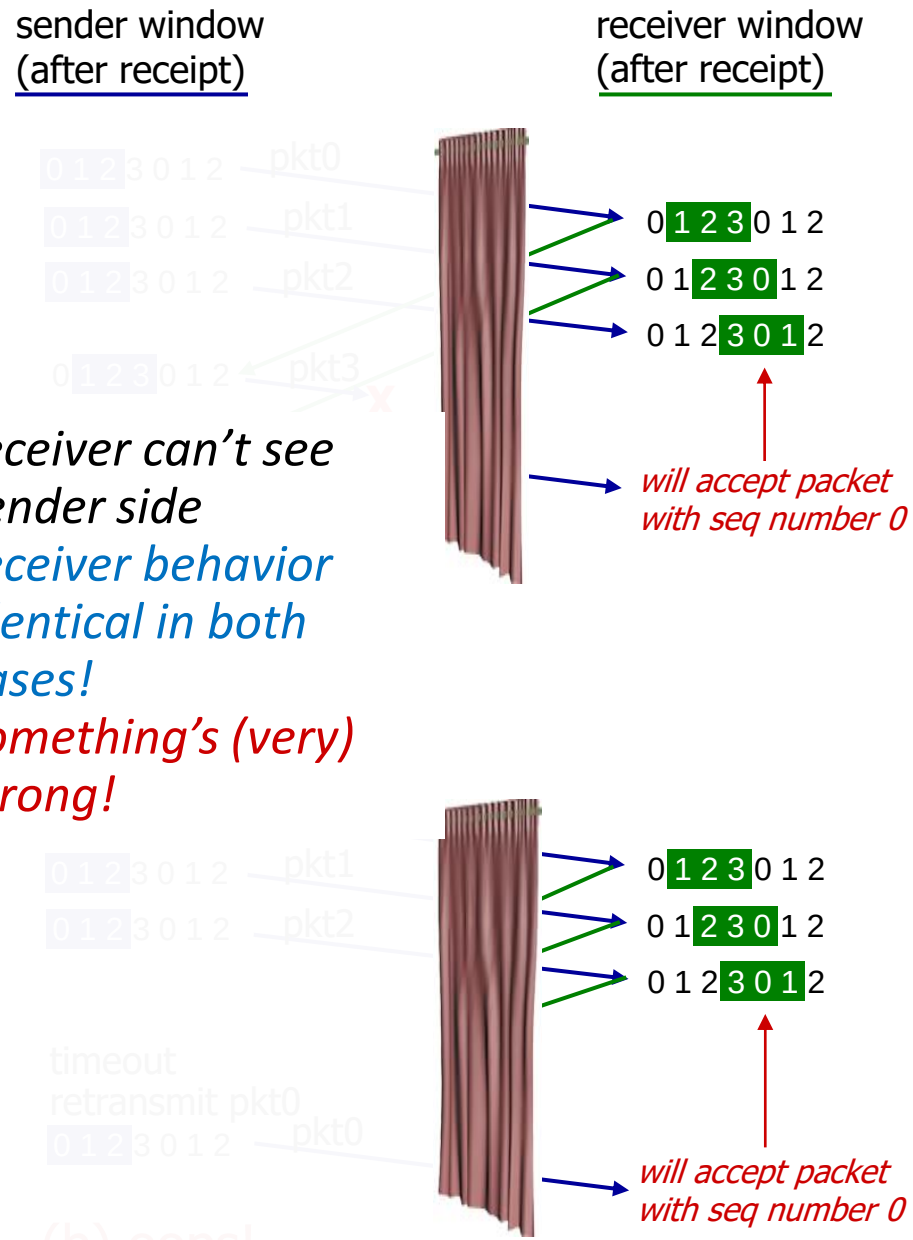
Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

Q: what relationship is needed between sequence # size and window size to avoid problem in scenario (b)?

window size <= half of seq # size



- *receiver can't see sender side*
- *receiver behavior identical in both cases!*
- *something's (very) wrong!*

Summary of rdt mechanisms and their uses

Mechanism	Use, comments
Checksum	Detect bit errors
Timer	Timeout/retransmit
Sequence #	Detect a lost packet /duplicate copies
Ack	Packet was received correctly (cumulative/individual)
NAK	Packet was NOT received correctly
Window, pipelining	Seq # in the window can be sent. Pipelining increases the send utilization.

Mechanism	Use, Comments
Checksum	Used to detect bit errors in a transmitted packet.
Timer	Used to timeout/retransmit a packet, possibly because the packet (or its ACK) was lost within the channel. Because timeouts can occur when a packet is delayed but not lost (premature timeout), or when a packet has been received by the receiver but the receiver-to-sender ACK has been lost, duplicate copies of a packet may be received by a receiver.
Sequence number	Used for sequential numbering of packets of data flowing from sender to receiver. Gaps in the sequence numbers of received packets allow the receiver to detect a lost packet. Packets with duplicate sequence numbers allow the receiver to detect duplicate copies of a packet.
Acknowledgment	Used by the receiver to tell the sender that a packet or set of packets has been received correctly. Acknowledgments will typically carry the sequence number of the packet or packets being acknowledged. Acknowledgments may be individual or cumulative, depending on the protocol.
Negative acknowledgment	Used by the receiver to tell the sender that a packet has not been received correctly. Negative acknowledgments will typically carry the sequence number of the packet that was not received correctly.
Window, pipelining	The sender may be restricted to sending only packets with sequence numbers that fall within a given range. By allowing multiple packets to be transmitted but not yet acknowledged, sender utilization can be increased over a stop-and-wait mode of operation. We'll see shortly that the window size may be set on the basis of the receiver's ability to receive and buffer messages, or the level of congestion in the network, or both.

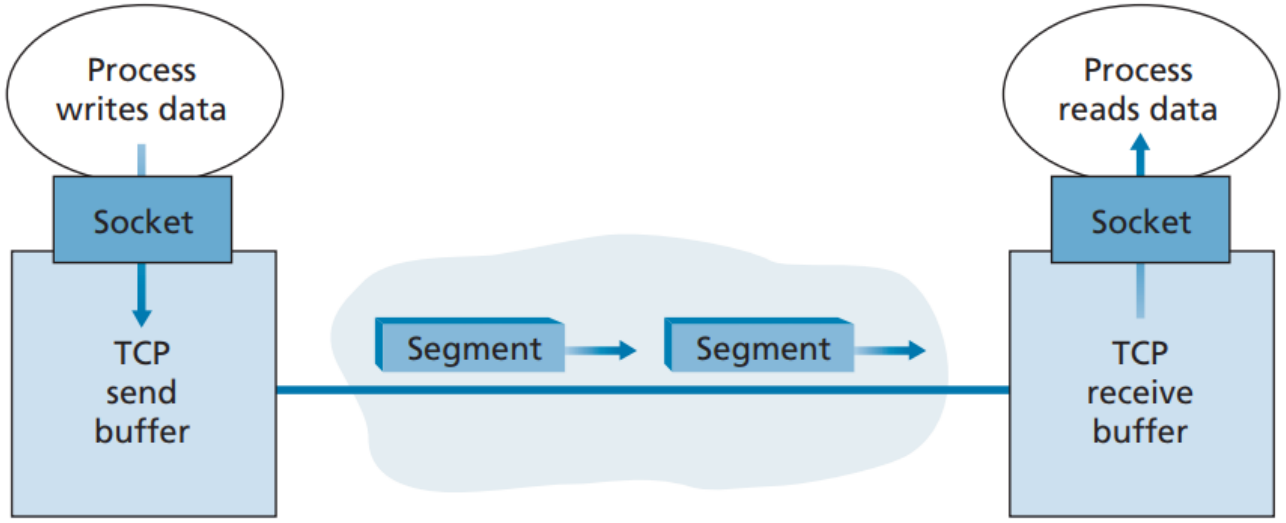
Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control

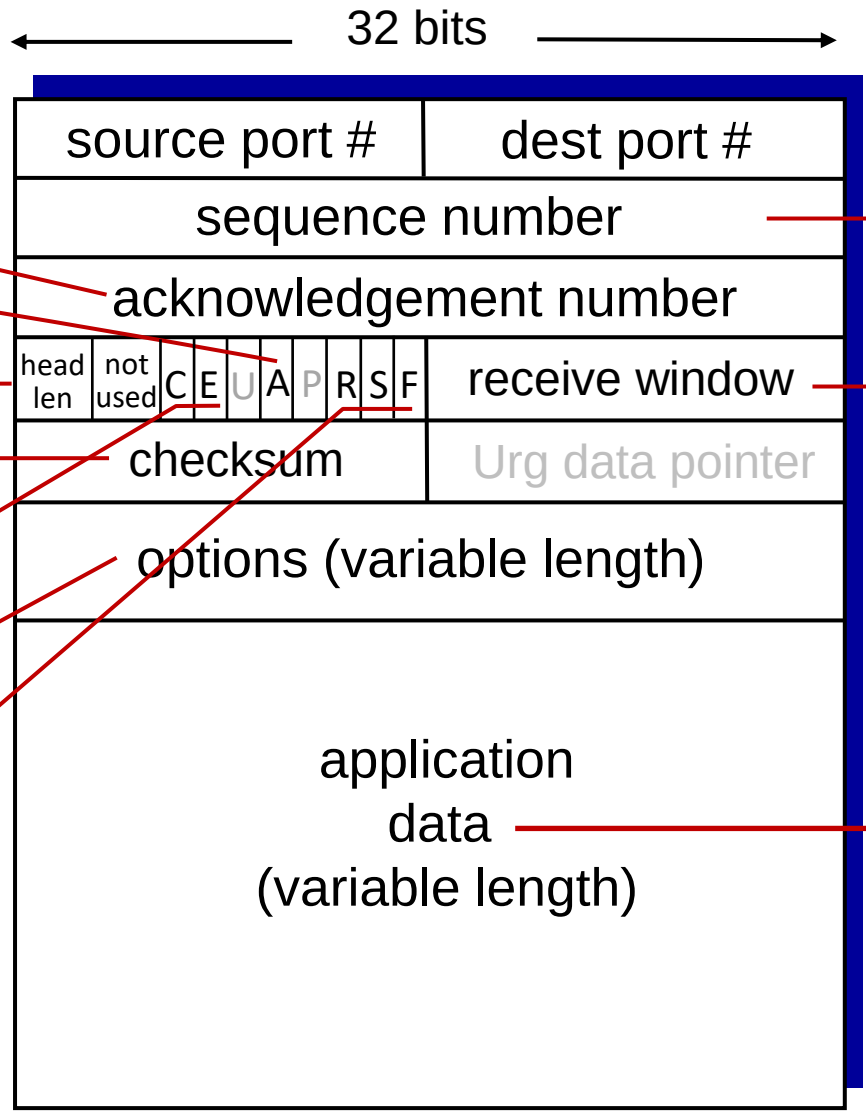


TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no “message boundaries"
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: Maximum Segment Size (without header, typical 1460 bytes)
- cumulative ACKs
- pipelining:
  - TCP congestion and flow control set window size
- connection-oriented:
  - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver



TCP segment structure



ACK: seq # of next expected byte;
A bit: this is an ACK

4-bit length (of TCP header)

Internet checksum

CWR: Congestion Window Reduced,
ECE: Explicit Congestion notification Echo

TCP options

RST, SYN, FIN: connection setup and teardown

segment seq #: counting bytes of data into byte stream (not segments!)

flow control: # bytes receiver willing to accept

In practice, the PSH, URG, and the urgent data pointer are not used.

data sent by application into TCP socket

TCP sequence numbers, ACKs

TCP is **full-duplex**

Sequence numbers:

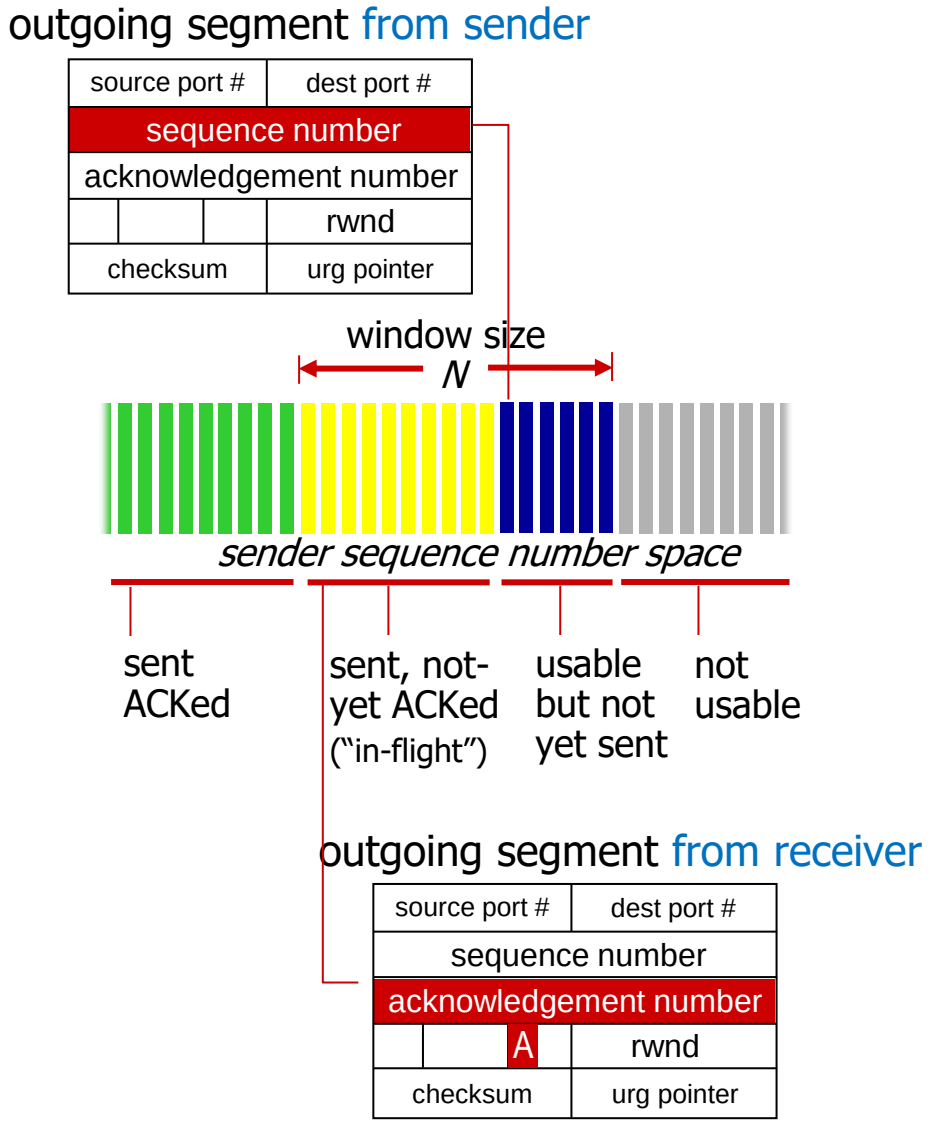
- byte stream "number" of first byte in segment's data

Acknowledgements:

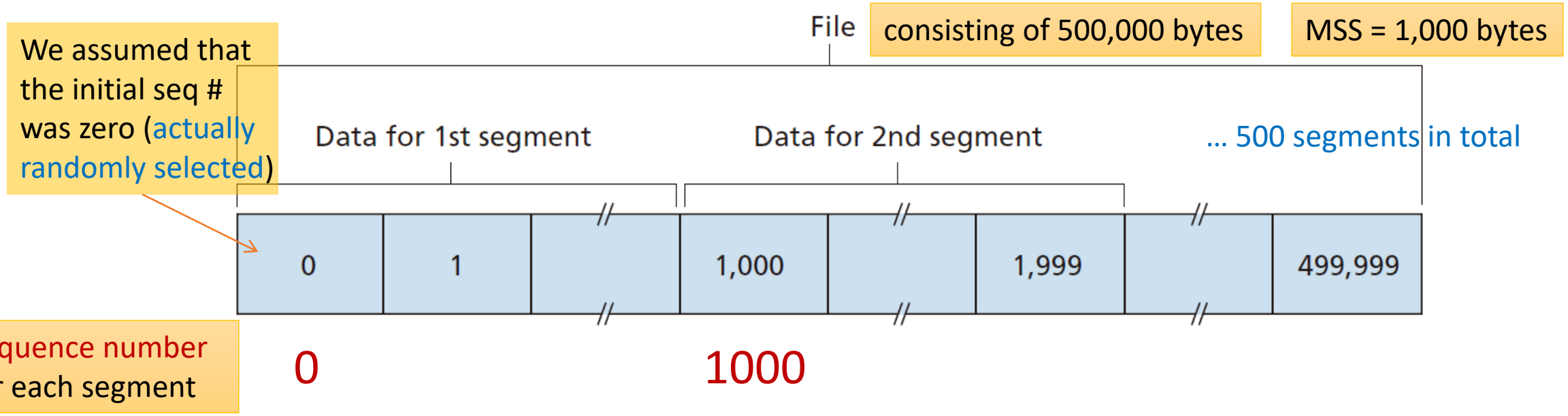
- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- *A:* TCP spec doesn't say, - up to implementor



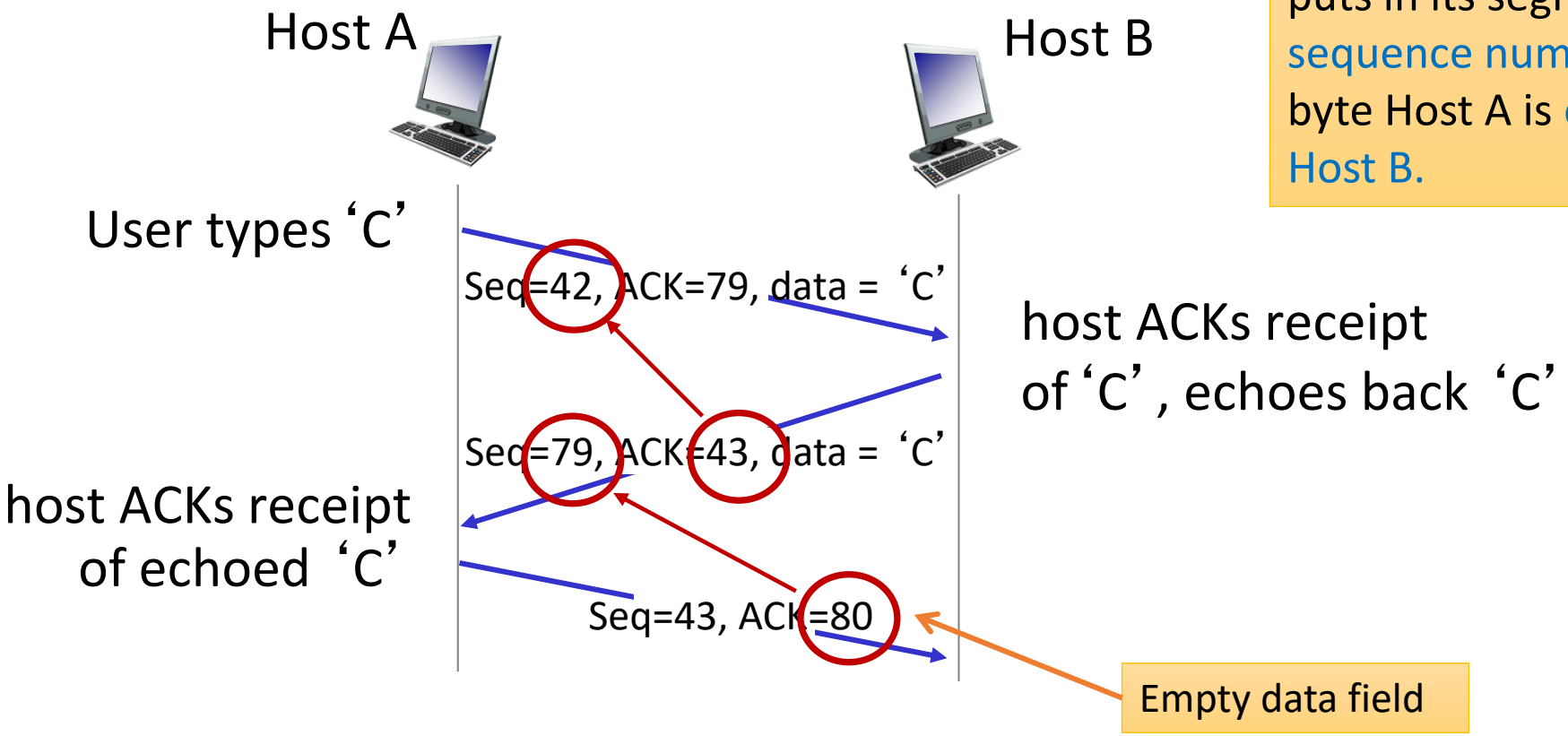
Dividing file data into TCP segments



- TCP views data as an unstructured, but ordered, stream of bytes.
- TCP's use of sequence numbers reflects this view
  - in that sequence numbers are over the stream of transmitted bytes and *not* over the series of transmitted segments.
- The **sequence number for a segment** is therefore the byte-stream number of the first byte in the segment.
- Each sequence number is inserted in the sequence number field in the header of the appropriate TCP segment.

TCP sequence numbers, ACKs

The ACK number that Host A puts in its segment is the sequence number of the next byte Host A is expecting from Host B.



simple telnet scenario

How receiver handles out-of-order segments

There are basically two choices:

(1) the receiver immediately discards out-of-order segments (which can simplify receiver design)

(2) the receiver keeps the out-of-order bytes and waits for the missing bytes to fill in the gaps.

Choice (2) is more efficient in terms of network bandwidth, and is the approach taken in practice.

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT, but RTT varies!
- *too short:* premature timeout, unnecessary retransmissions
- *too long:* slow reaction to segment loss

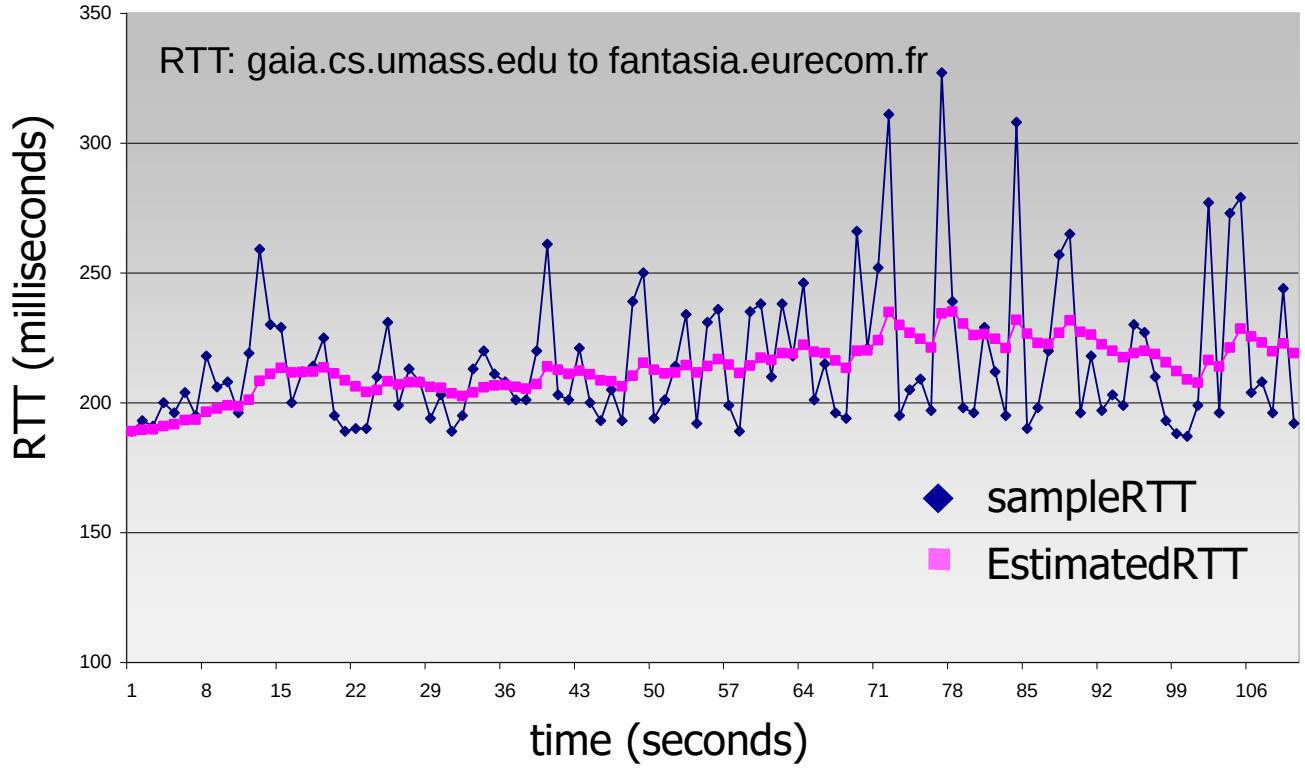
Q: how to estimate RTT?

- `SampleRTT:` measured time from segment transmission until ACK receipt
  - ignore retransmissions
- `SampleRTT` will vary, want estimated RTT "smoother"
  - average several *recent* measurements (called EstimatedRTT), not just current `SampleRTT`

TCP round trip time, timeout

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- Exponential Weighted Moving Average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$



TCP reliable data transfer

- TCP creates rdt service on top of IP's unreliable service
- Pipelined segments
- Cumulative acks
- TCP uses single retransmission timer

- Retransmissions are triggered by:
  - timeout events
  - duplicate acks
- Initially consider simplified TCP sender:
  - ignore duplicate acks
  - ignore flow control, congestion control

TCP Sender (simplified)

event: data received from application

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unACKed segment
  - expiration interval: **`TimeOutInterval`**

event: timeout

- retransmit segment that caused timeout
- restart timer

event: ACK received

- if ACK acknowledges previously unACKed segments
  - update what is known to be ACKed
  - start timer if there are still unACKed segments

TCP sender (simplified)

Comment:

- SendBase-1: last cumulatively ack'ed byte

Example:

- SendBase-1 = 71; y = 73, so the rcvr wants 73+ ; y > SendBase, so that new data is acked

```
NextSeqNum = InitialSeqNum
SendBase = InitialSeqNum

loop (forever) {
  switch(event)

  event: data received from application above
      create TCP segment with sequence number NextSeqNum
      if (timer currently not running)
          start timer
      pass segment to IP
      NextSeqNum = NextSeqNum + length(data)

  event: timer timeout
      retransmit not-yet-acknowledged segment with
          smallest sequence number
      start timer

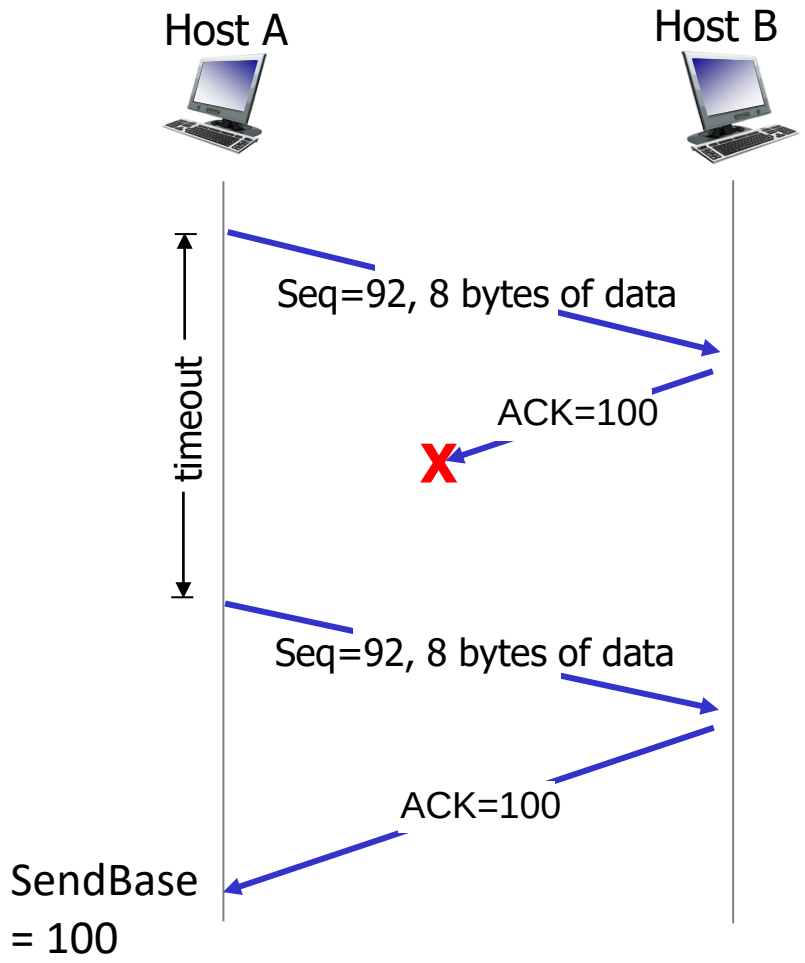
  event: ACK received, with ACK field value of y
      if (y > SendBase) {
          SendBase = y
          if (there are currently not-yet-acknowledged segments)
              start timer
          }

} /* end of loop forever */
```

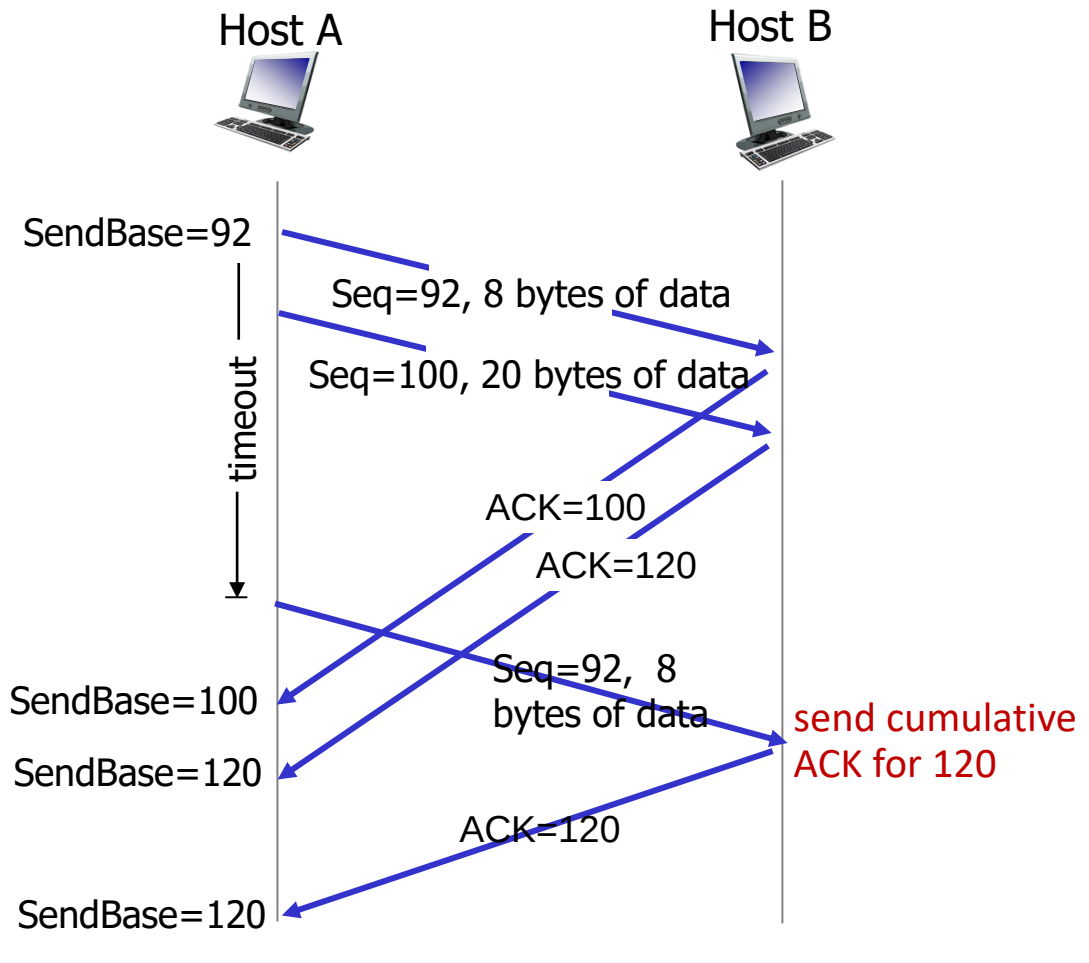
TCP Receiver: ACK generation [RFC 5681]

Event at receiver	*TCP receiver action*
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

TCP: retransmission scenarios

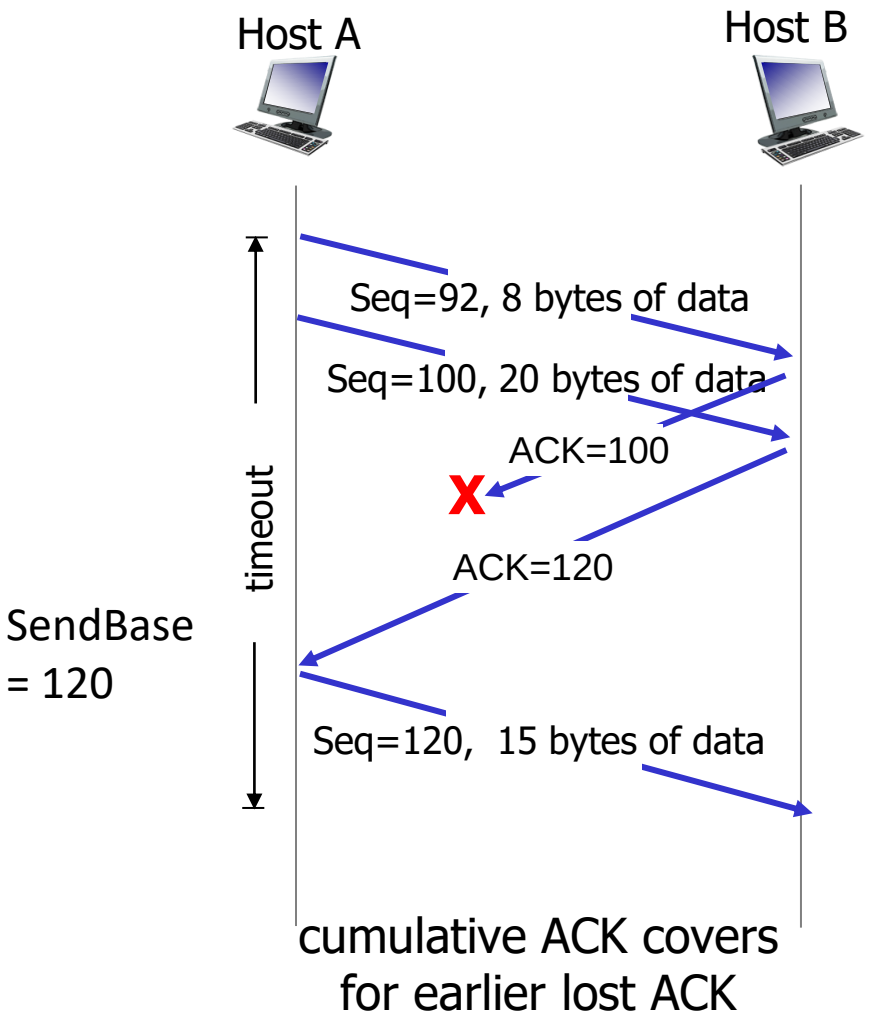


lost ACK scenario



premature timeout

TCP: retransmission scenarios



TCP fast retransmit

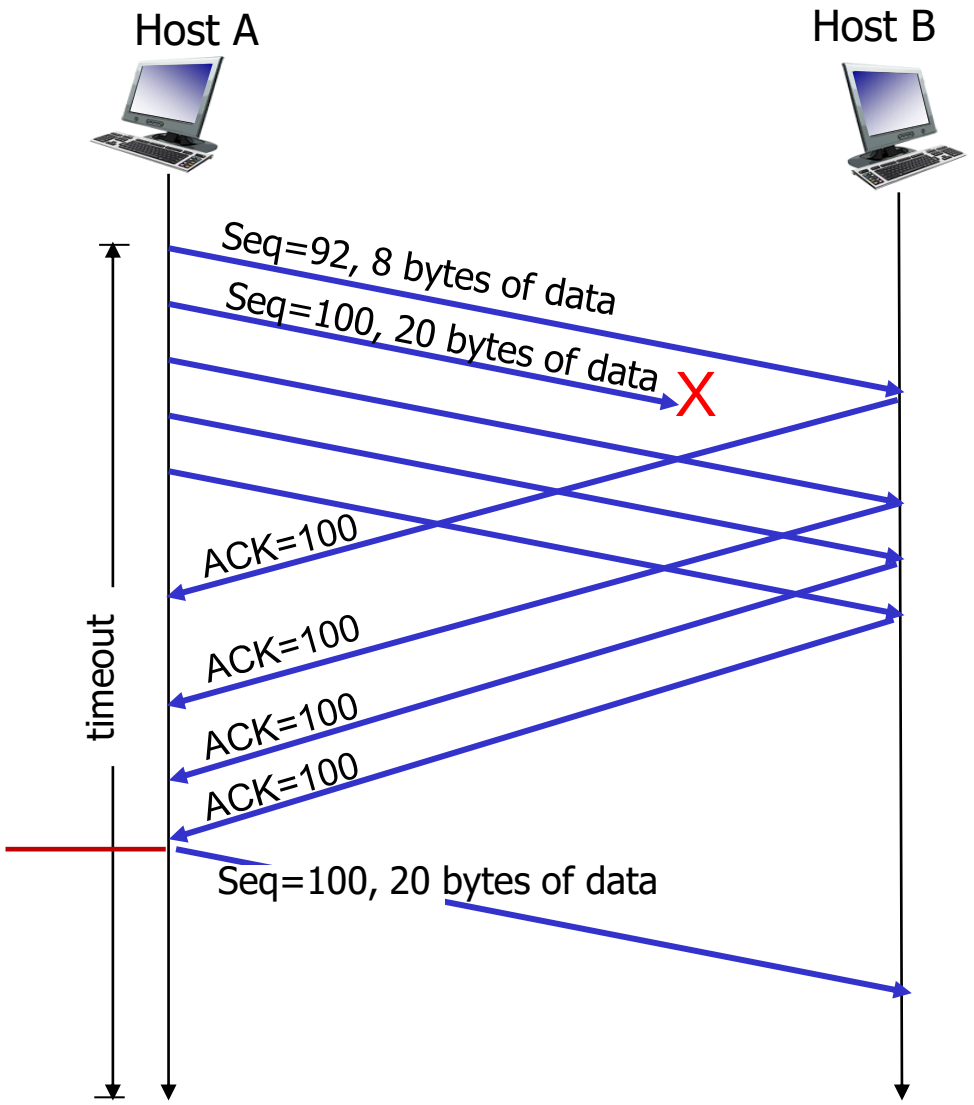
TCP fast retransmit

if sender receives 3 additional ACKs for same data ("triple duplicate ACKs"), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout



Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



Fast retransmit algorithm:

```
event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase = y
                if (there are currently not-yet-acknowledged segments)
                    start timer
              }
            else {
                increment count of dup ACKs received for y
                if (count of dup ACKs received for y = 3) {
                    resend segment with sequence number y
                 }
```

a duplicate ACK for already ACKed segment

fast retransmit

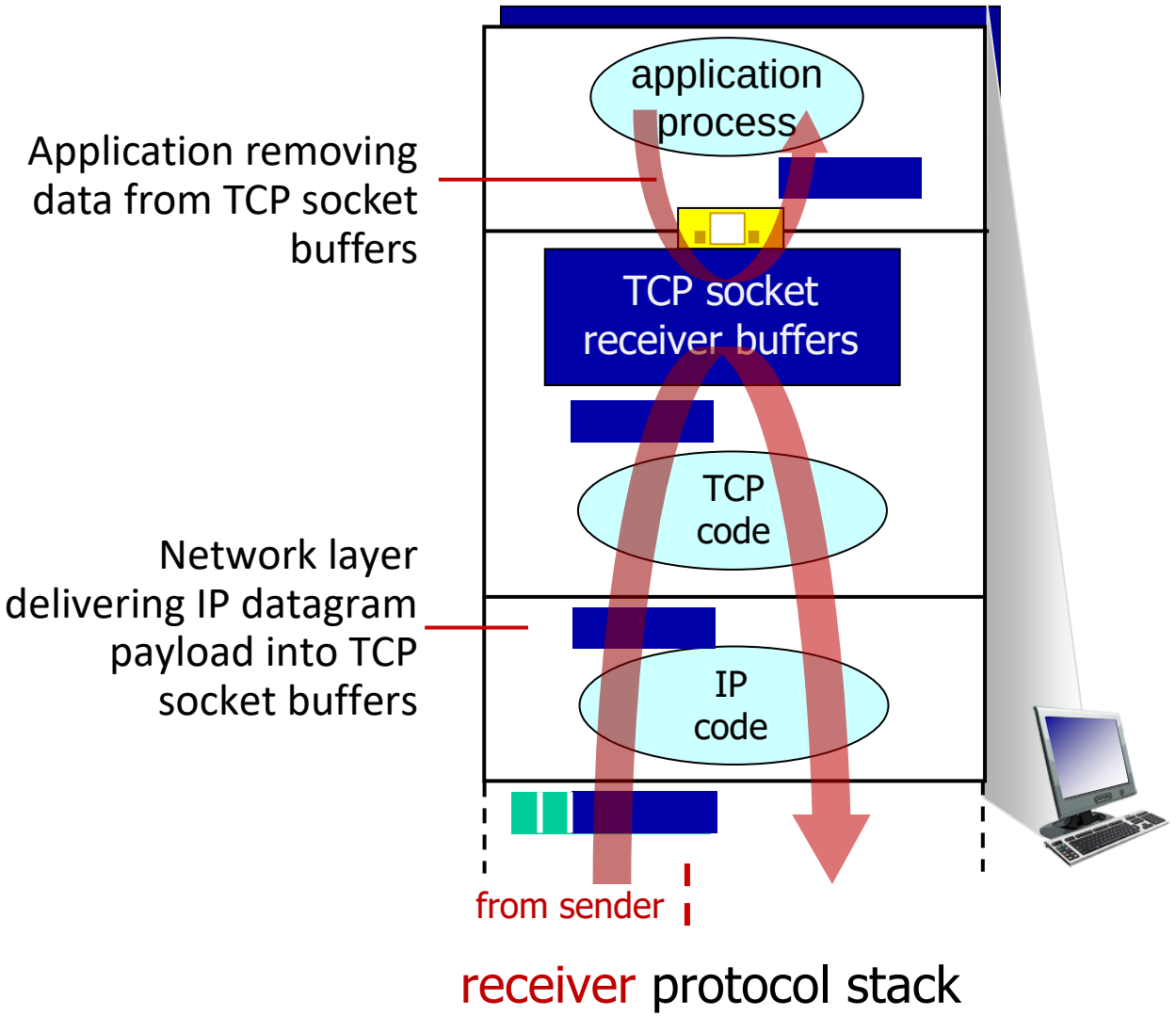
Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control



TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

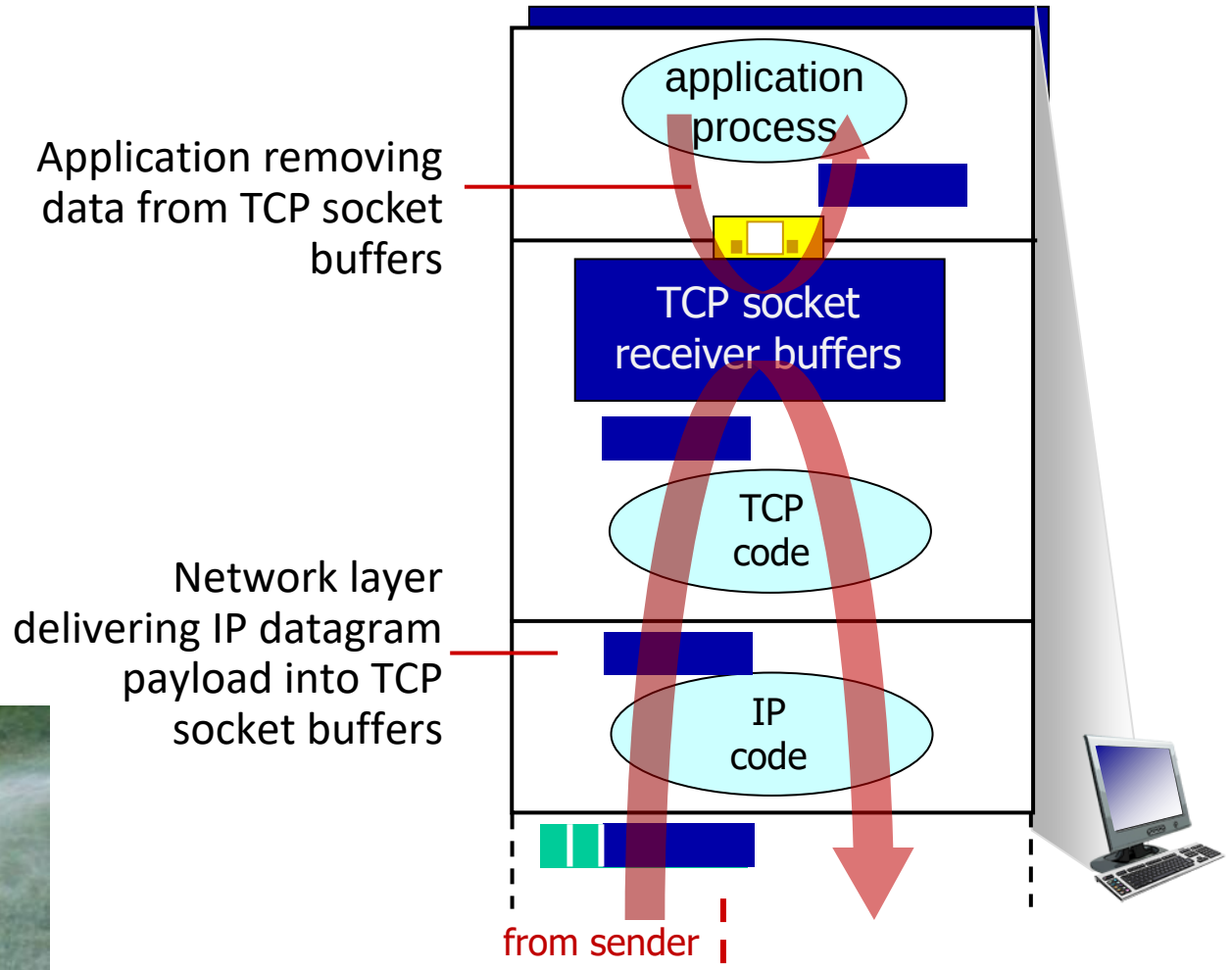


TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



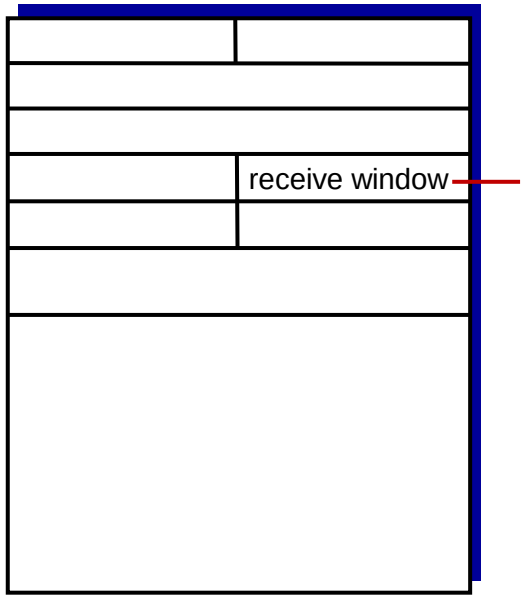
"no one can drink from a firehose"



receiver protocol stack

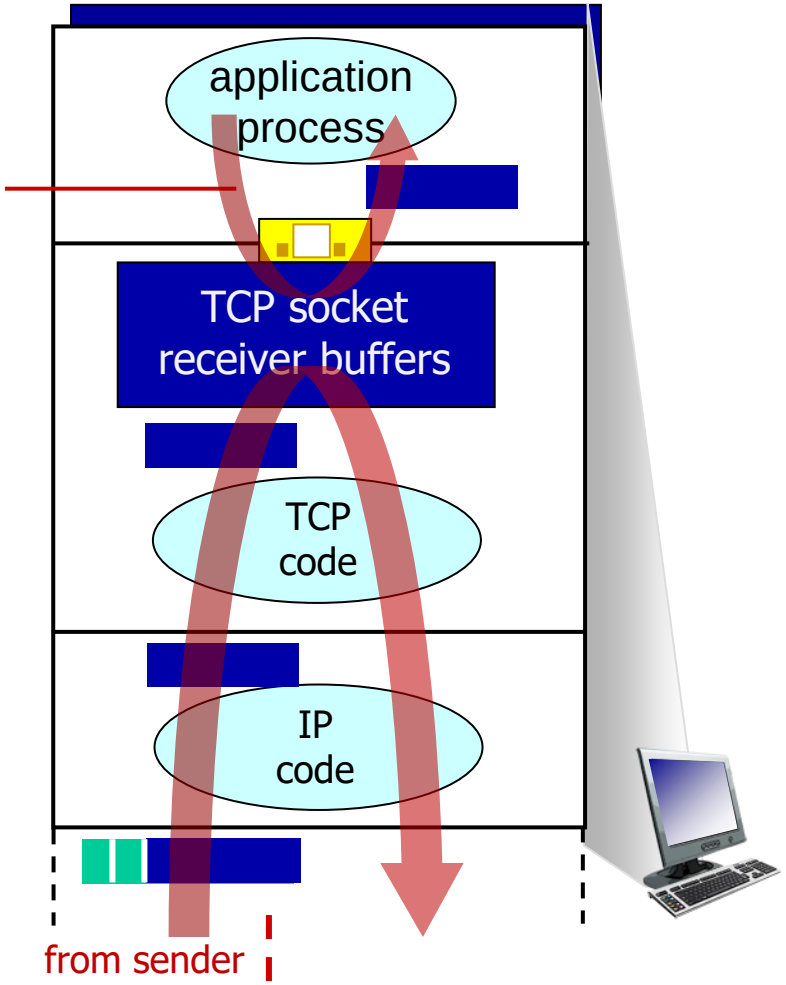
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



flow control: # bytes receiver willing to accept

Application removing data from TCP socket buffers



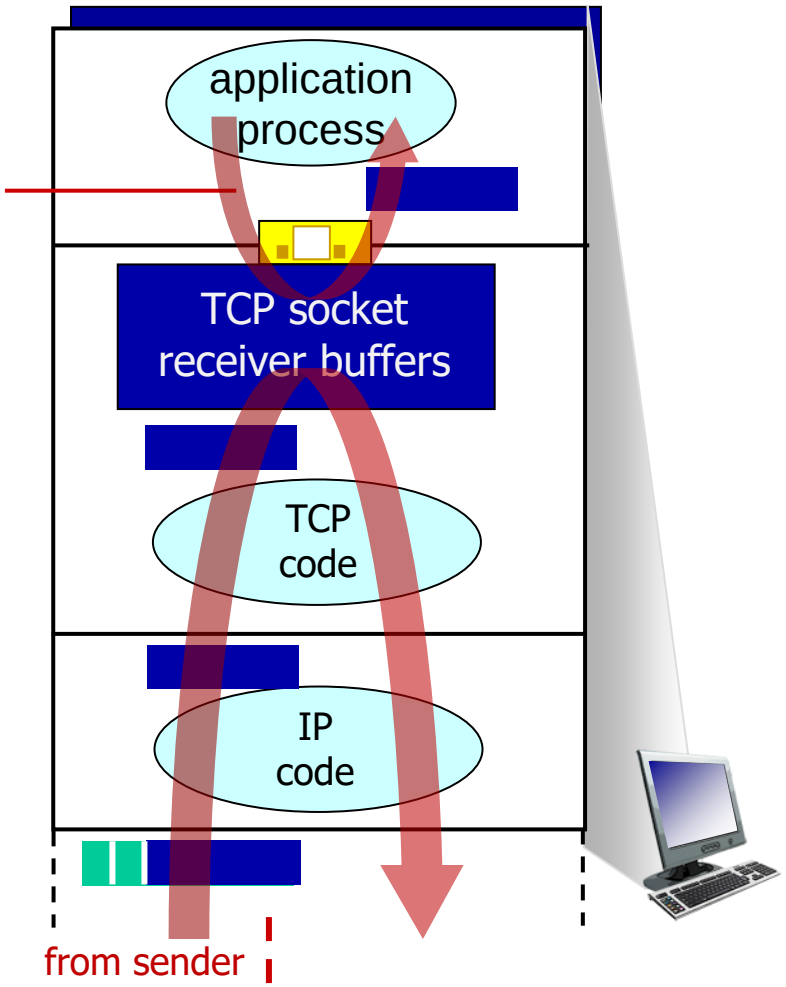
receiver protocol stack

TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast

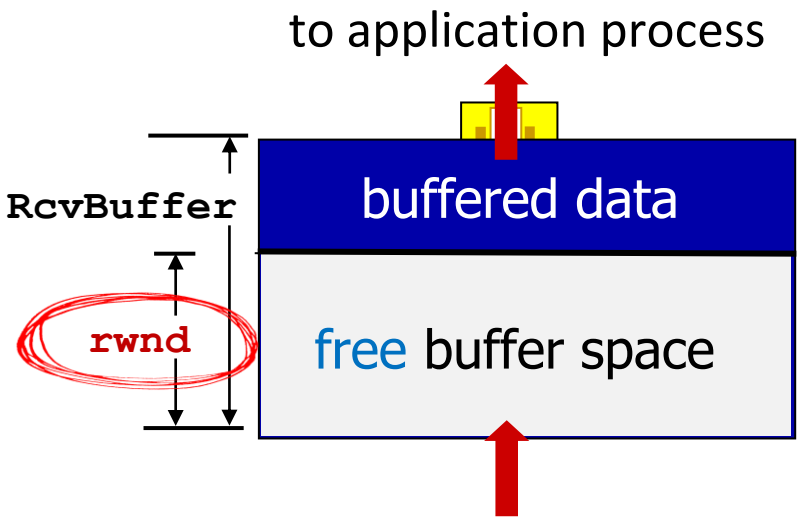


receiver protocol stack

TCP flow control

spare room in buffer at TCP receiver-side
= **RcvWindow**
= **RcvBuffer-[LastByteRcvd - LastByteRead]**

- TCP receiver "advertises" free buffer space in **rwnd** field in TCP header
  - **RcvBuffer** size set via socket options (typical default is **4096 bytes**)
  - many operating systems (OSes) auto-adjust **RcvBuffer**
- sender limits amount of unACKed ("in-flight") data to received **rwnd**
  - guarantees receive buffer will not overflow
  - **LastByteSent - LastByteAcked ≤ rwnd**

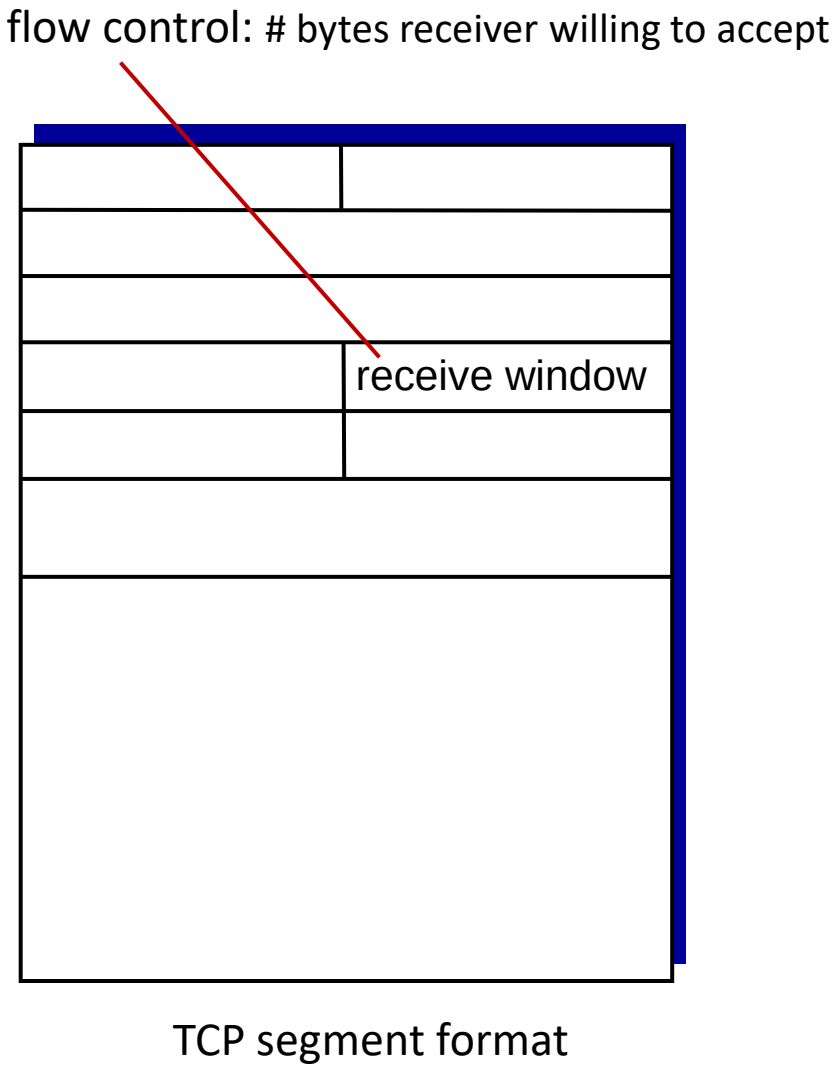


TCP receiver-side buffering

TCP flow control

Initially, receiver sets **`rwnd = RcvBuffer`**.

- TCP receiver "advertises" free buffer space in **`rwnd`** field in TCP header
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust **`RcvBuffer`**
- sender limits amount of unACKed ("in-flight") data to received **`rwnd`**
  - guarantees receive buffer will not overflow
  - **`LastByteSent - LastByteAcked`** $\leq$ **`rwnd`**

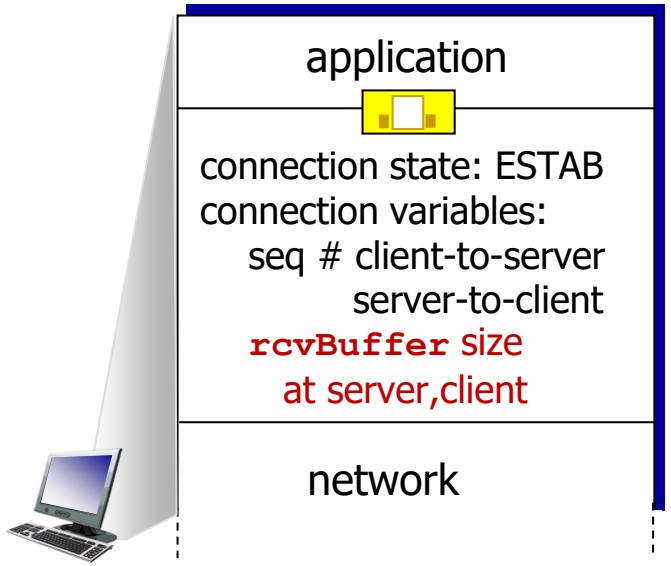


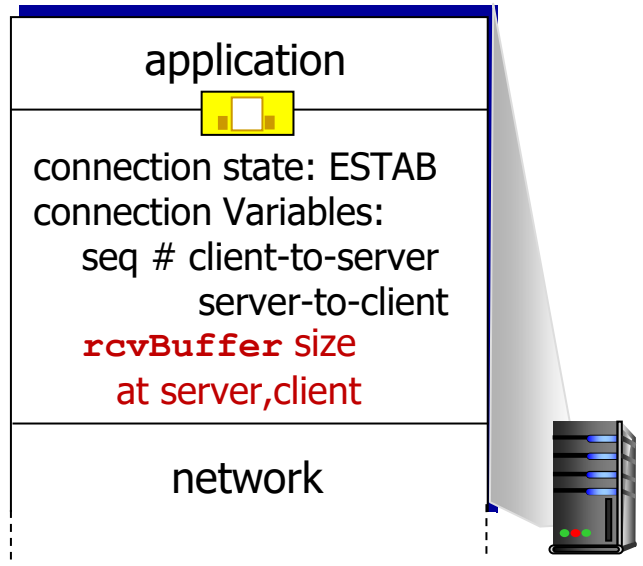
TCP segment format

TCP connection management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters (e.g., starting seq #s)



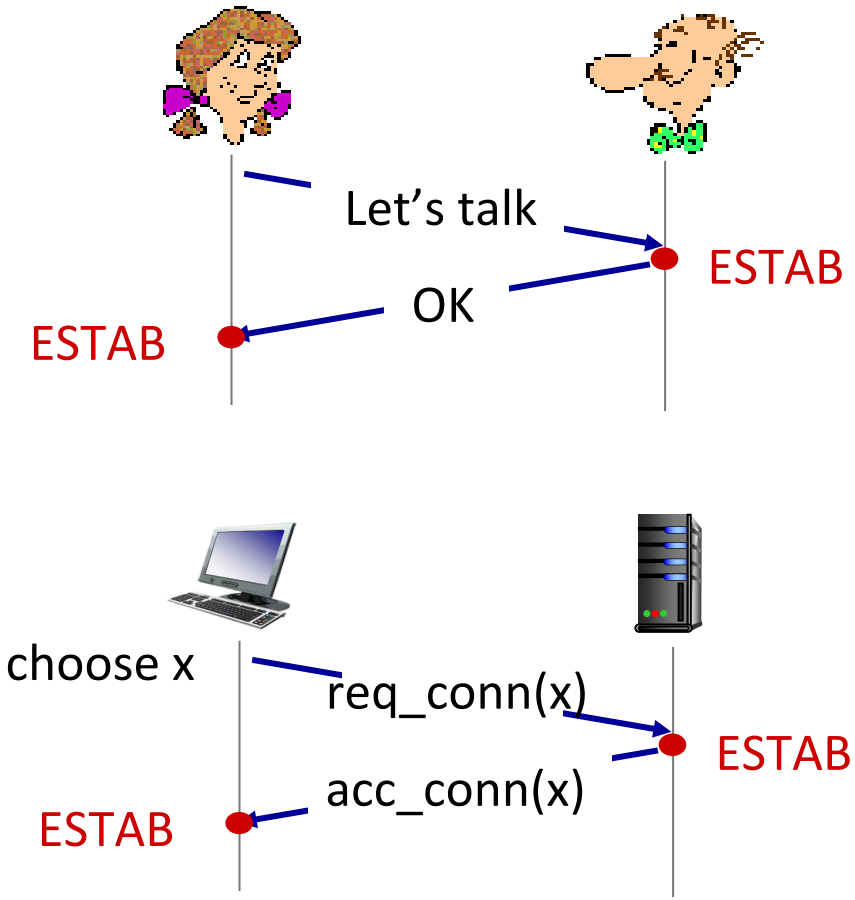



```
Socket clientSocket =
  newSocket("hostname","port number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

Agreeing to establish a connection

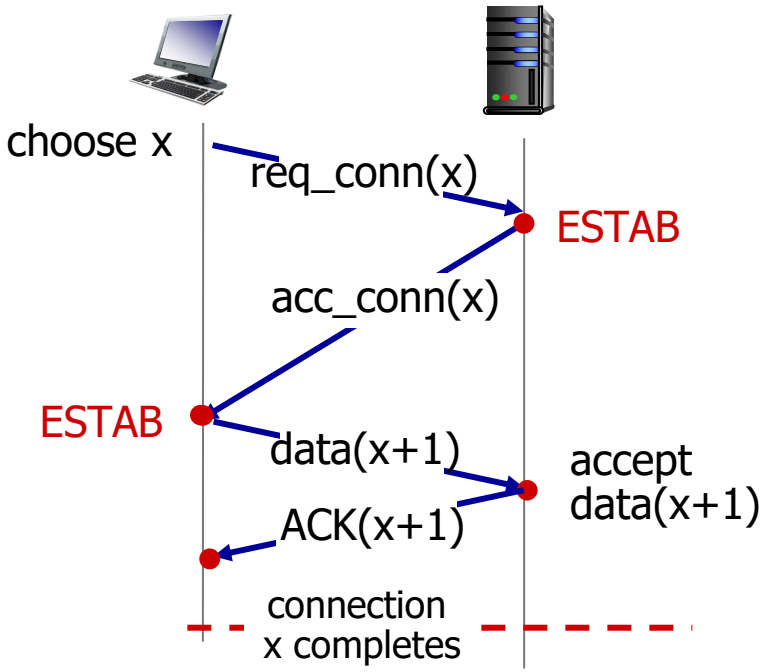
2-way handshake:



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

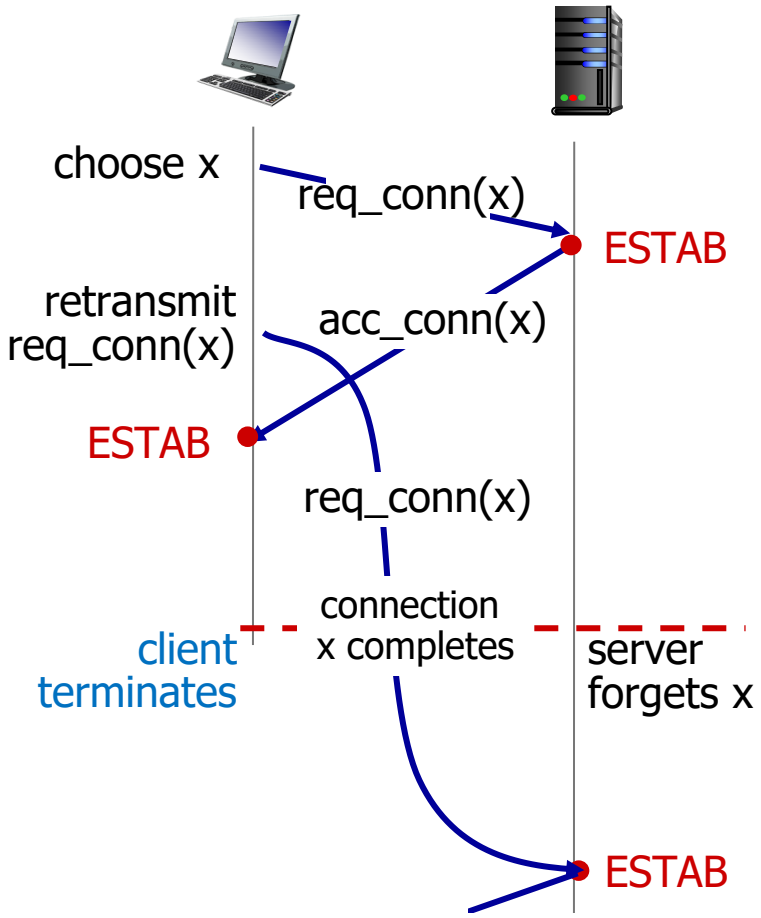
2-way handshake scenarios



No problem!

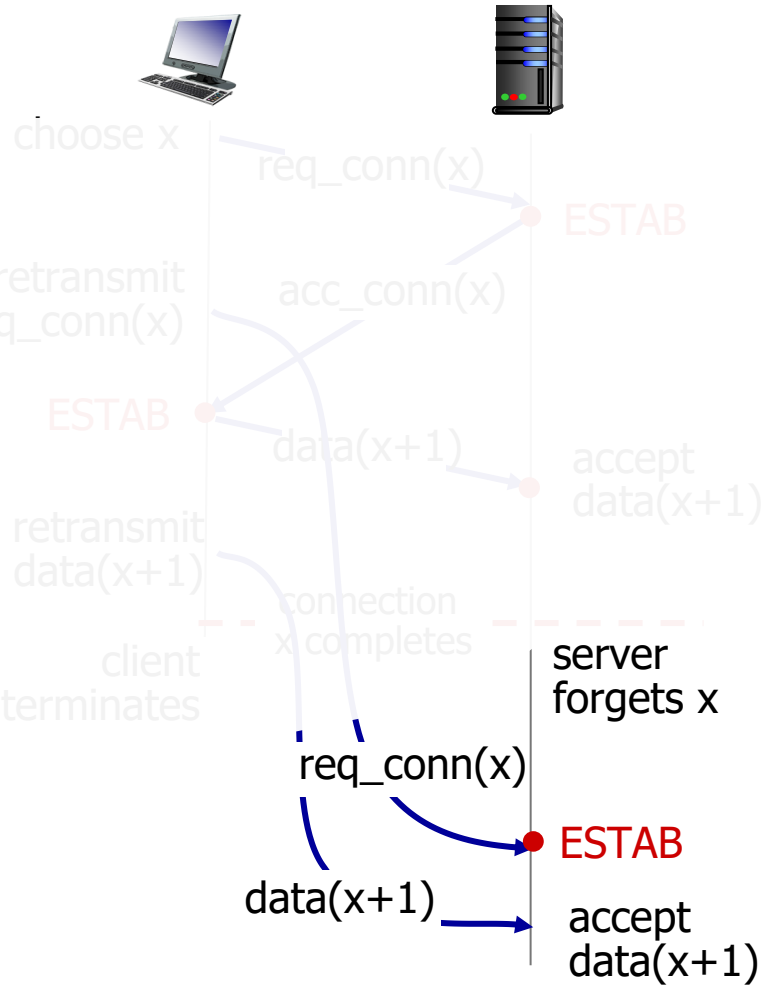


2-way handshake scenarios



Problem: half open connection! (no client)

2-way handshake scenarios



Problem: dup data accepted!

TCP 3-way handshake

Client/server initial seq #, i.e. x, y, are randomly chosen.

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName,serverPort))
```



choose init seq num, x
send TCP SYN msg

ACKbit=0, no app data

SYNSENT

SYNbit=1, Seq=x

choose init seq num, y
send TCP SYNACK
msg, acking SYN



no app data

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data

ESTAB

ACKbit=1, ACKnum=y+1

with/without app data

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

LISTEN

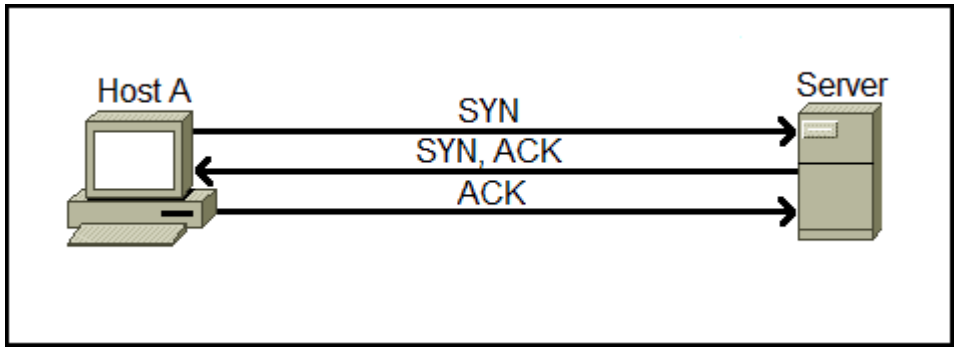
SYN RCVD

ESTAB

A human 3-way handshake protocol



TCP Connection Management



Recall: TCP sender, receiver establish "connection" before exchanging data segments

- initialize TCP variables:
  - seq. #s
  - buffers, flow control info (e.g. **`RcvWindow`**)
- *client:* connection initiator

```
Socket clientSocket = new
Socket("hostname","port number");
```

- *server:* contacted by client

```
Socket connectionSocket =
welcomeSocket.accept();
```

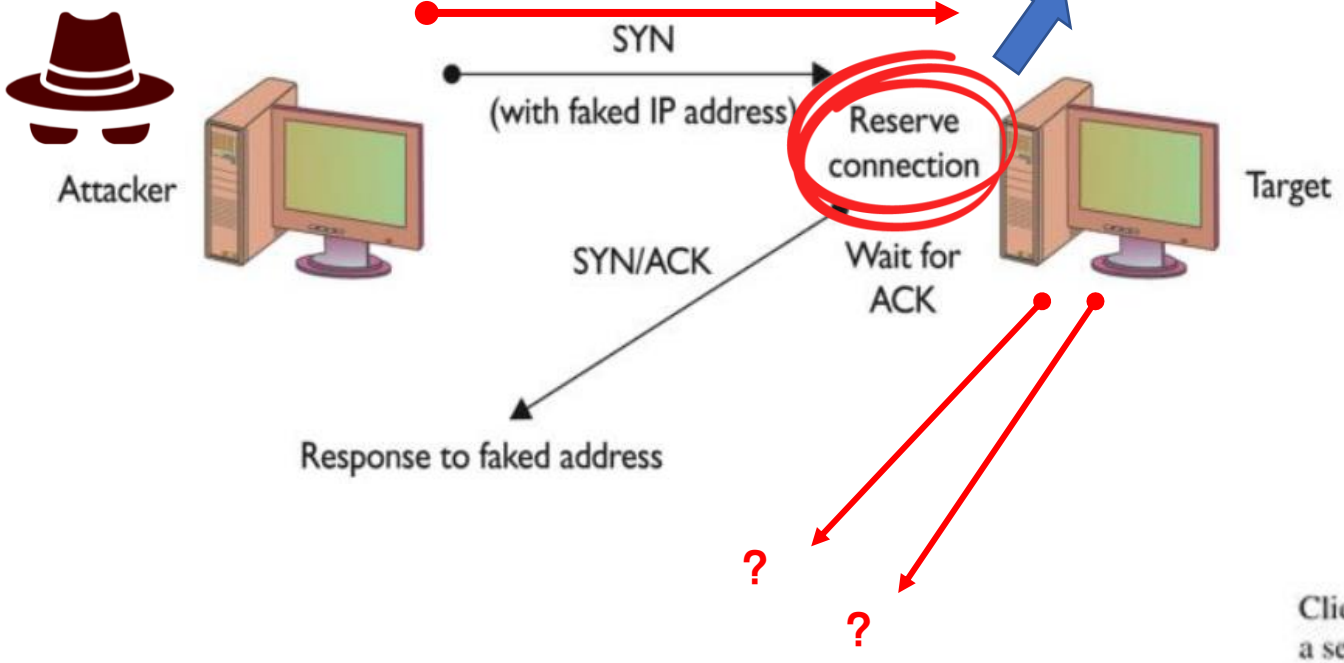
Three way handshake:

Step 1: client host sends TCP SYN segment to server
- specifies initial seq #
- no data

Step 2: server host receives SYN, replies with SYNACK segment
- server allocates buffers, no data
- specifies server initial seq. #

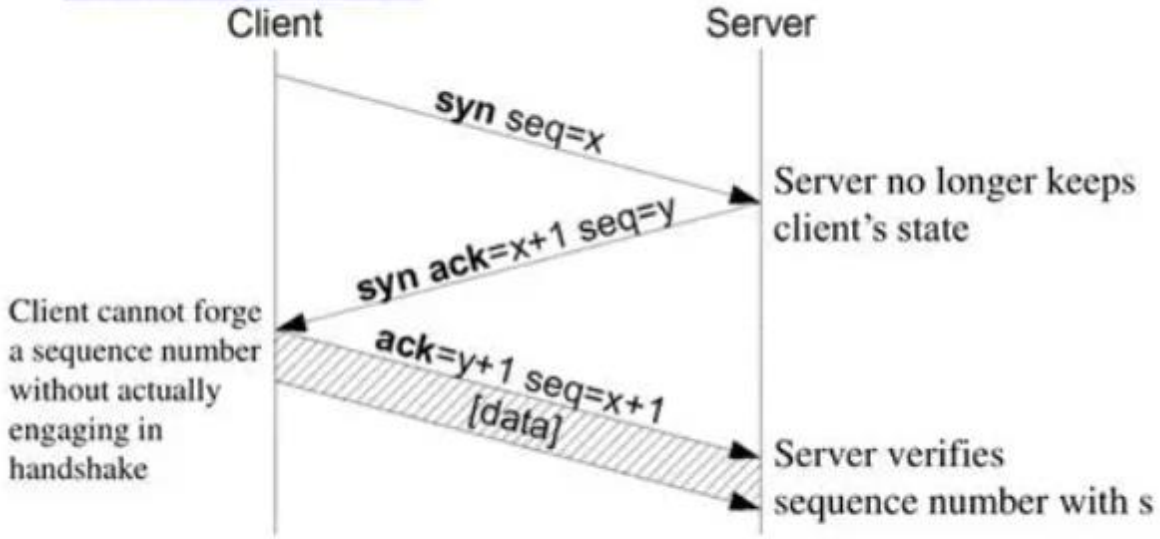
Step 3: client receives SYNACK, replies with ACK segment, which may contain data

SYN Flood Attack



The server delay the resource allocation

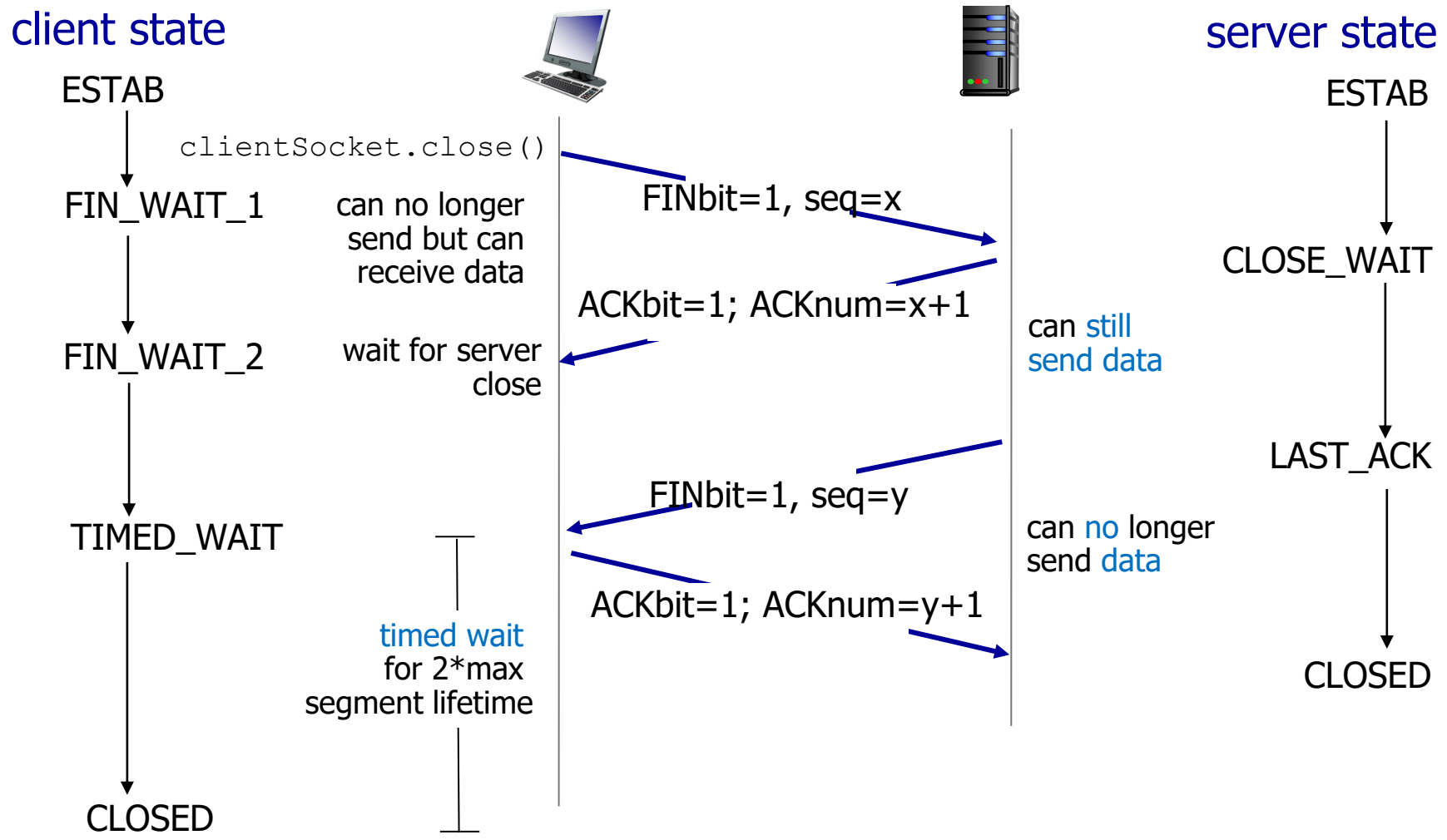
SYN Cookie



y = H(source, destination, secret, counter) computed by server
s is secret known only by server
H is a one-way hash function



Closing a TCP connection



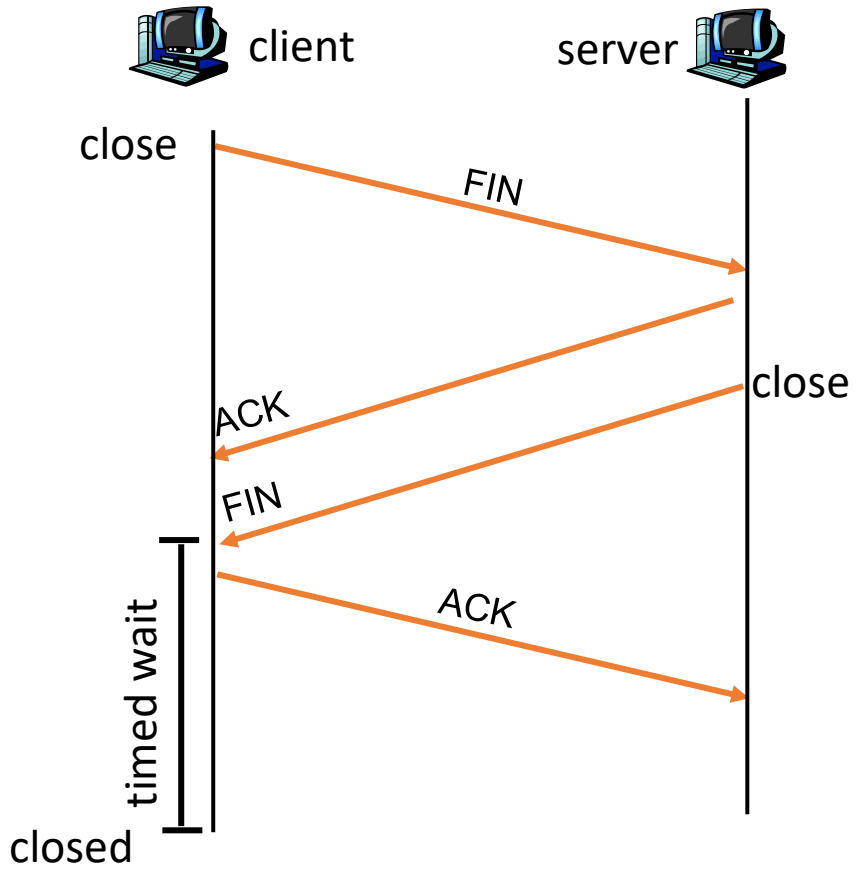
TCP Connection Management (cont.)

Closing a connection:

client closes socket:
`clientSocket.close();`

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.



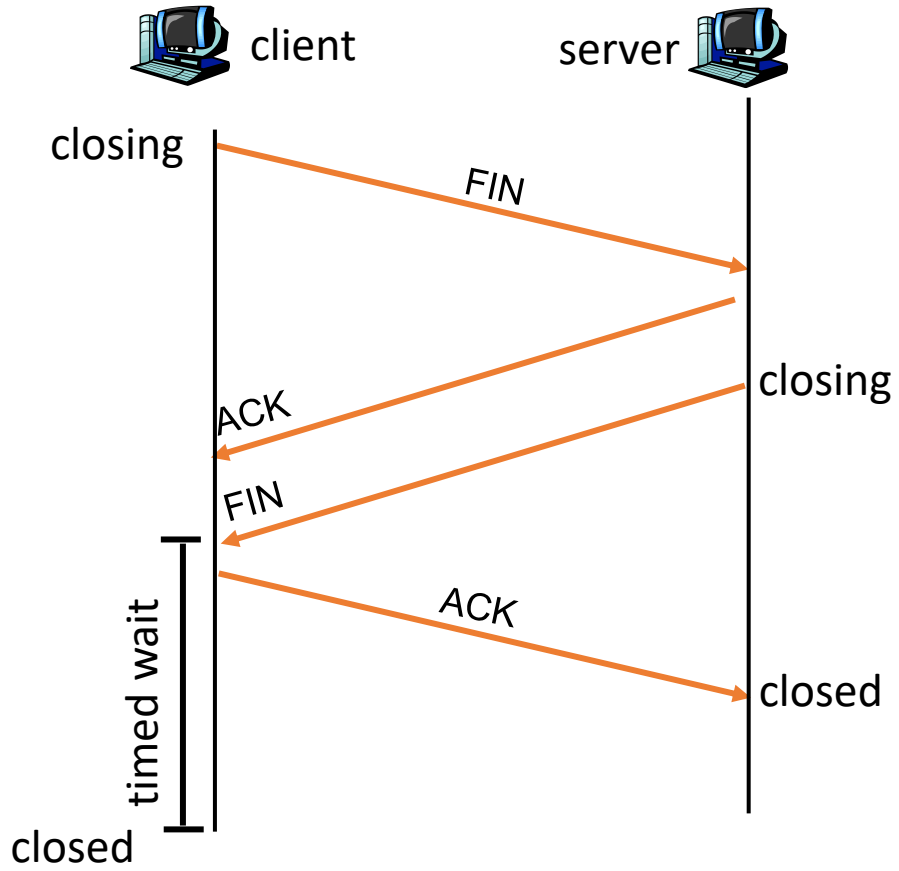
TCP Connection Management (cont.)

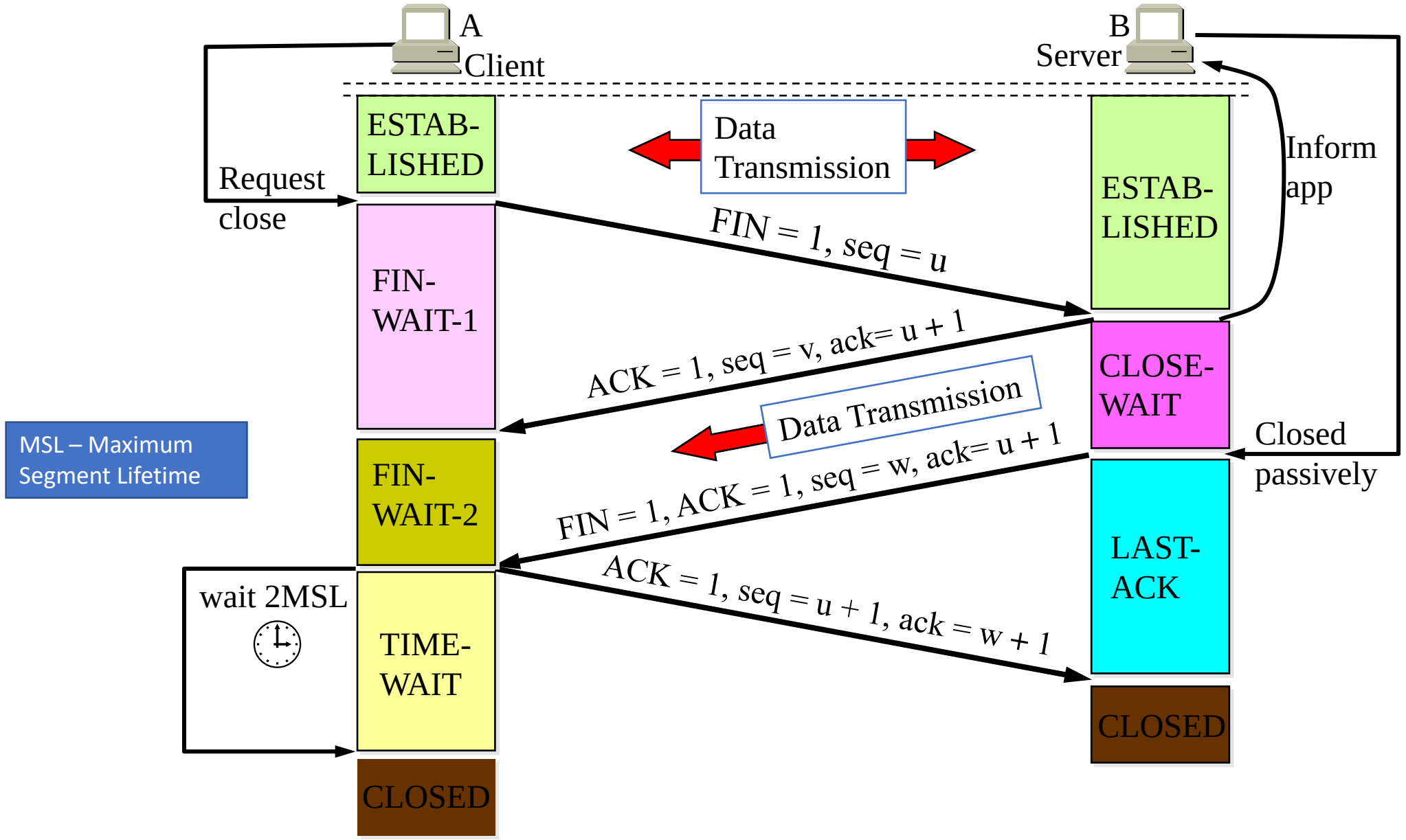
Step 3: client receives FIN, replies with ACK.

- Enters "timed wait" - will respond with ACK to received FINs
- Wait for timed wait interval for outstanding packets to get out of the network

Step 4: server, receives ACK. Connection closed.

Note: with small modification, can handle simultaneous FINs.




A
Client
B
Server
ESTAB-
LISHED
Data
Transmission
ESTAB-
LISHED
Inform
app
Request
close
FIN = 1, seq = u
FIN-
WAIT-1
ACK = 1, seq = v, ack= u + 1
CLOSE-
WAIT
Data Transmission
MSL – Maximum
Segment Lifetime
FIN = 1, ACK = 1, seq = w, ack= u + 1
Closed
passively
FIN-
WAIT-2
LAST-
ACK
wait 2MSL
ACK = 1, seq = u + 1, ack = w + 1
TIME-
WAIT
CLOSED
CLOSED

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- **Principles of congestion control**
- TCP congestion control
- Evolution of transport-layer functionality



Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
  - long delays (queueing in router buffers)
  - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!







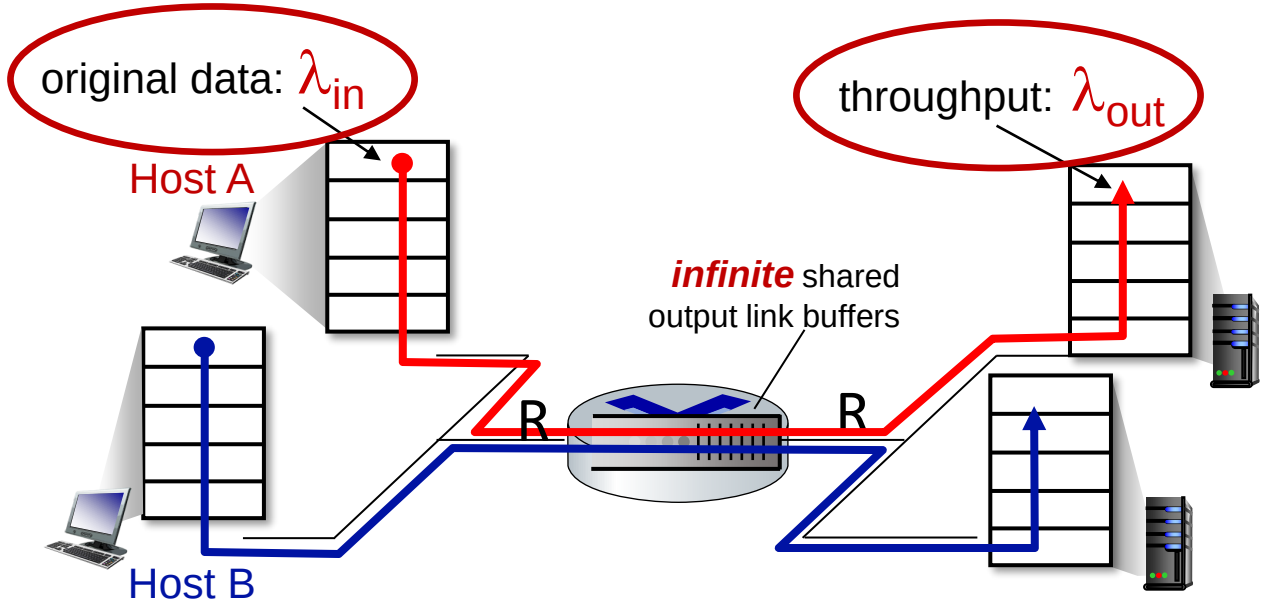
congestion control:
too many senders,
sending too fast

flow control: one sender
too fast for one receiver

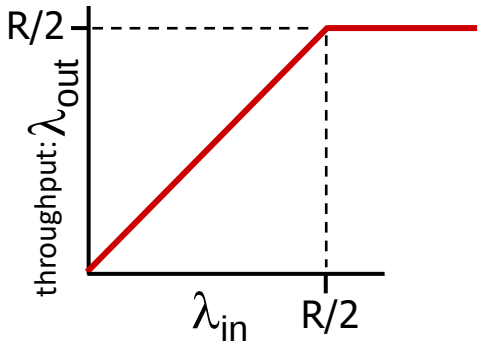
Causes/costs of congestion: scenario 1

Simplest scenario:

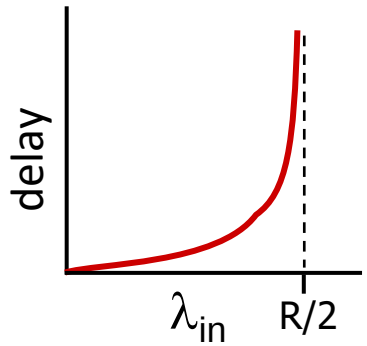
- one router, infinite buffers
- input, output link capacity: R
- two flows (two senders, two receivers)
- no retransmissions needed



Q: What happens as arrival rate λ_{in} approaches R/2?



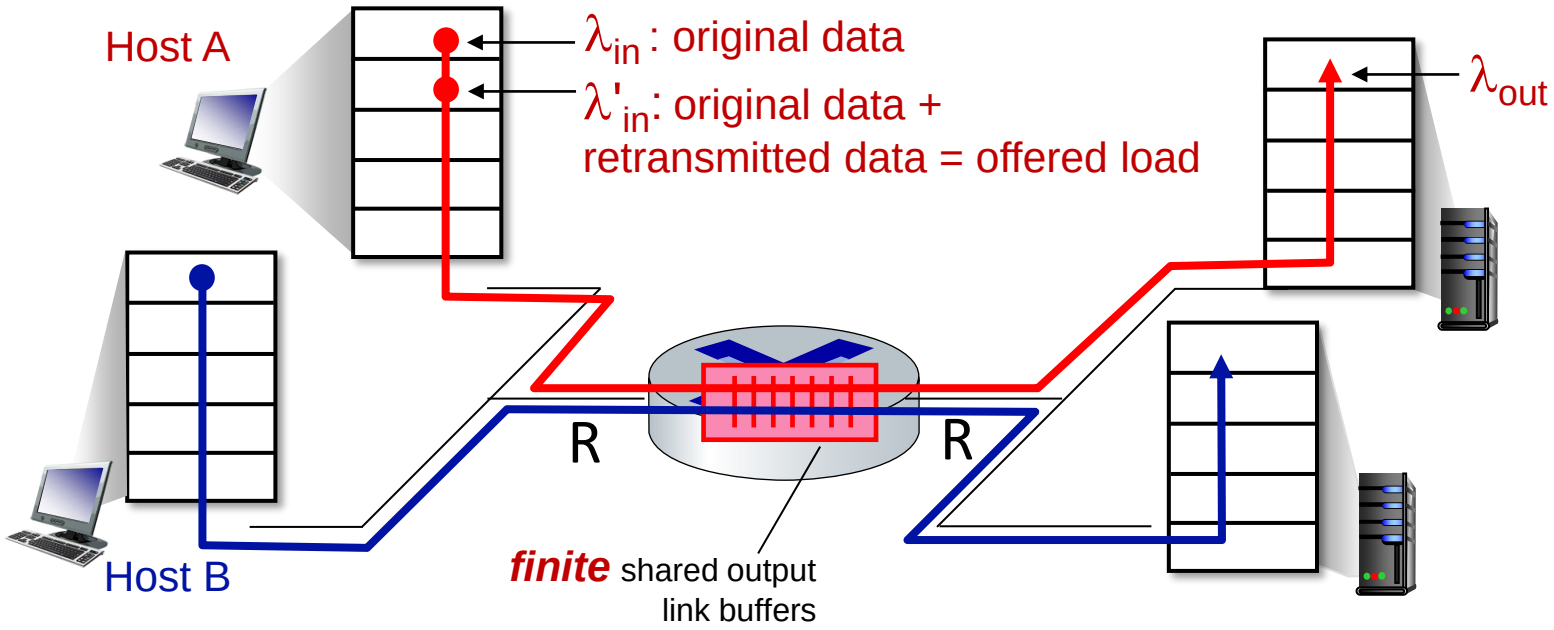
maximum per-connection throughput: R/2



large (queuing) delays as arrival rate λ_{in} approaches capacity

Causes/costs of congestion: scenario 2

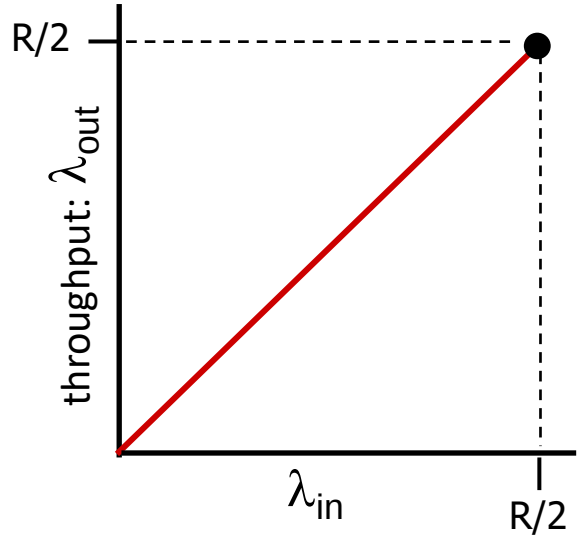
- one router, *finite* buffers
- sender retransmits lost, timed-out packet
  - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
  - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$

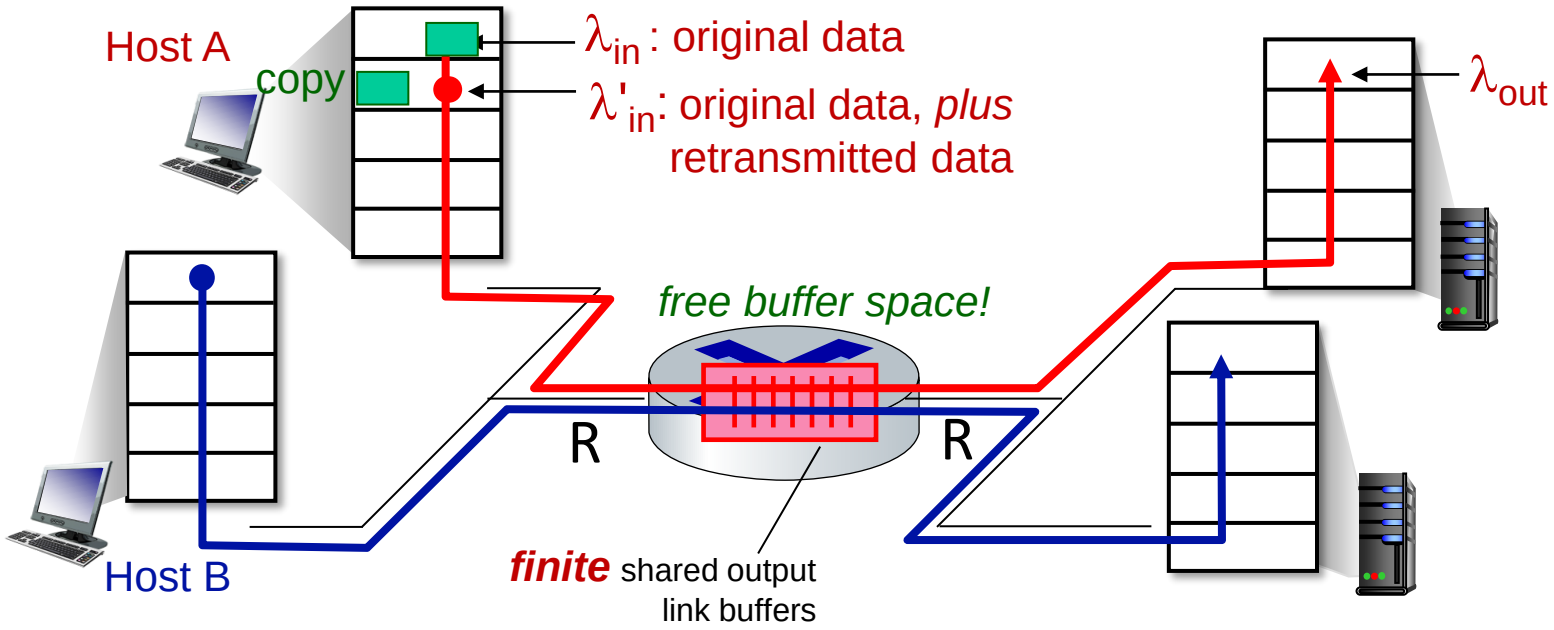


Causes/costs of congestion: scenario 2

Idealization: perfect knowledge

- sender sends only when router buffers available

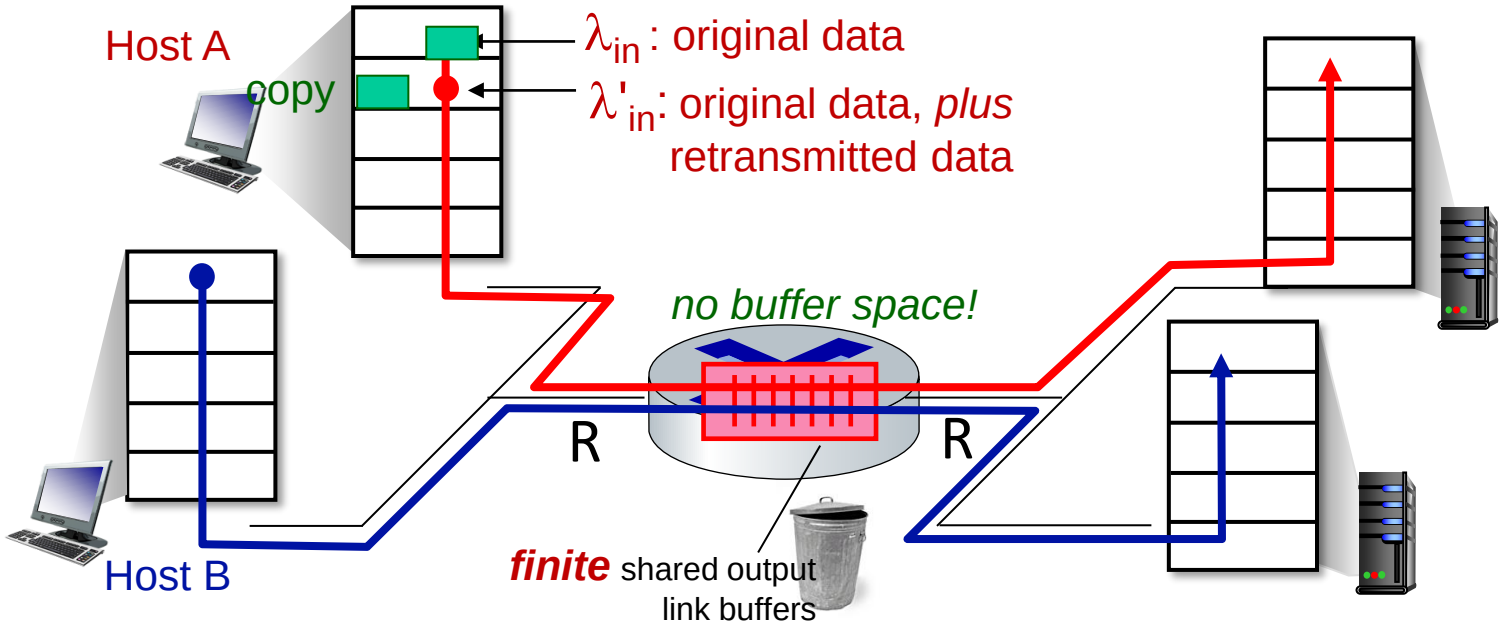




Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

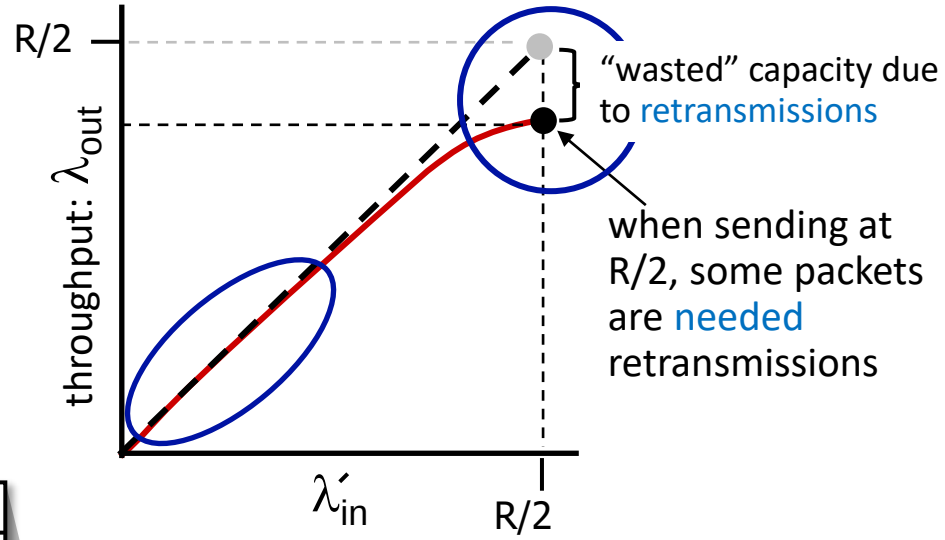
- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost

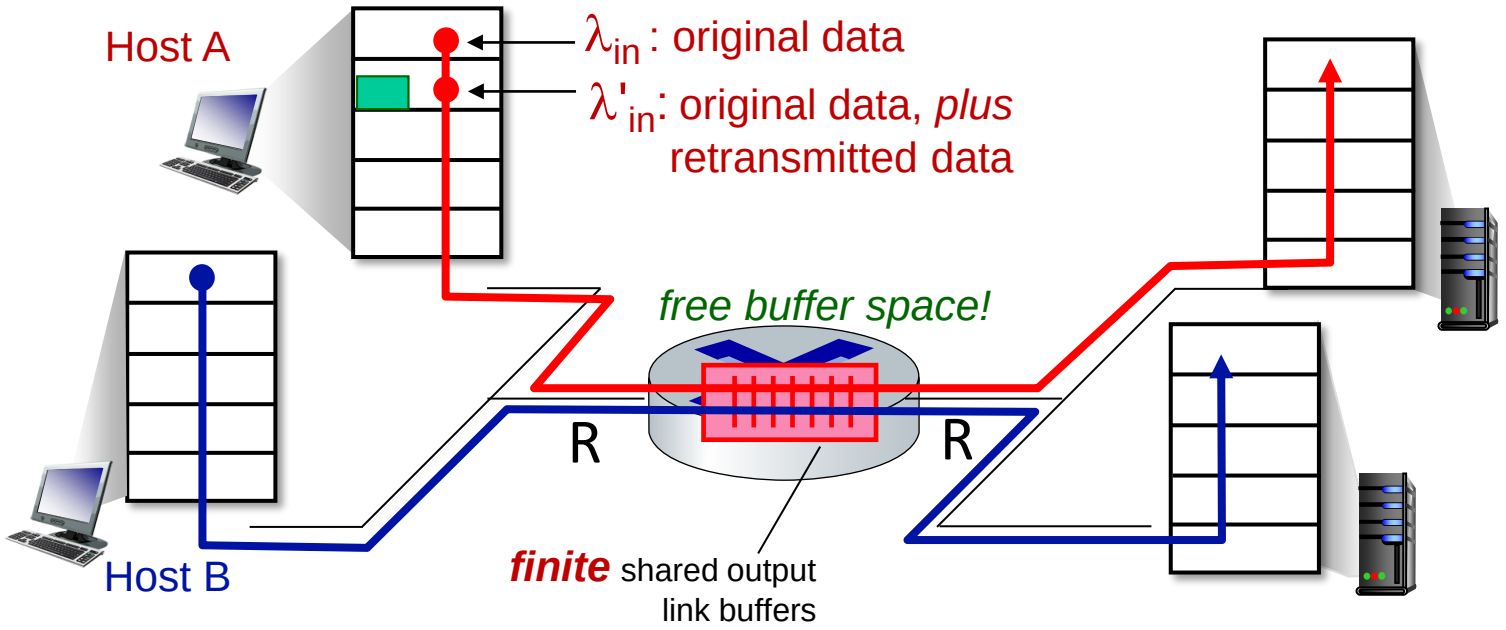


Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost

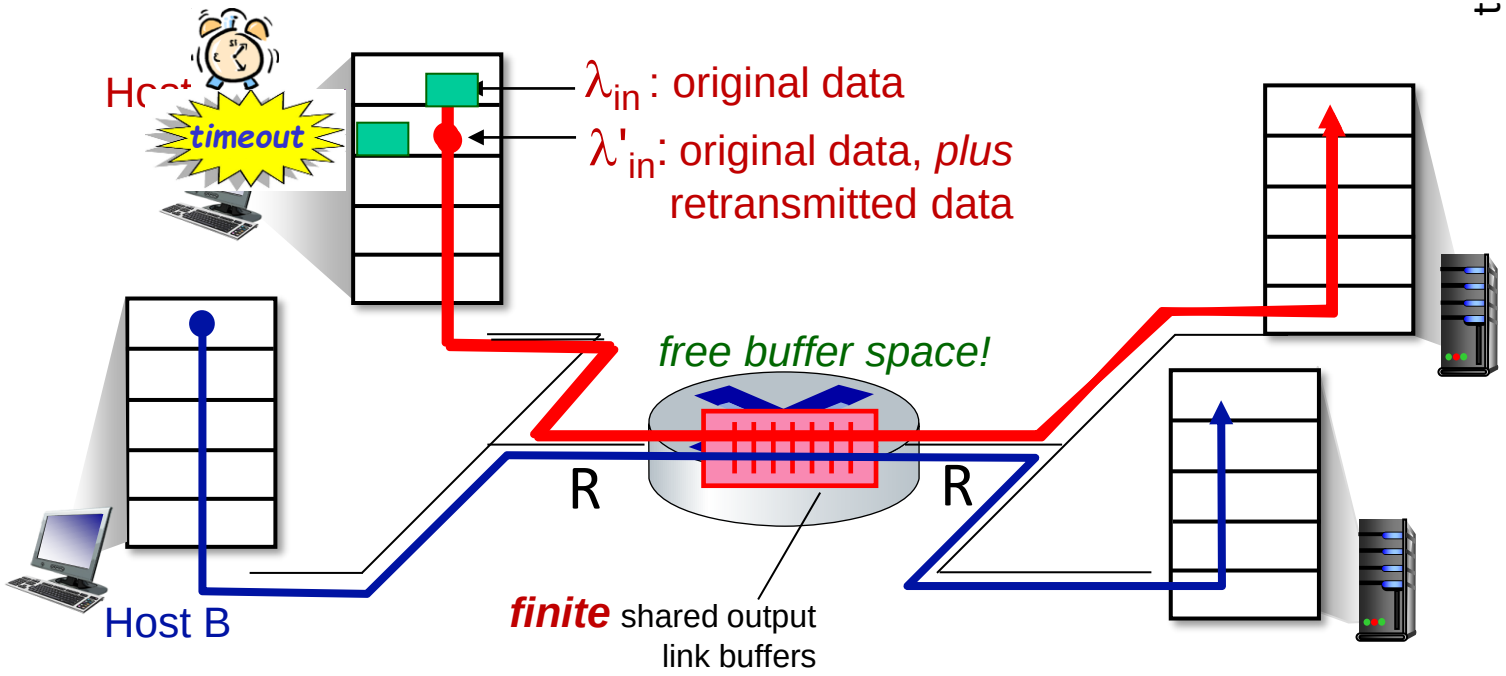


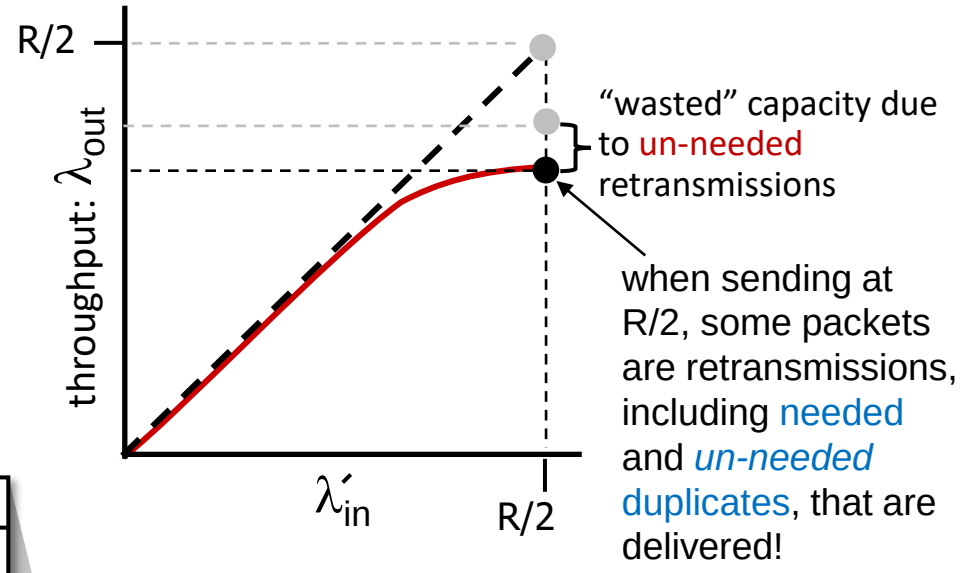


Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered

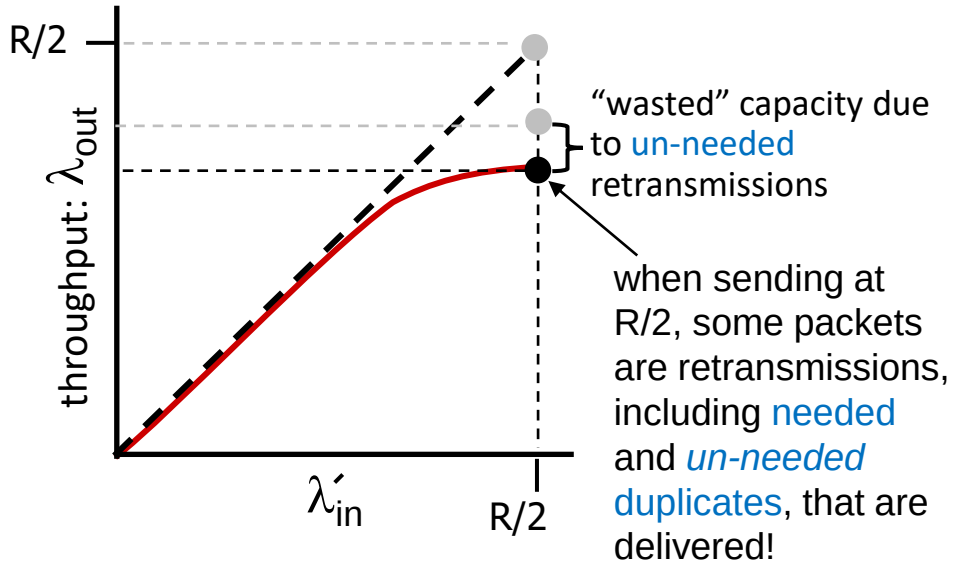




Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered



"costs" of congestion:

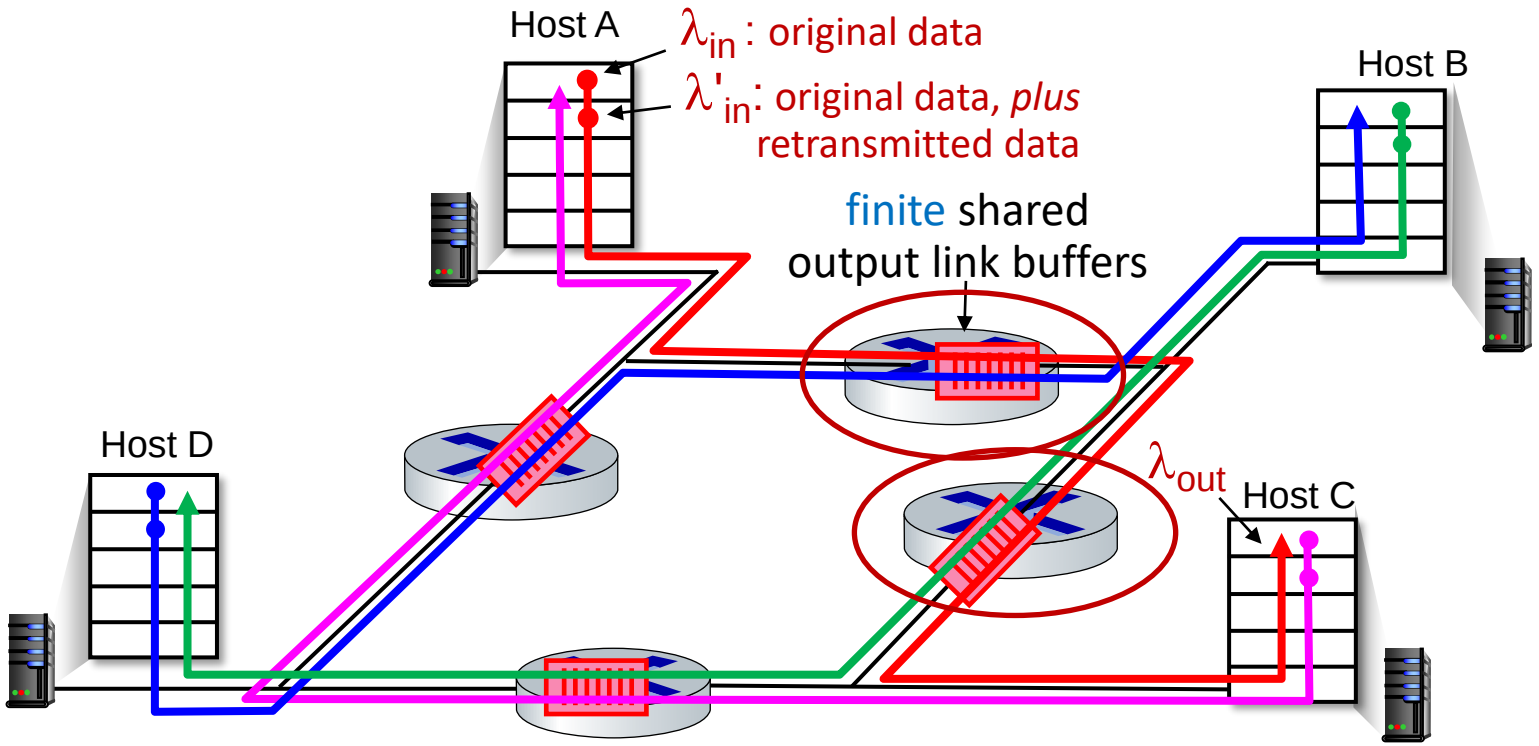
- more work (retransmission) for given receiver throughput
- unneeded retransmissions: link carries multiple copies of a packet
  - decreasing maximum achievable throughput

Causes/costs of congestion: scenario 3

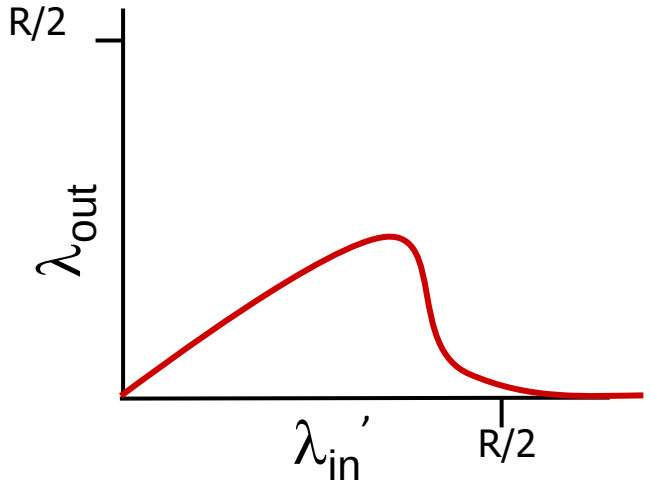
- *four* senders with same λ_{in}
- *multi-hop* paths
- timeout/retransmit

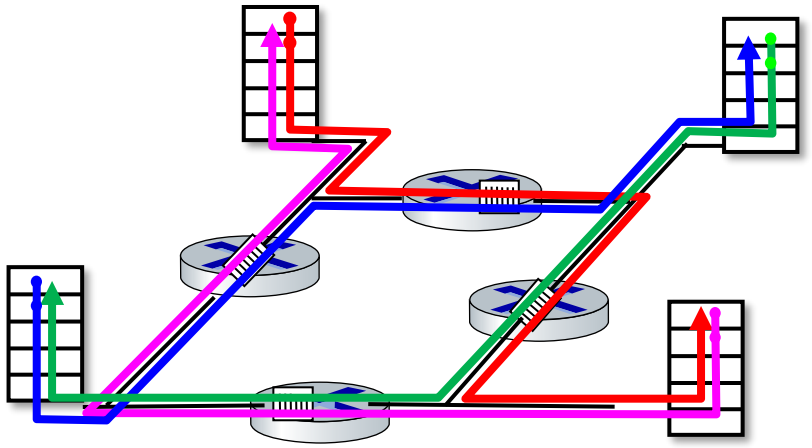
Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper queue are dropped, blue throughput → 0



Causes/costs of congestion: scenario 3



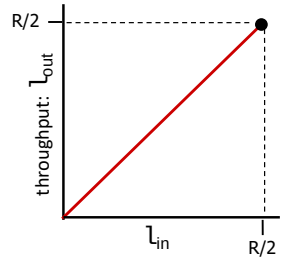


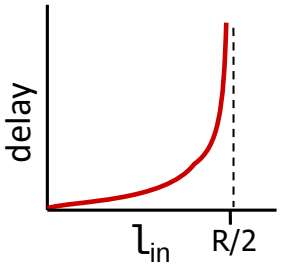
another "cost" of congestion:

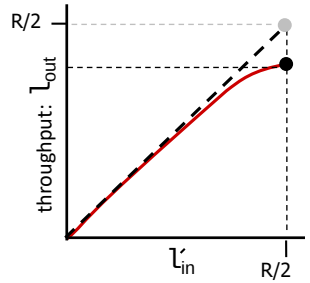
- when packet dropped, any upstream transmission capacity and buffering used for that packet was wasted!

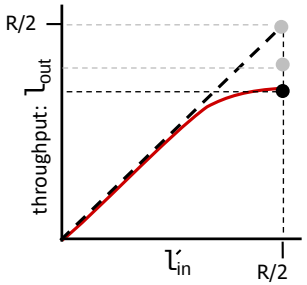
Causes/costs of congestion: insights

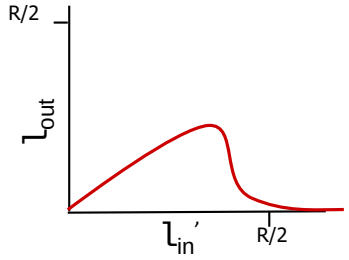
- throughput can never exceed capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream



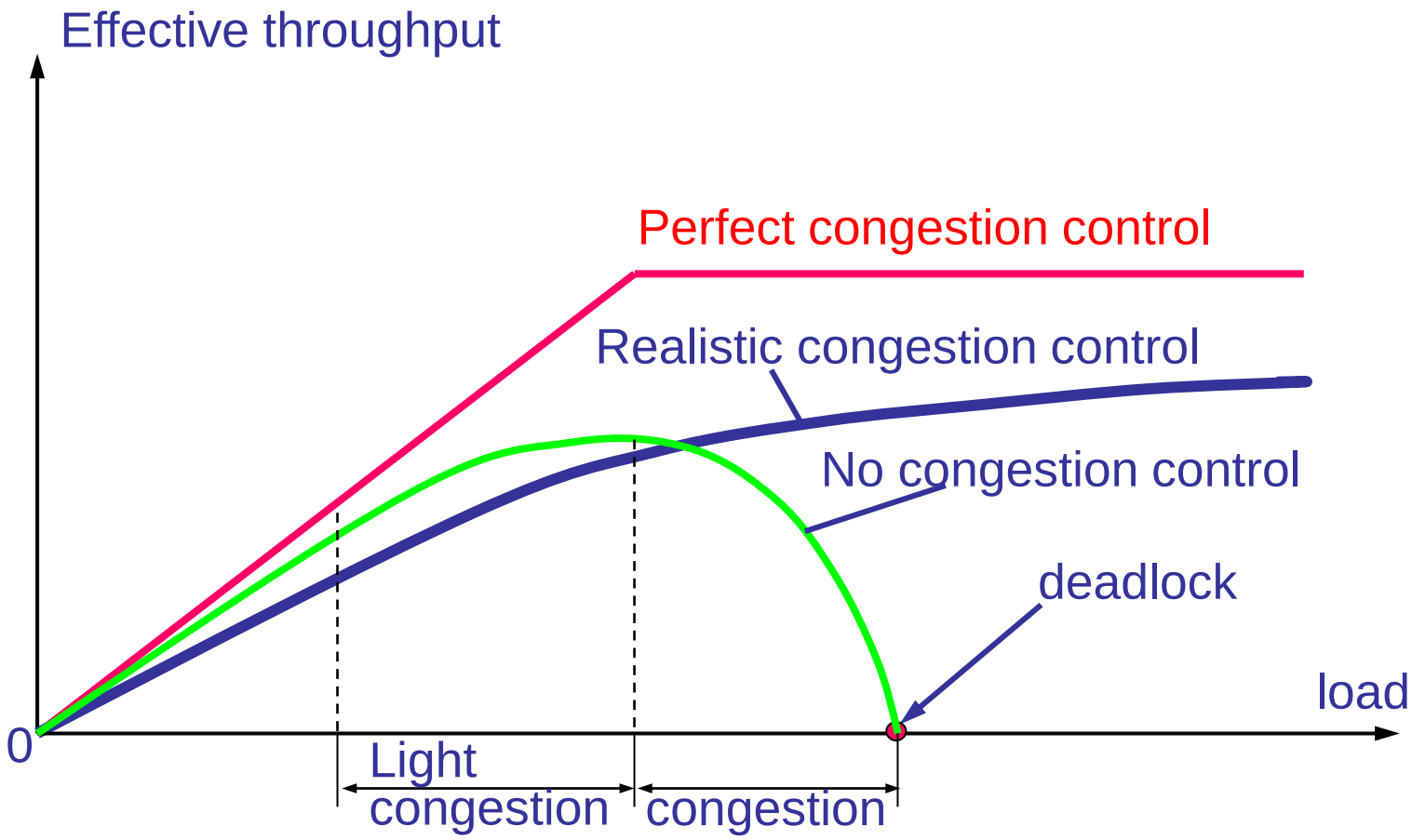








Function of congestion control



Congestion Collapse

- Definition: *Increase in network load results in decrease of useful work done*
- Many possible causes
  - Retransmissions of packets still in flight
  - Undelivered packets
    - Packets consume resources and are dropped elsewhere in network
  - Fragments
    - Mismatch of transmission and retransmission units
  - Control traffic
    - Large percentage of traffic is for control
  - Stale or unwanted packets
    - Packets that are delayed on long queues

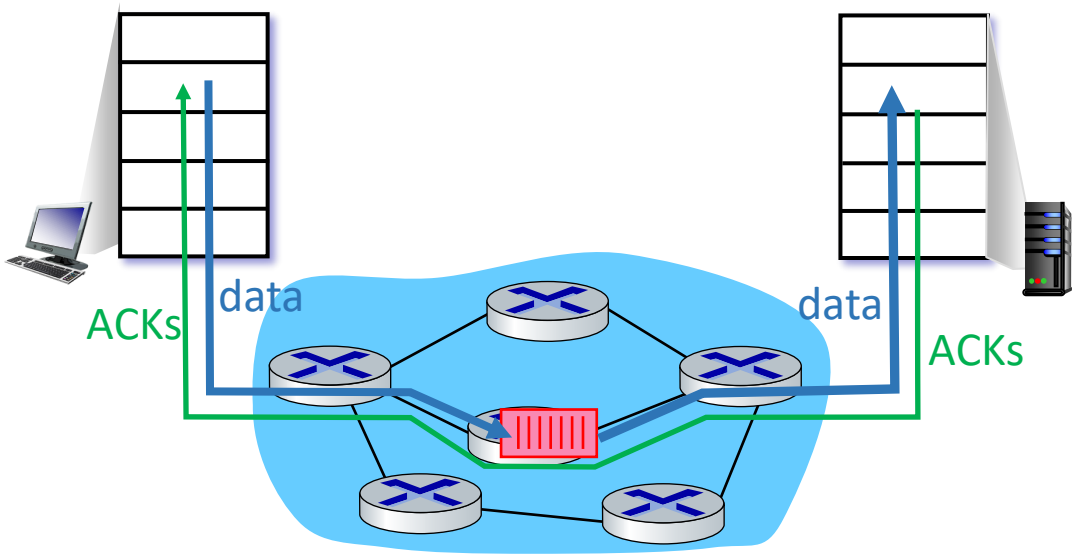
Approaches towards congestion control

1. End-end congestion control:

- no explicit feedback from network
- congestion *inferred* from observed loss, delay

three duplicate ACKs

delayed ACKs

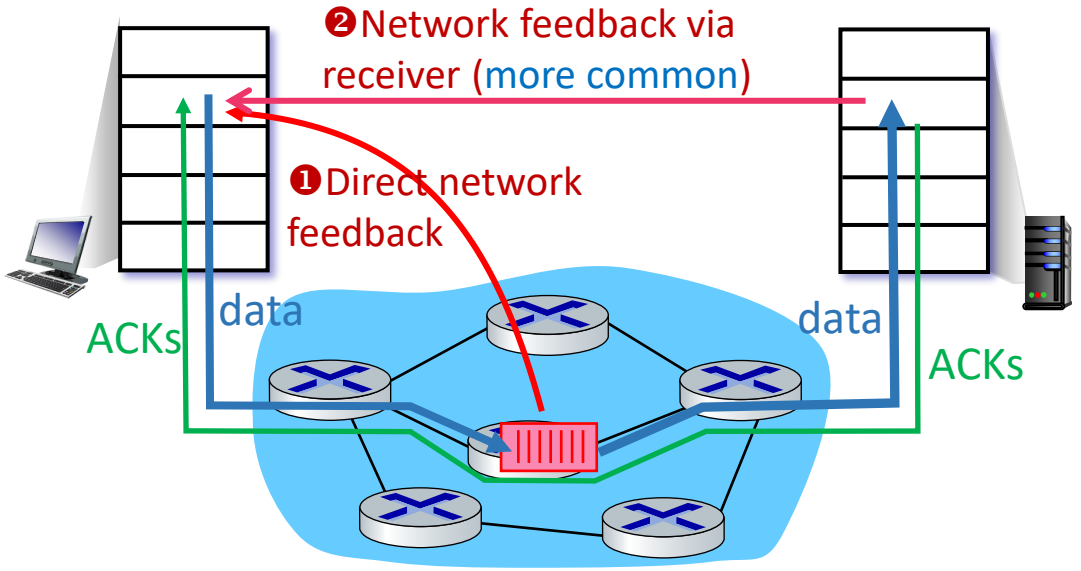


- approach taken by TCP since the late 1980's

Approaches towards congestion control

2. Network-assisted congestion control:

- routers provide *direct* feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate
  - TCP ECN, ATM, DECbit protocols



ECN: Explicit Congestion Notification

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



TCP congestion control: AIMD

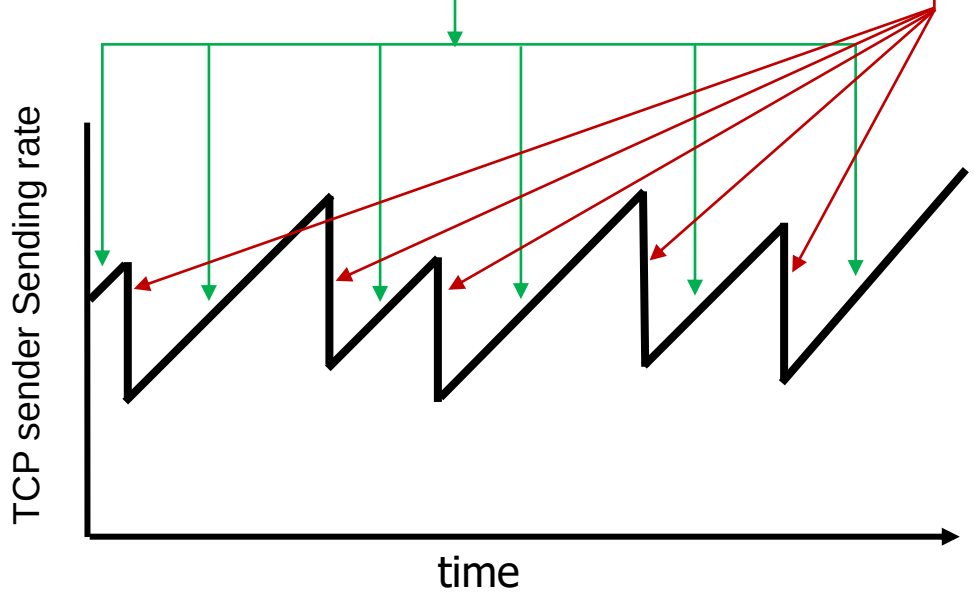
- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

increase sending rate by 1 maximum segment size (MSS) every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



TCP Reno

AIMD sawtooth behavior: *probing* for bandwidth

TCP AIMD: more

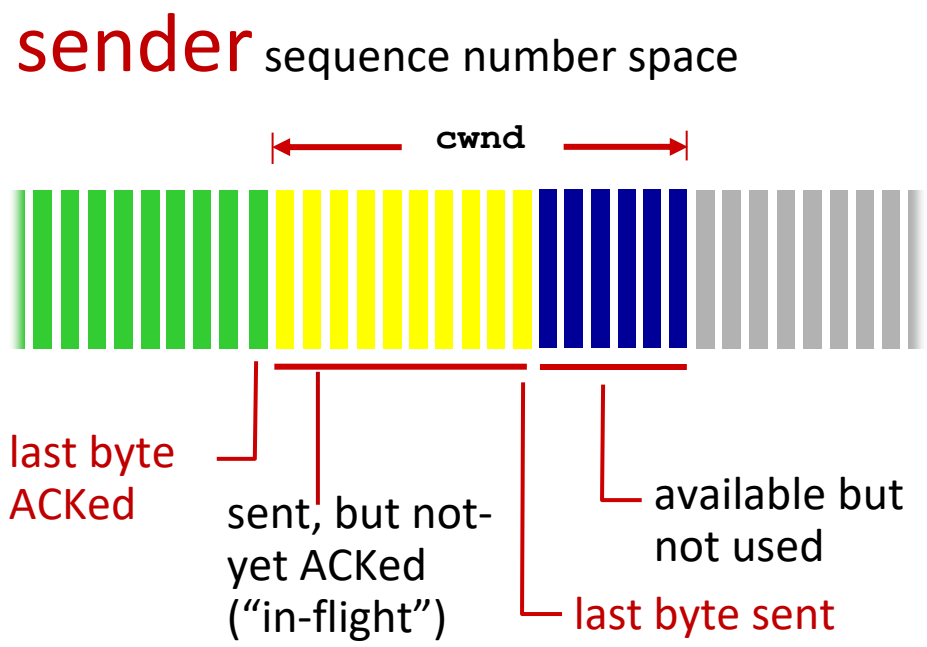
Multiplicative decrease detail: sending rate is

- Cut in half on loss detected by triple duplicate ACK (TCP Reno, new version, incorporated fast recovery)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe, early version)

Why AIMD?

- AIMD – a distributed, asynchronous algorithm – has been shown to:
  - optimize congested flow rates network wide!
  - have desirable stability properties

TCP congestion control: details



congestion window (cwnd) = send window

TCP sending behavior:

- *roughly:* send **cwnd** bytes, wait RTT for ACKS, then send more bytes

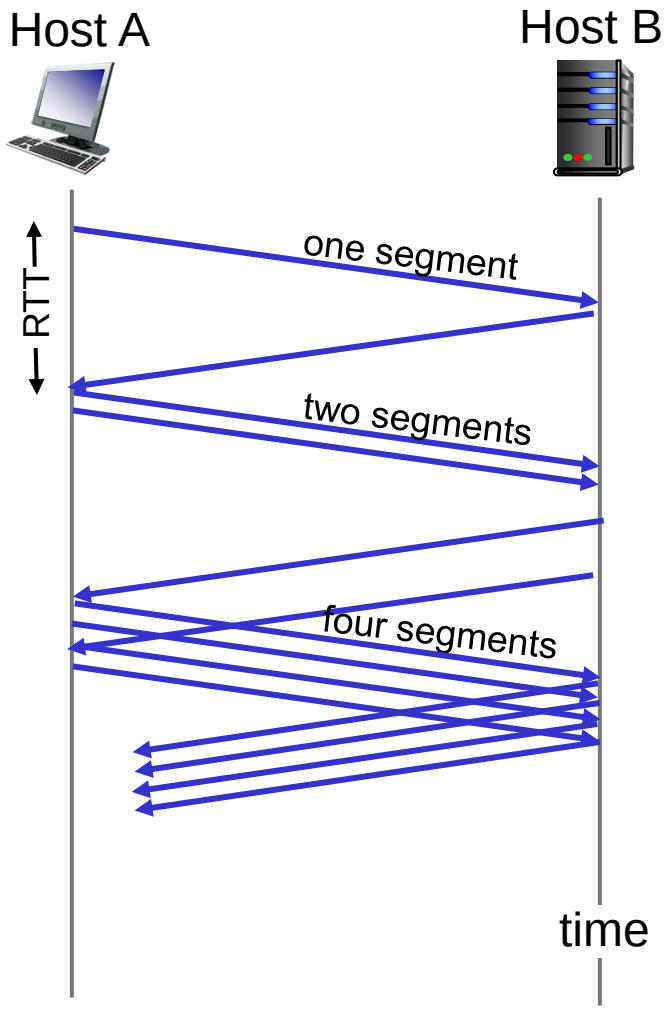
$$\text{TCP rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- TCP sender limits transmission: **LastByteSent-LastByteAcked ≤ min{cwnd, rwnd}**
- **cwnd** is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

TCP slow start

- when connection begins, increase rate exponentially until first loss event:
  - initially **`cwnd`** = 1 MSS
  - double **`cwnd`** every RTT
  - done by incrementing **`cwnd`** for every ACK received
  - TCP Reno is an "**ACK-pacing**" flow control protocol

- *summary:* initial rate is slow, but ramps up exponentially fast



Refinement: inferring loss

- After 3 dup ACKs

(fast retransmit):

  - **cwnd** is cut in half
  - window then grows linearly

- But after timeout event:
  - **cwnd** instead set to 1 MSS;
  - window then grows exponentially to a threshold, then grows linearly

Philosophy:

- 3 dup ACKs indicates network capable of delivering some segments

- timeout indicates a "more alarming" congestion scenario







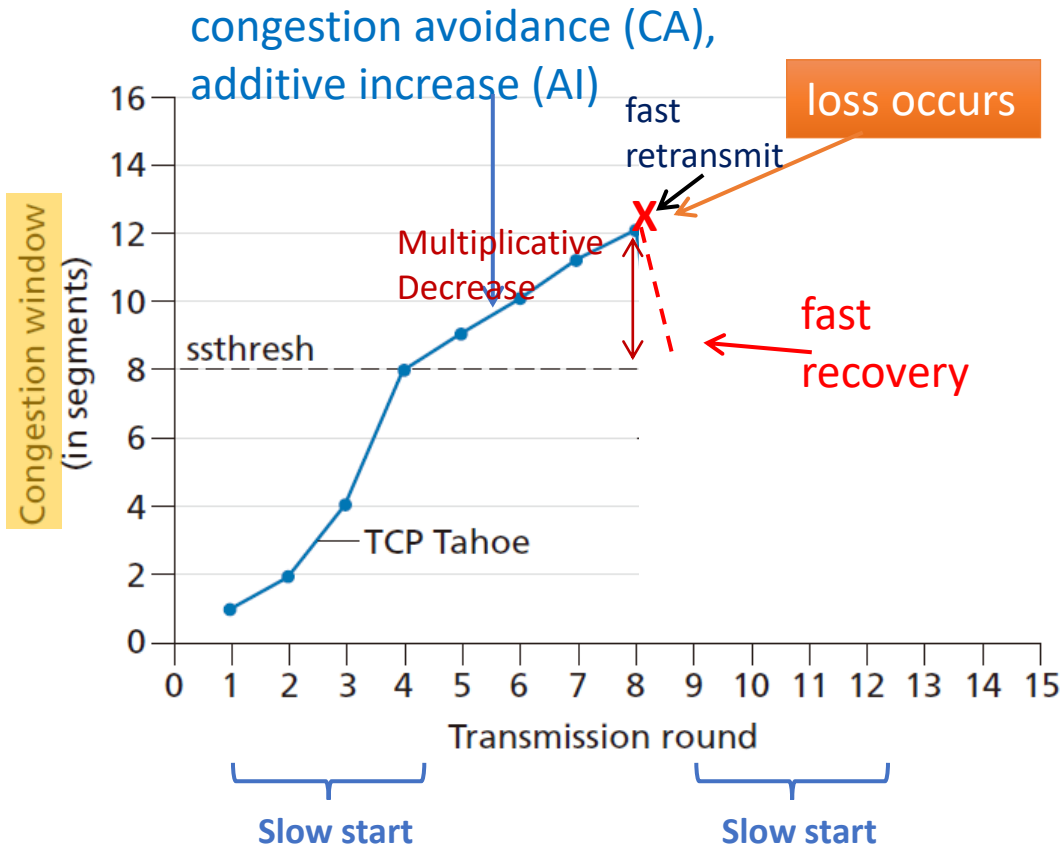
TCP: from slow start to congestion avoidance

Q: when should the exponential increase switch to linear?

A: when **`cwnd`** gets to 1/2 of its value before timeout.

Implementation:

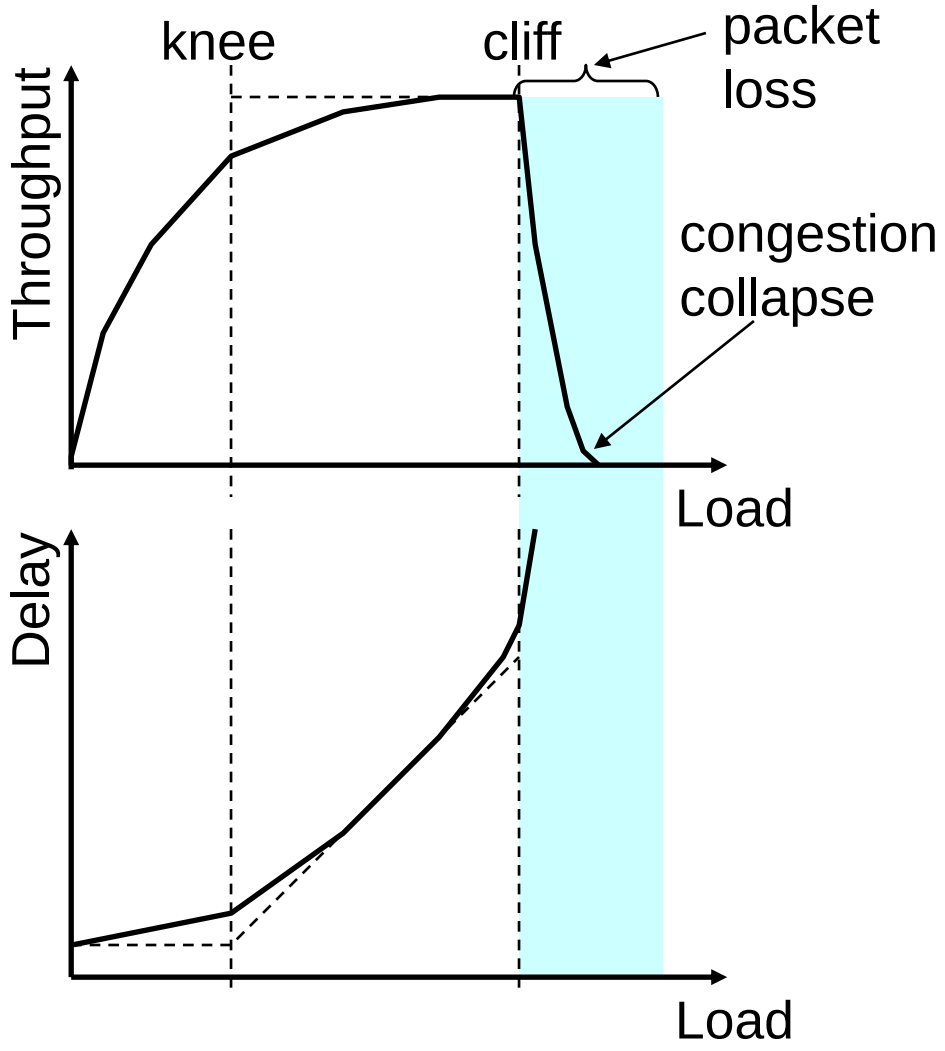
- variable **`ssthresh`**
- on loss event, **`ssthresh`** is set to 1/2 of **`cwnd`** just before loss event



ssthresh -- slow start threshold

Congestion: A Close-up View

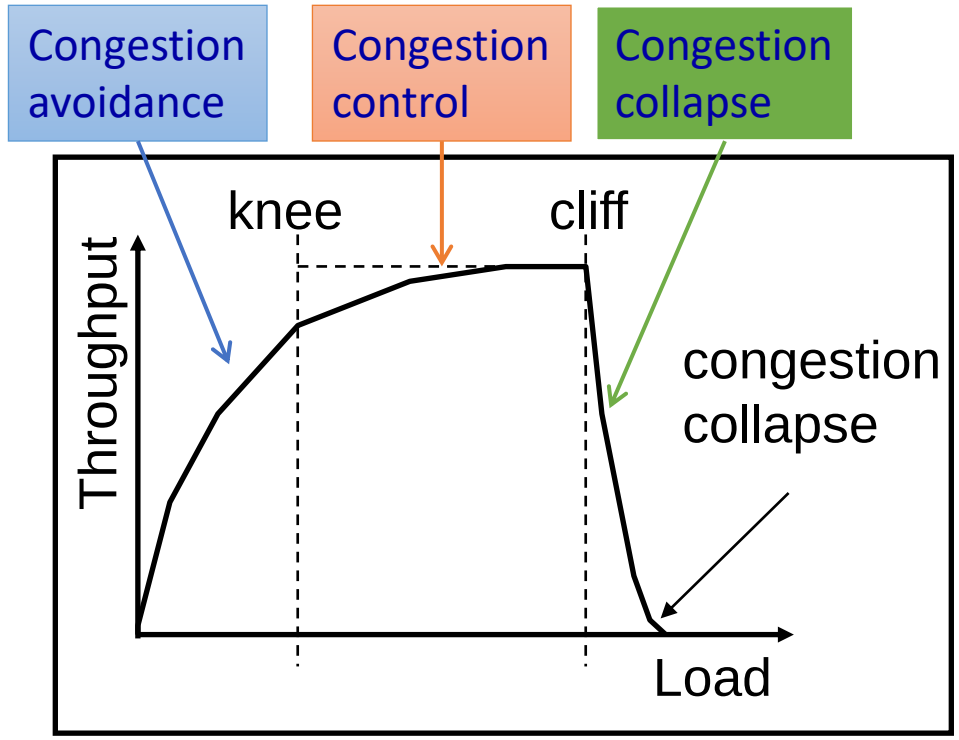
- knee – point after which
  - throughput increases very slowly
  - delay increases fast
- cliff – point after which
  - throughput starts to decrease very fast to zero (congestion collapse)**?**
  - delay approaches infinity



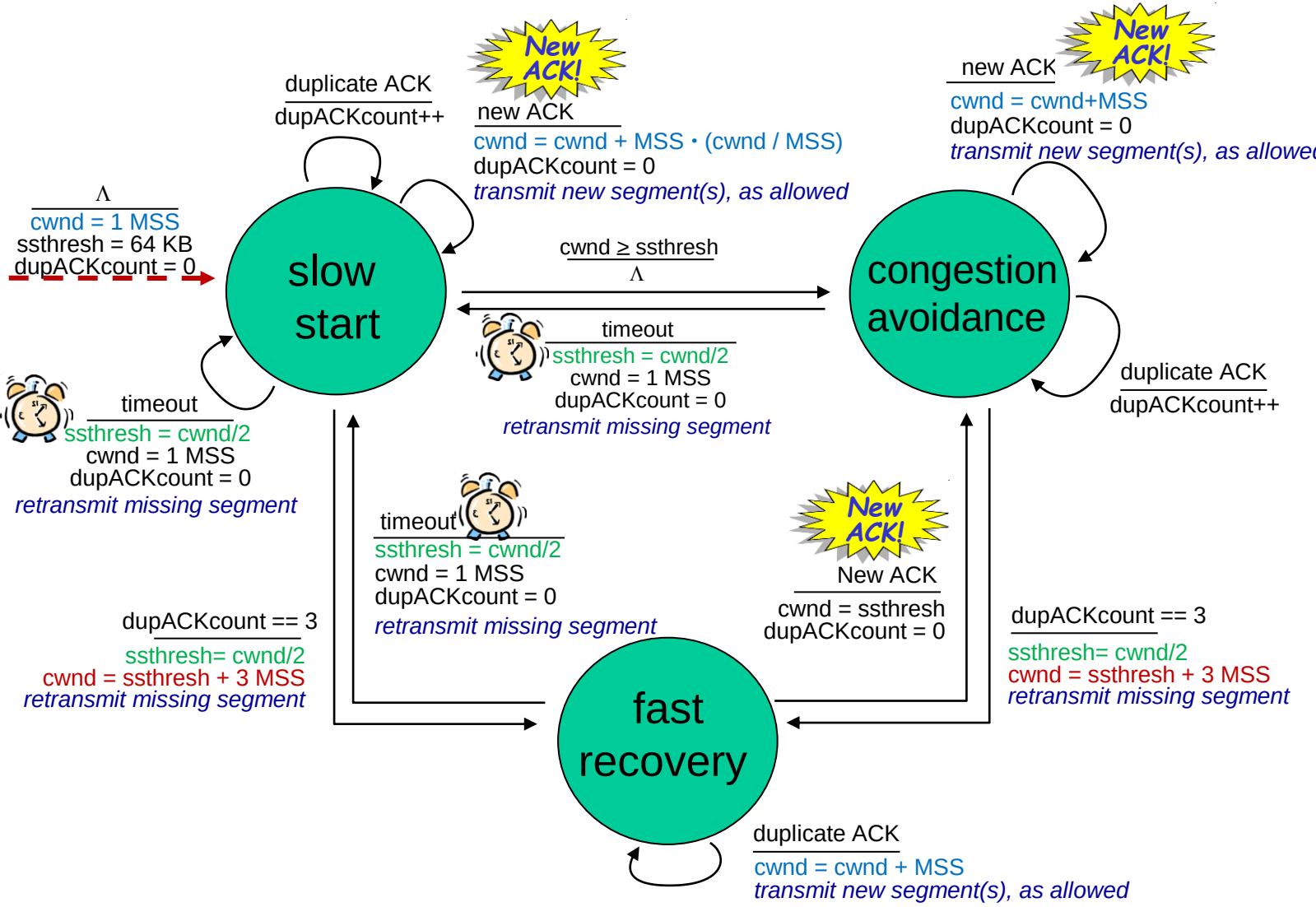
Congestion Control vs. Congestion Avoidance

- Congestion avoidance goal
  - stay left of knee
  - detection and prevention actions before overload occur
- Congestion control goal
  - stay left of cliff
  - response to the occurred network overload

❒ Congestion collapse
  - right of cliff



Summary: TCP congestion control



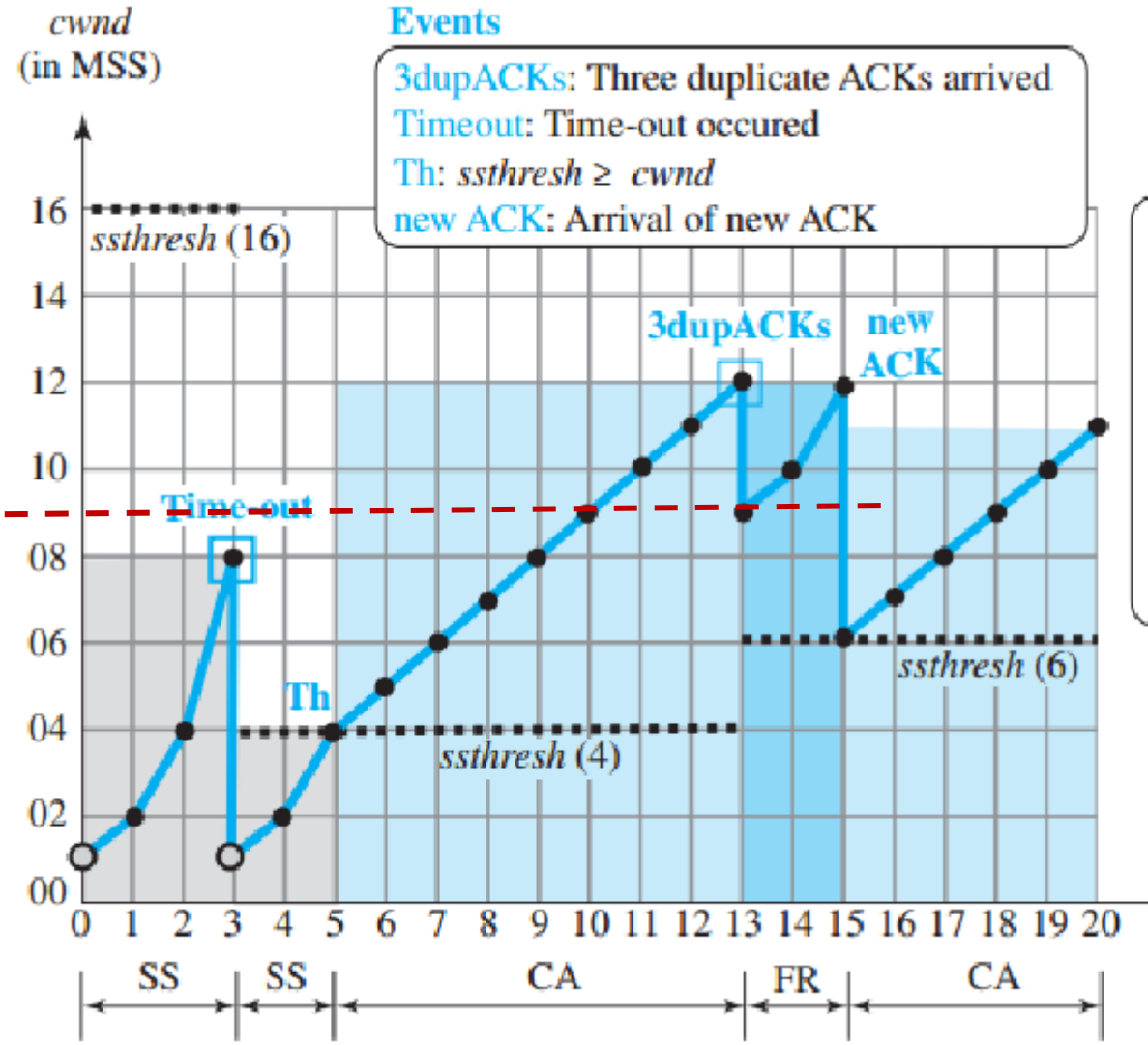
Because TCP uses ACK to trigger (or clock) its increase in cwnd size, TCP is said to be **self-clocking**.



Summary: TCP congestion control

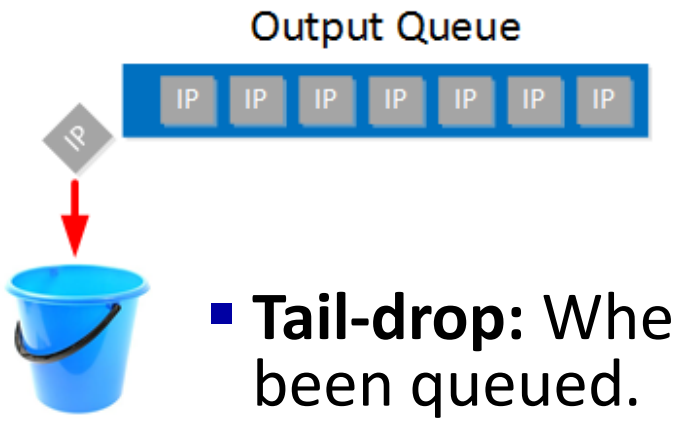
- When **cwnd** is below **Threshold**, sender in slow-start (SS) phase, window grows exponentially.
- When **cwnd** is above **Threshold**, sender is in congestion-avoidance (CA) phase, window grows linearly.
- When a triple duplicate ACK occurs, **Threshold** set to **cwnd/2** and **cwnd** set to **Threshold + 3 MSS**.
- When timeout occurs, **Threshold** set to **cwnd/2** and **cwnd** is set to **1 MSS**.

$\frac{12}{2} + 3 = 9$



TCP sender congestion control

State	Event	TCP Sender Action	Commentary
Slow Start (SS)	ACK receipt for previously unacked data	**cwnd** = **cwnd** + MSS * (**cwnd**/MSS), If (**cwnd** > Threshold) set state to "Congestion Avoidance"	Resulting in a doubling of **cwnd** every RTT
Congestion Avoidance (CA)	ACK receipt for previously unacked data	**cwnd** = **cwnd** + MSS	Additive Increase, resulting in increase of **cwnd** by 1 MSS every RTT
SS or CA	Loss event detected by triple duplicate ACK	Threshold = **cwnd**/2, **cwnd** = Threshold + 3 MSS, Set state to "Congestion Avoidance"	Fast recovery, implementing multiplicative decrease. **cwnd** will not drop below 1 MSS.
SS or CA	Timeout	Threshold = **cwnd**/2, **cwnd** = 1 MSS, Set state to "Slow Start"	Enter slow start
SS or CA	Duplicate ACK	Increment duplicate ACK count for segment being acked	**cwnd** and Threshold not changed

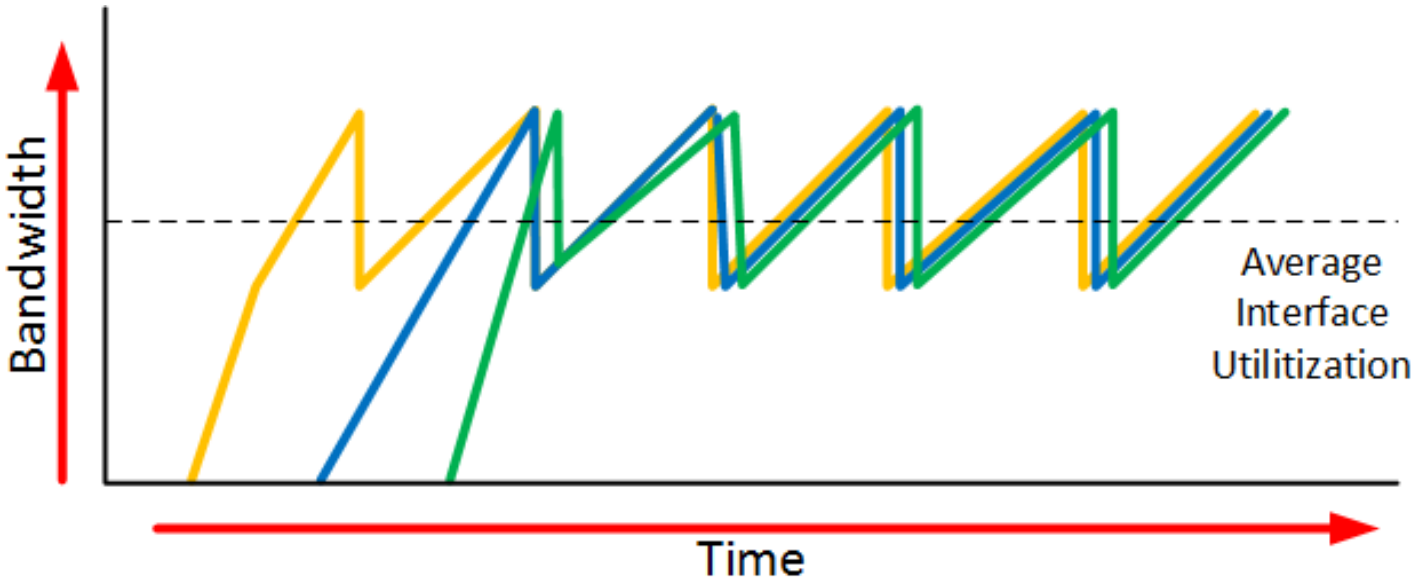


Congestion, Tail Drop and TCP

- **Tail-drop:** When a queue is full, the router drops packets that should have been queued.
- Tail-drop policy used by early routers has a significant effect on TCP congestion control and may lead to TCP global synchronization.

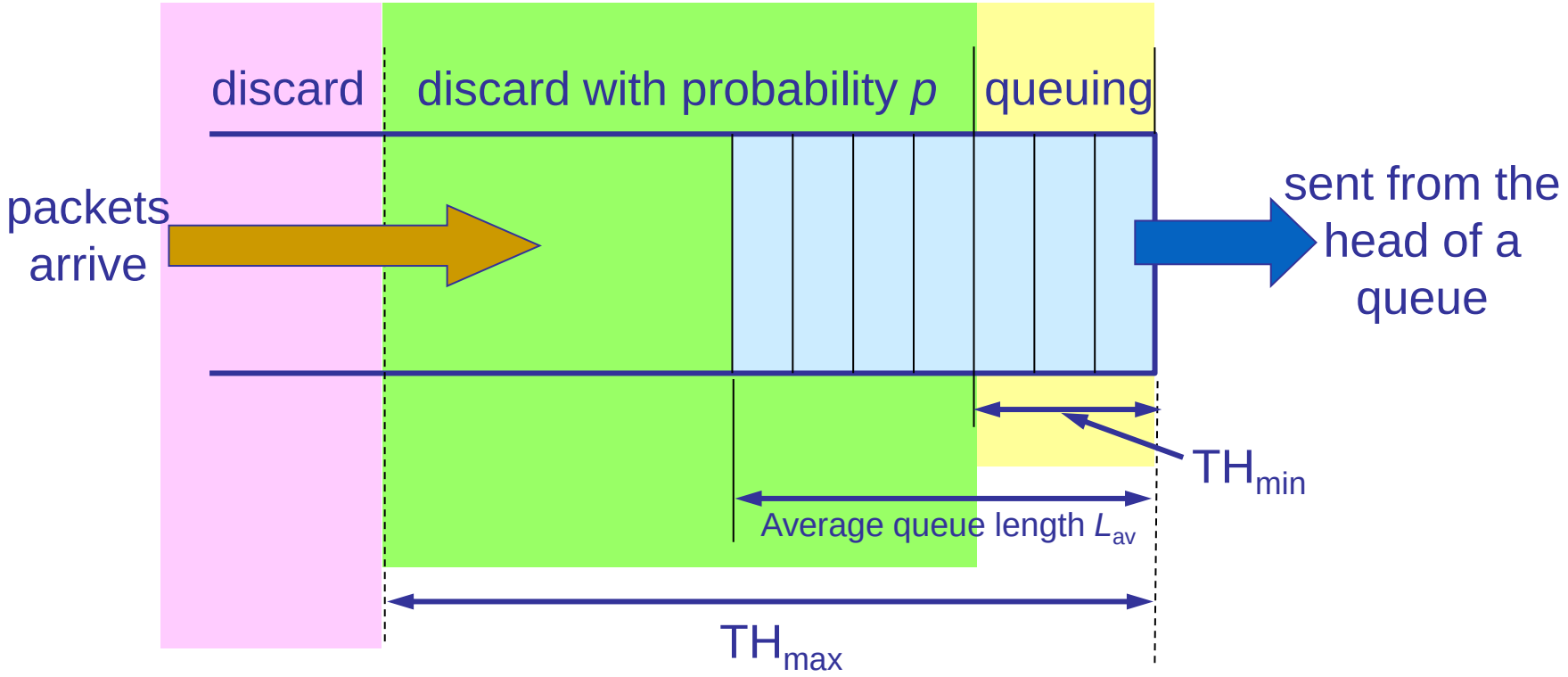
Reasons:

- datagrams are typically multiplexed
- under a tail-drop policy, the router will discard one segment from N connections ➔ the simultaneous loss causes all N instances of TCP to enter slow-start at the same time
- after the network recovers, throughput will suddenly increase a lot



Random early detection (RED)

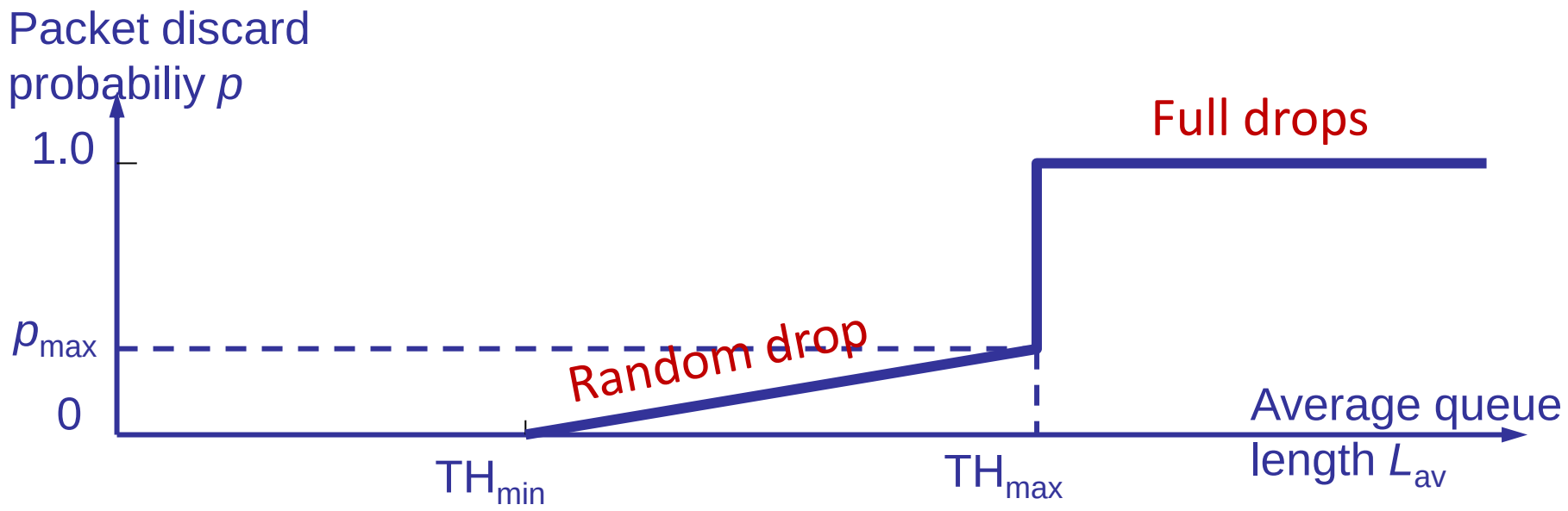
- Q: How can a router avoid tail-drop and global synchronization?
- A: RED (instead of waiting until the queue overflows, a router slowly and randomly drops datagrams as congestion increases)



Relations among p, TH_{min} and TH_{max}

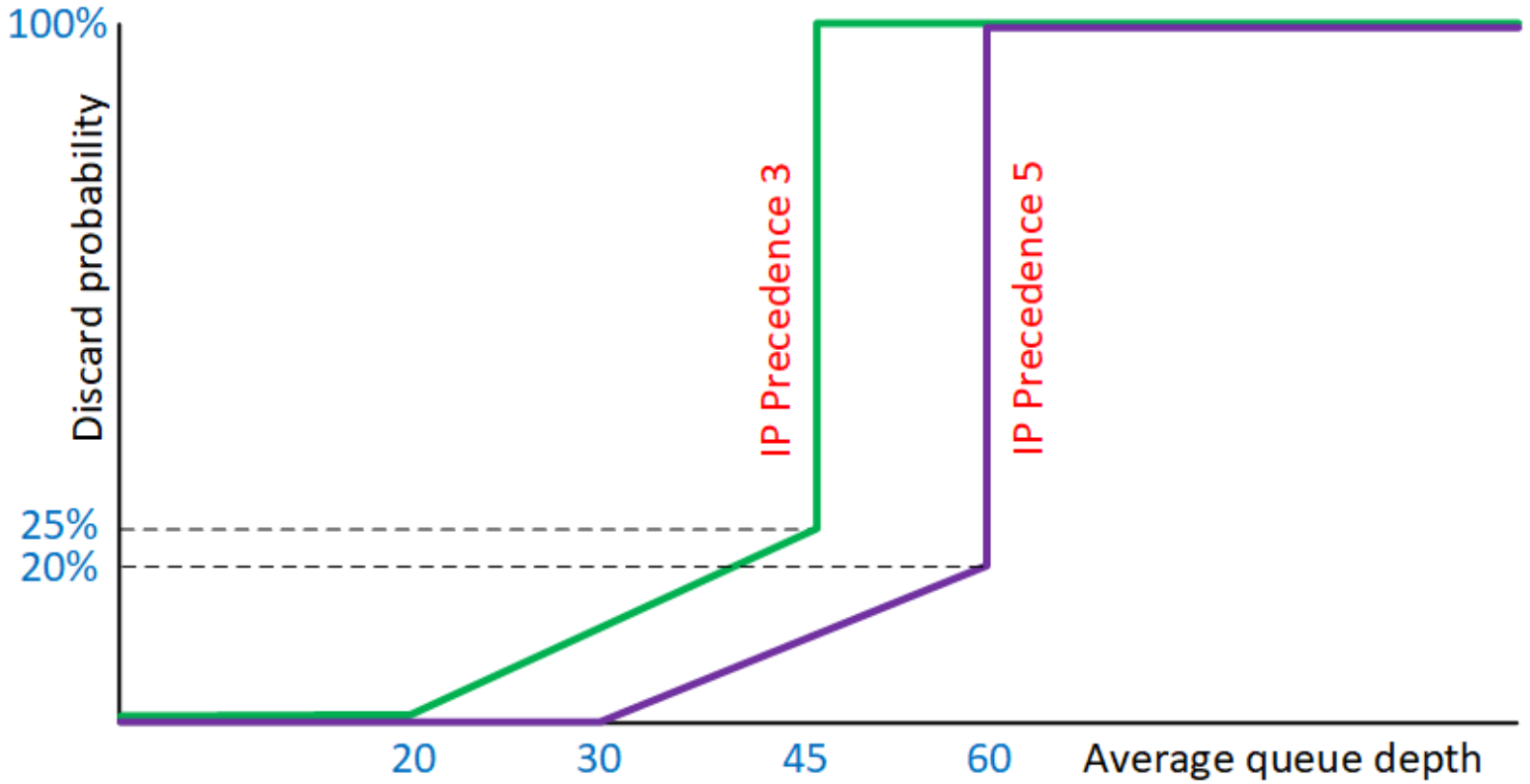
- *If* $L_{AV} < TH_{min}$, $p = 0$;
- *If* $L_{AV} > TH_{max}$, $p = 1$;
- *If* $TH_{min} < L_{AV} < TH_{max}$, $0 < p < 1$.

The default option for these thresholds is the number of packets, but you can also use the number of bytes or even ms/μs for these thresholds.



Weighted Random Early Detection (WRED) actual deployment

- Some packets are more important than others so instead of dropping completely random packets, we use different traffic profiles for different packets.
- We can discard packets based on criteria like the CoS, IP Precedence, DSCP, and some other options.



Network-Assisted ECN and Delayed-based Congestion Control

ECN: explicit congestion notification

Network-assisted ECN

- ECN-capable TCP
- DCCP – Datagram Congestion Control Protocol
  - provides a low-overhead, congestion-controlled UDP-like unreliable service that utilizes ECN
- DCTCP (Data Center TCP)
- DCQCN (Data Center Quantized Congestion Notification)
  - Designed specifically for data center networks, also makes use of ECN

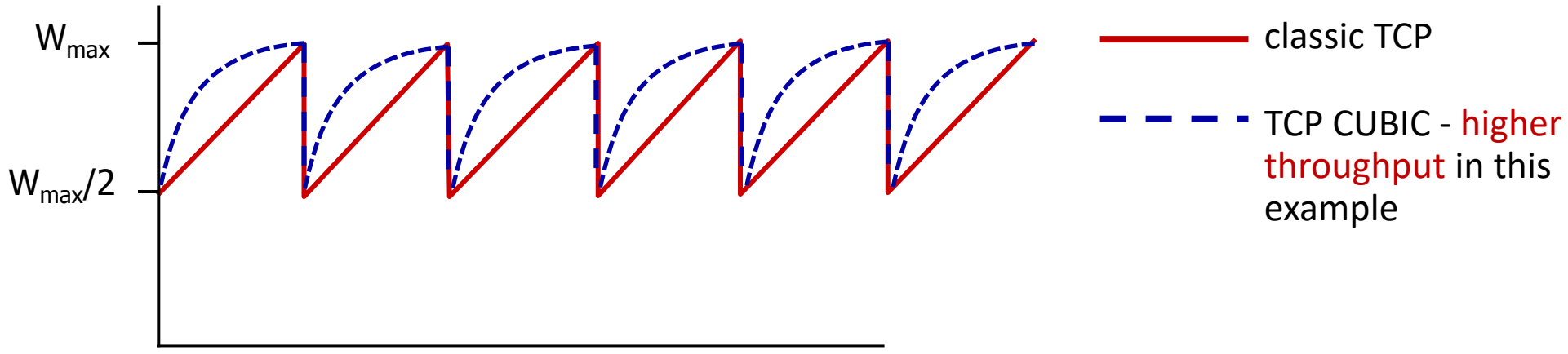
Delay-based congestion control protocol

- TCP Vegas
  - TCP senders should *"Keep the pipe just full, but no fuller"*
- TCP BBR congestion control protocol
- TIMELY for data center networks
- Compound TCP (CTPC)
- FAST
  - for high-speed and long distance networks.

There are many (many!) more versions of TCP!

TCP CUBIC

- Is there a better way than AIMD to "probe" for usable bandwidth?
- Insight/intuition:
  - W_{max}: sending rate at which congestion loss was detected
  - congestion state of bottleneck link probably (?) hasn't changed much
  - after cutting rate/window in half on loss, initially ramp to to W_{max} *faster*, but then approach W_{max} more *slowly*



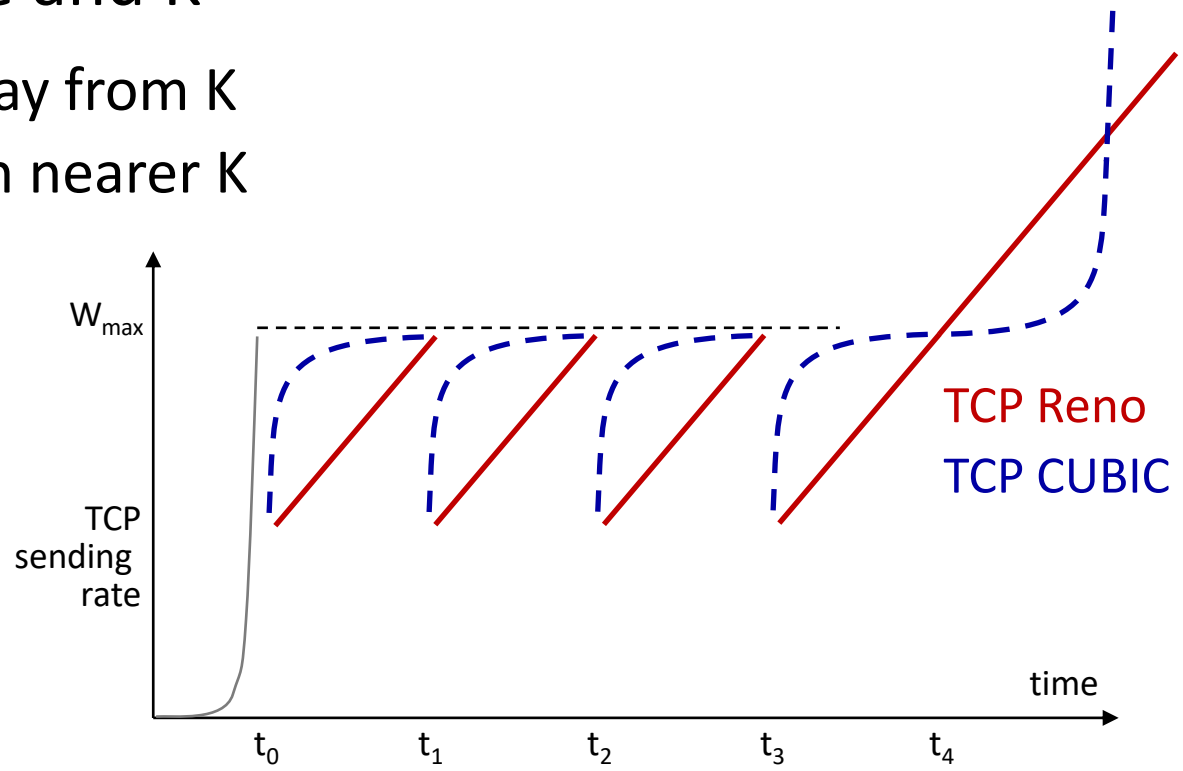
TCP CUBIC

$$W_{cubic} = C(T-K)^3 + Wmax$$

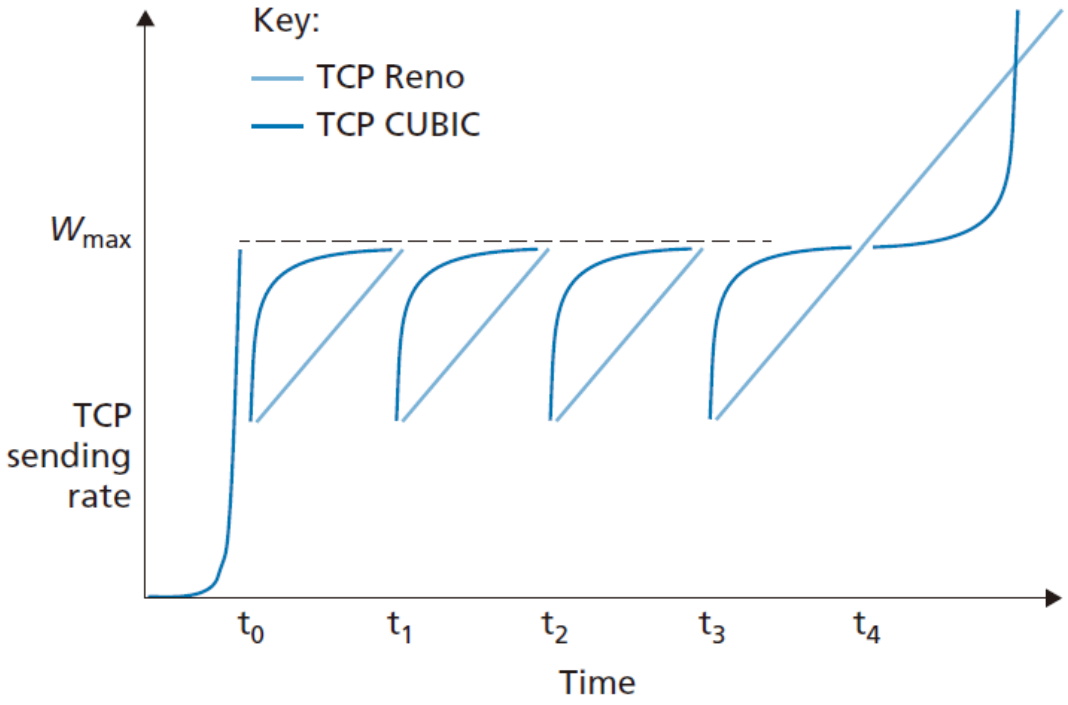
$$K = \sqrt[3]{\frac{W_{max}B}{C}}$$

- K: point in time when TCP window size will reach W_{max}
  - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers

C is a scaling constant
B: a constant fraction for multiplicative increase
T: time since last packet drop
W_{max}: cwnd when last packet dropped



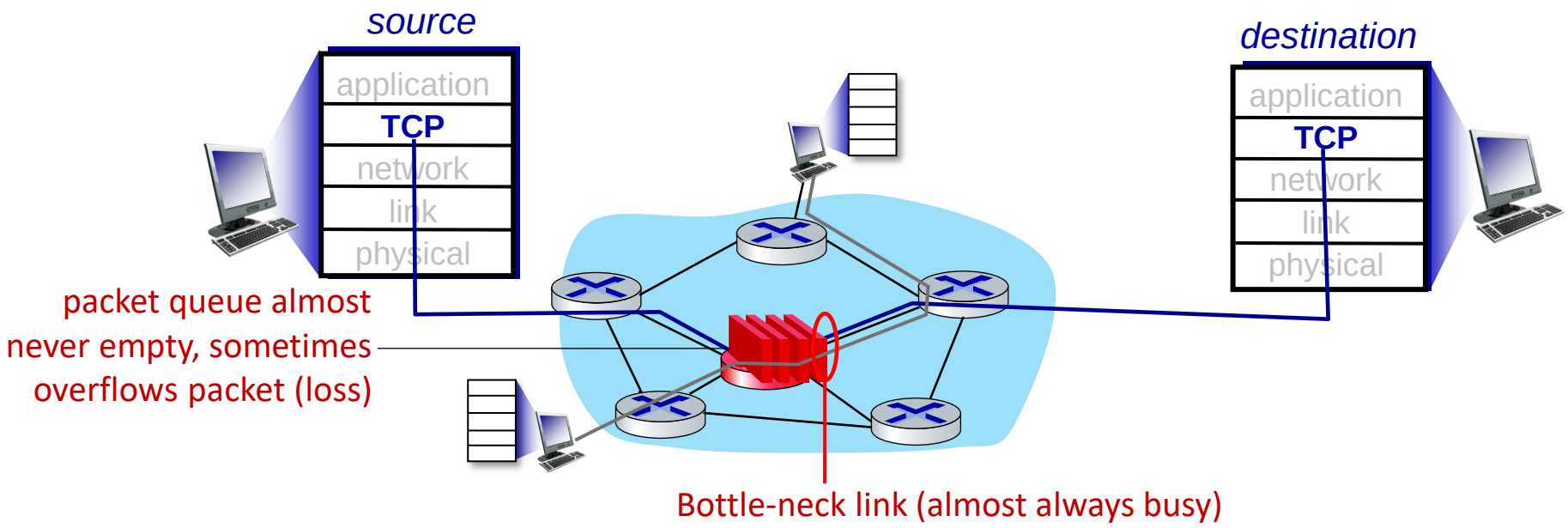
The Idealized Performance of TCP Reno and TCP CUBIC



- when congestion loss occurs at $t1$, $t2$, and $t3$, CUBIC more quickly ramps up close to W_{max} (thereby enjoying more overall throughput than TCP Reno).
- TCP CUBIC has recently gained wide deployment.
  - default version of TCP used in the Linux operating system.

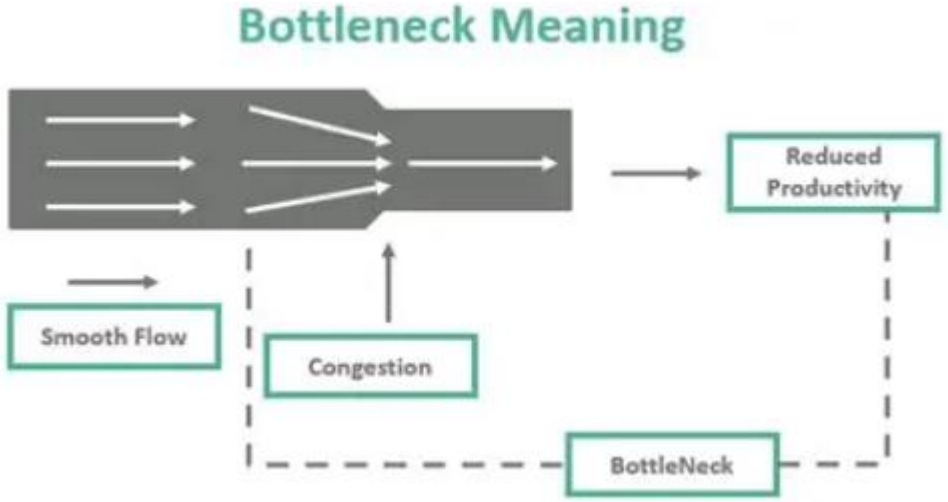
TCP and the congested "bottleneck link"

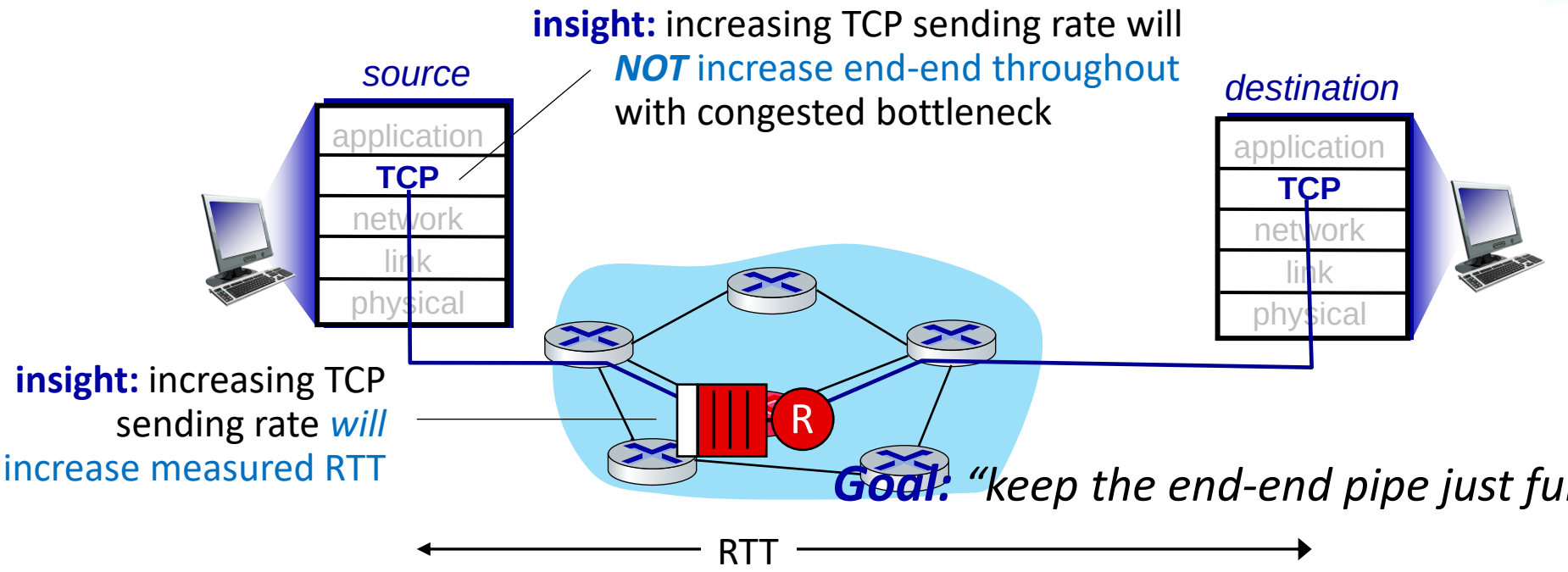
- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*



TCP and the congested "bottleneck link"

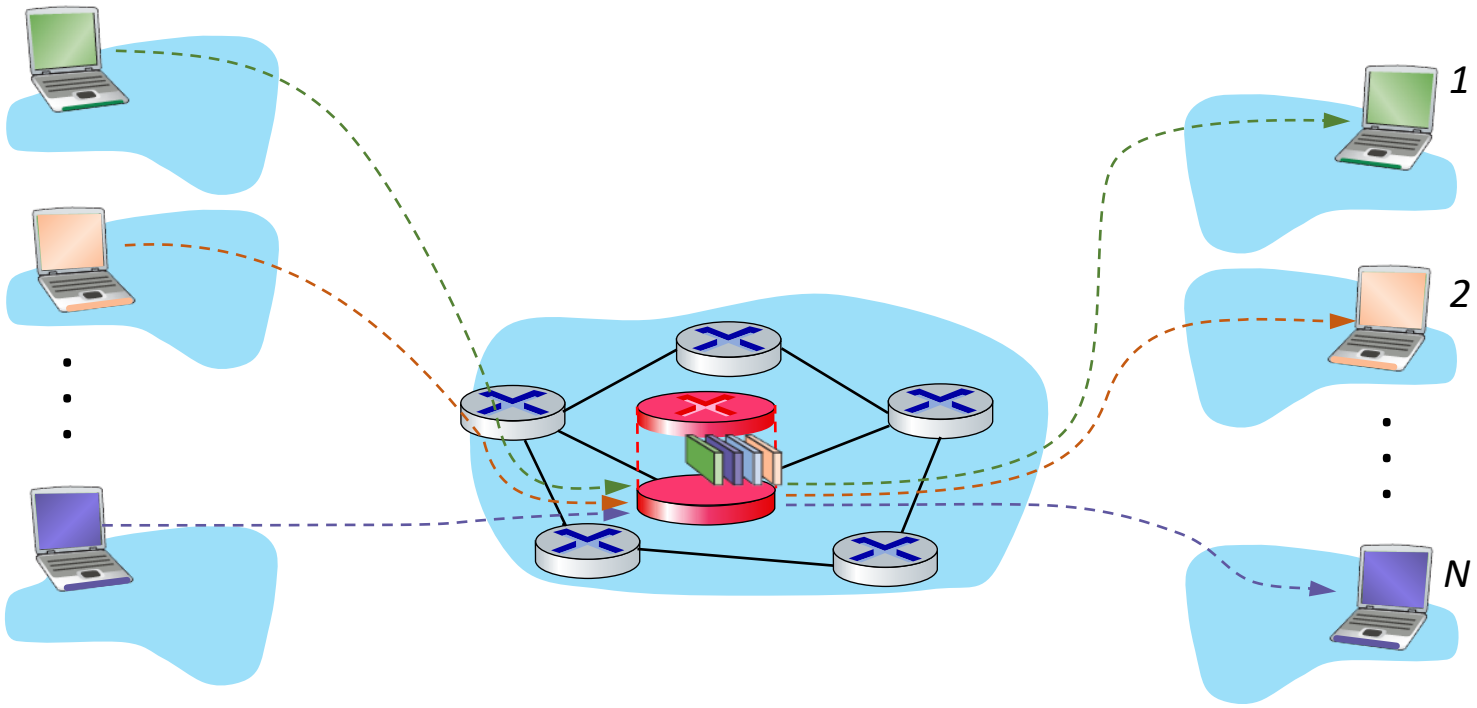
- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link





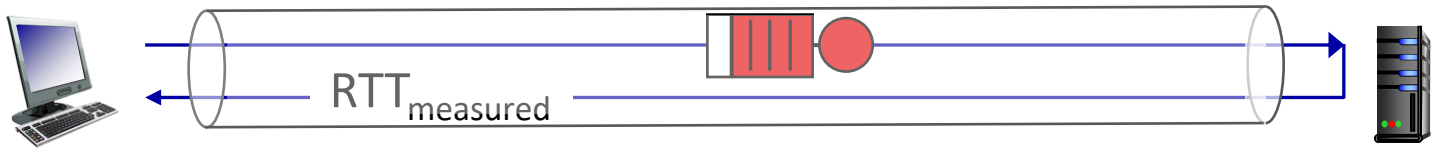
TCP and the congested "bottleneck link"

- multiple flows collectively congest bottleneck link:
  - each their executing own TCP congestion control
  - each of *N* flows should ideally receive throughput of *R/N*



Keeping the pipe just full enough

Keeping pipe "just full enough, but no fuller": keep bottleneck link busy transmitting, but avoid high delays/buffering



$$\text{measured throughput} = \frac{\text{\# bytes sent (inflight) in RTT interval}}{\text{RTT}}$$

Intuition:

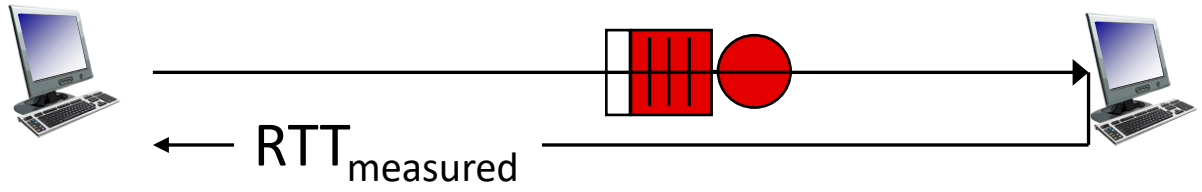
ideal amount of inflight data for a connection = *connection's share of bottleneck_bandwidth • min_RTT*

$n_{inflight}$

connection's "*bandwidth-delay product*"

Delay-based TCP congestion control

Keeping sender-to-receiver pipe "just full enough, but no fuller": keep bottleneck link busy transmitting, but avoid high delays/buffering



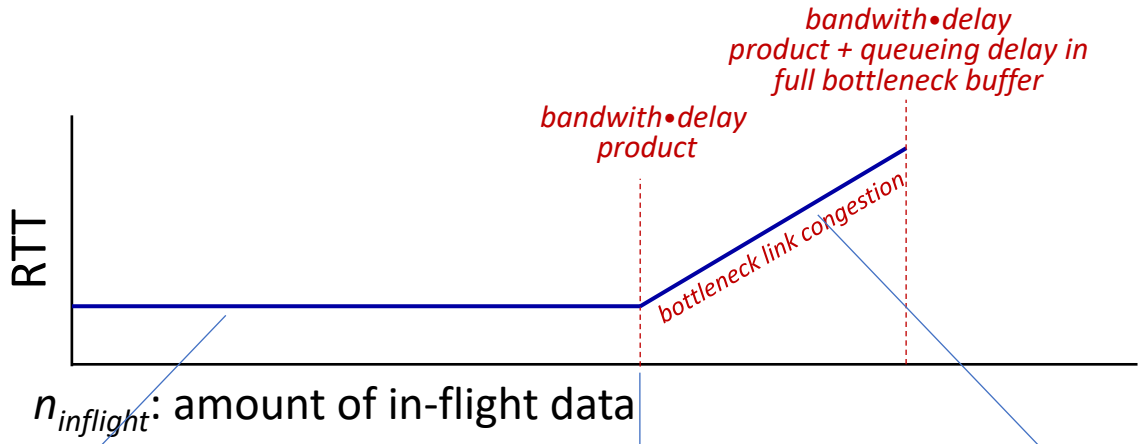
$$\text{measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{\text{RTT}_{\text{measured}}}$$

Delay-based approach:

- RTT_{min} - minimum observed RTT (uncongested path)
- uncongested throughput with congestion window `cwnd` is $\text{cwnd}/\text{RTT}_{\text{min}}$

if measured throughput "very close" to uncongested throughput
    increase `cwnd` linearly /* since path not congested */
else if measured throughput "far below" uncongested throughout
    decrease `cwnd` linearly /* since path is congested */

Keeping the pipe just full, but no fuller

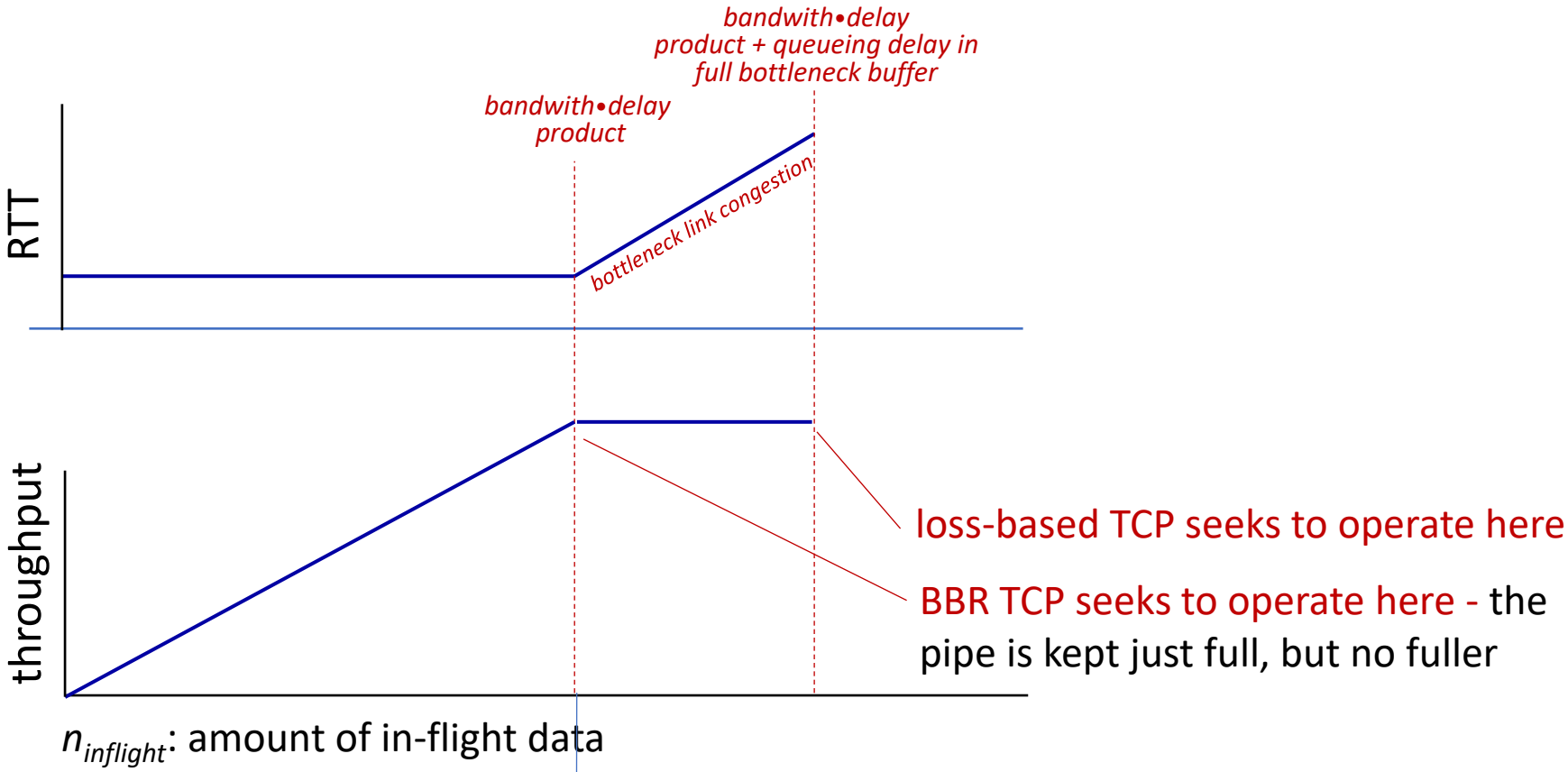


increasing $n_{inflight}$ in this region will not increase RTT, since arrival rate of packet data to queue is less that transmission rate

Here, arrival rate of packet data to bottleneck link is equals link transmission rate

Increasing $n_{inflight}$ in this region increases RTT since arrival rate of packet data to bottleneck link exceeds link transmission rate

Keeping the pipe just full, but no fuller

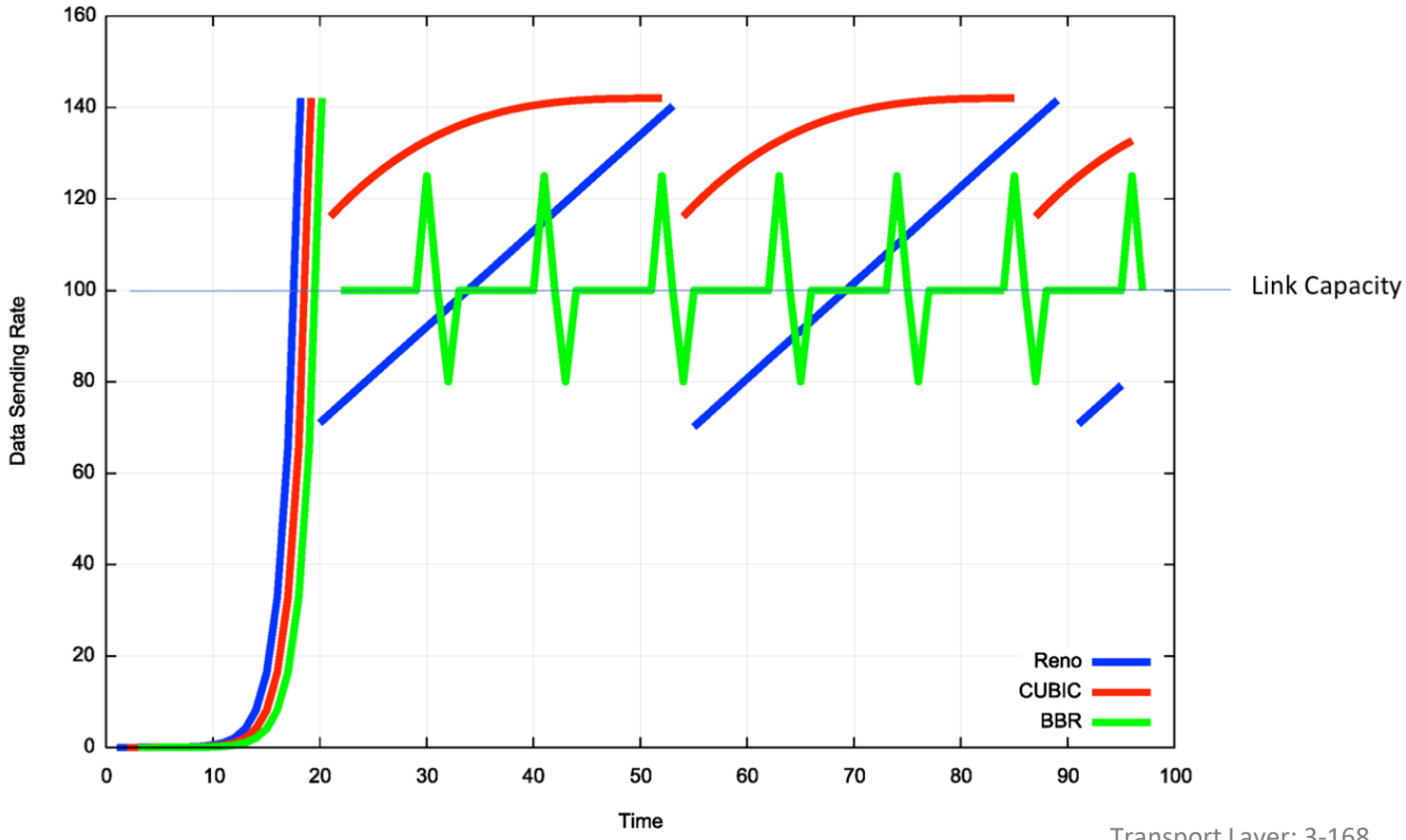


Here, arrival rate of packet data to bottleneck link is equals link transmission rate

Delay-based TCP congestion control

- congestion control without inducing/forcing loss
- **Goal:** maximizing throughout ("keeping the just pipe full… ") while keeping delay low ("…but not fuller")
- Various deployed TCPs
  - TCP Vegas
  - Congestion-based BBR deployed on Google's (internal) backbone network

BBR = **B**ottleneck **B**andwidth and **R**ound-trip propagation time



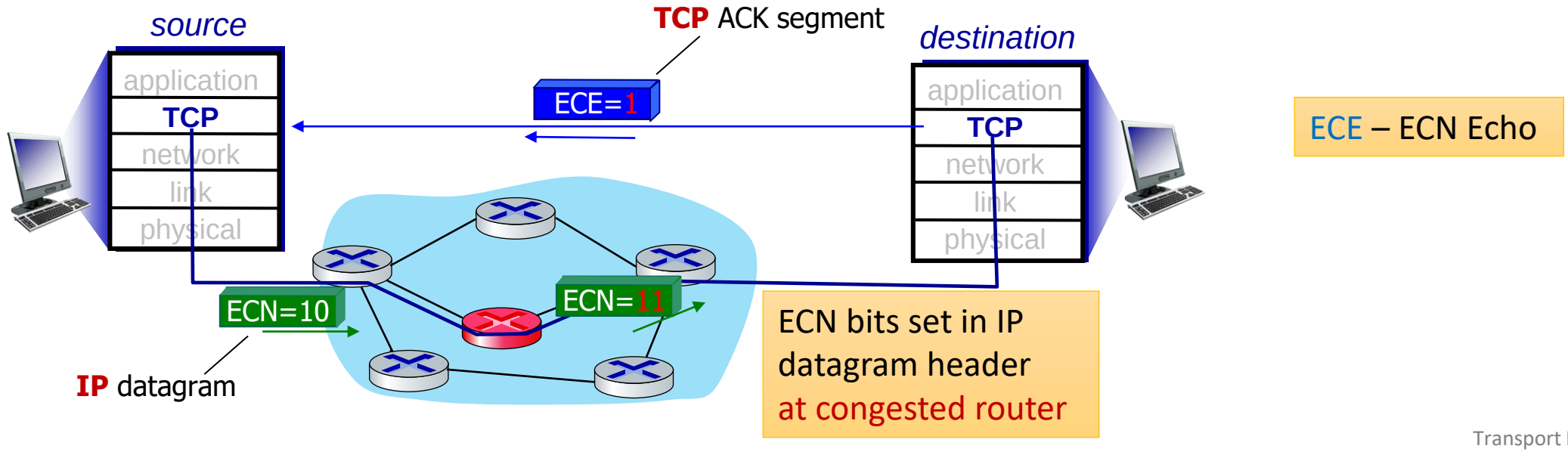
BBR (Bottleneck Bandwidth and RTT)

- regulates $n_{inflight}$
- at longer time intervals:
  - reduces $n_{inflight}$ to drain pipe, measure new *min_RTT*
- at shorter time intervals:
  - *acceleration:* increases sending rate, $n_{inflight}$, until reaches throughput plateau, (saturating available per-flow link capacity)
  - *cruising:* sends at the rate that network is delivering data, as evidenced through received ACKs.
  - *deceleration:* purposefully reduces $n_{inflight}$, decreasing queue pressure, looking for lower *min_RTT*

Explicit Congestion Notification (ECN)

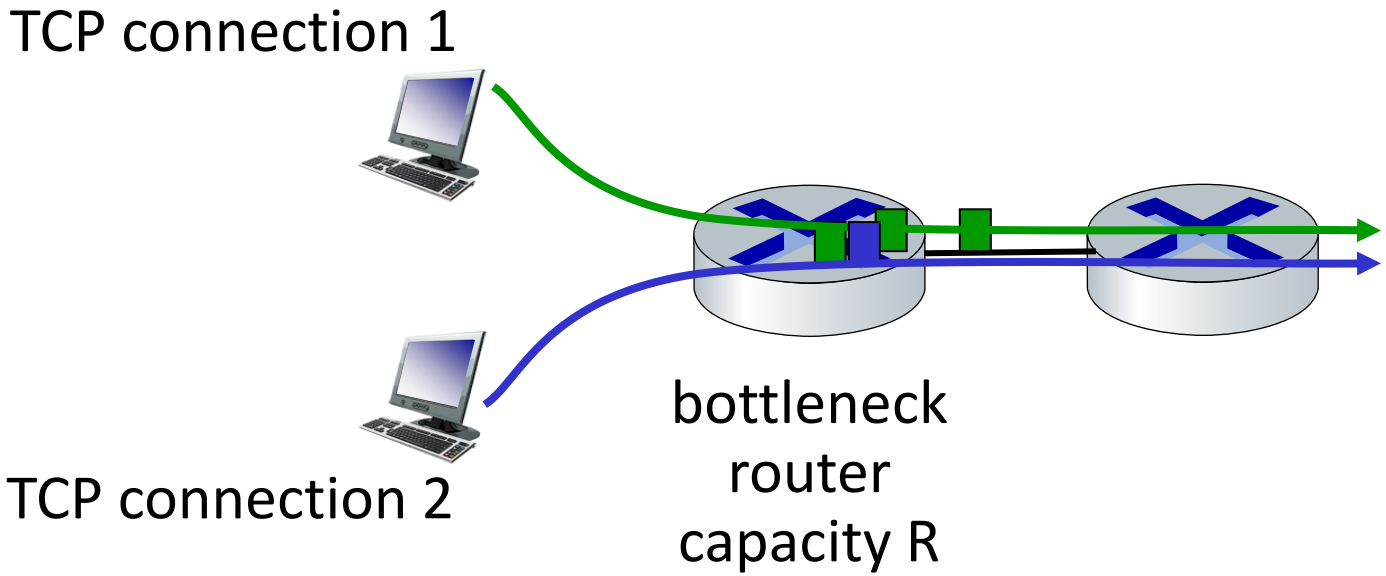
TCP deployments often implement *network-assisted* congestion control:

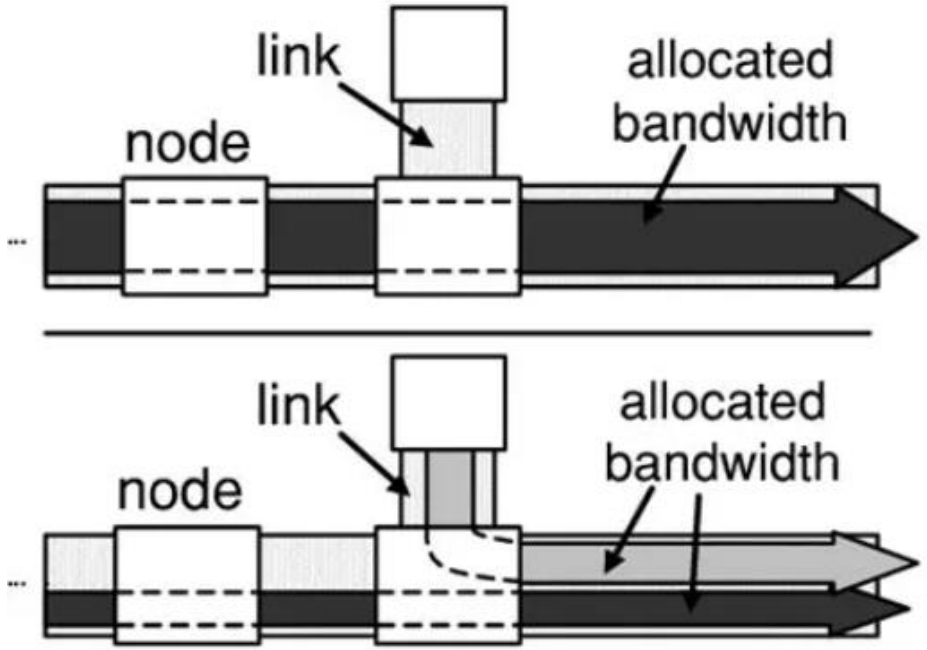
- **ECN**: two bits in IP header (ToS field) marked *by network router* to indicate congestion
  - *policy* to determine marking chosen by network operator
  - congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involve both IP (IP header ECN bit marking) and TCP (TCP header C, E bit marking)



TCP fairness

Fairness goal: if *K* TCP sessions share same bottleneck link of bandwidth *R*, each should have average rate of *R/K*



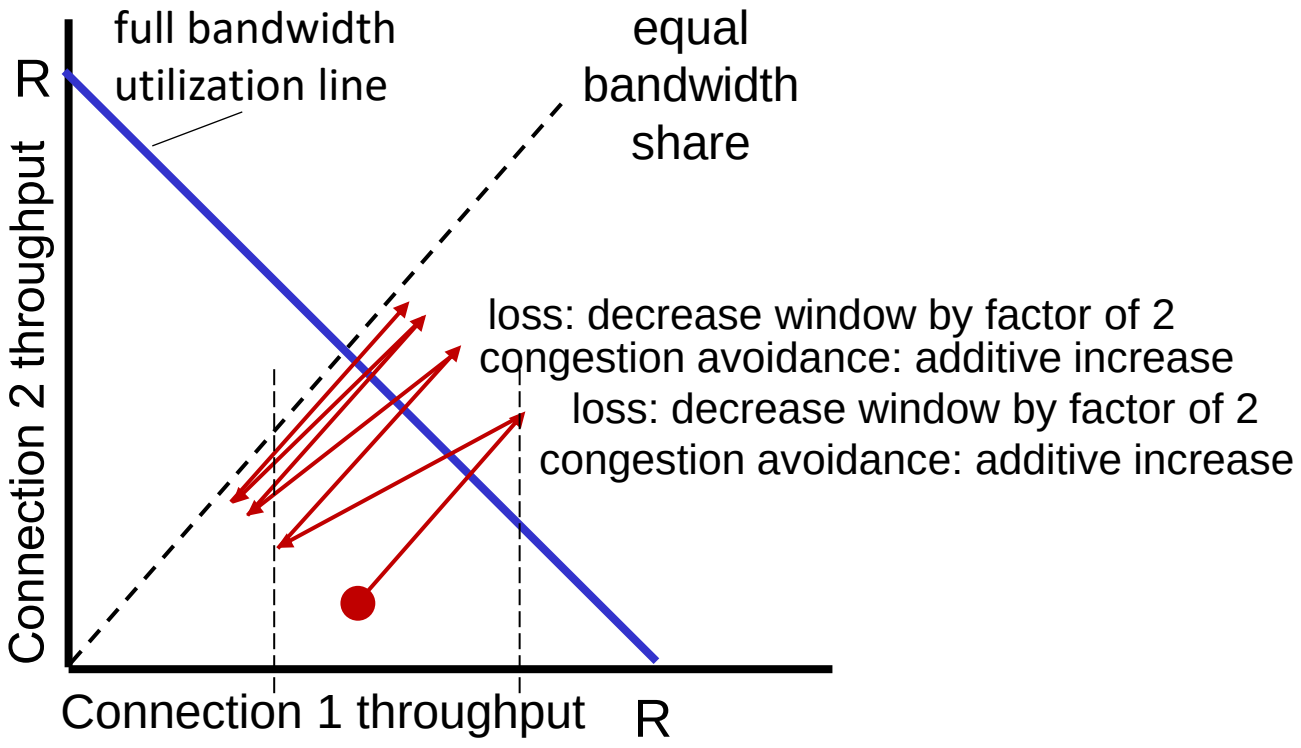


Two TCP connections sharing a single bottleneck link

Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance (CA-AIMD)

Fairness: must all network apps be "fair"?

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- there is no "Internet police" policing use of congestion control



Fairness, parallel TCP connections

- application can open *multiple* parallel connections between two hosts
- web browsers do this, e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets > R/2

Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



Evolving transport-layer functionality

- TCP, UDP: principal transport protocols for 40 years
- different "flavors" of TCP developed, for specific scenarios:

	Scenario	Challenges
CUBIC/BBR	Long, fat pipes (large data transfers)	Many packets "in flight"; loss shuts down pipeline
CUBIC (+ECN) / BBR	Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
CUBIC/BBR	Long-delay links	Extremely long RTTs
DCTCP/DCQCN	Data center networks	Latency sensitive
LEDBAT	Background traffic flows	Low priority, "background" TCP flows

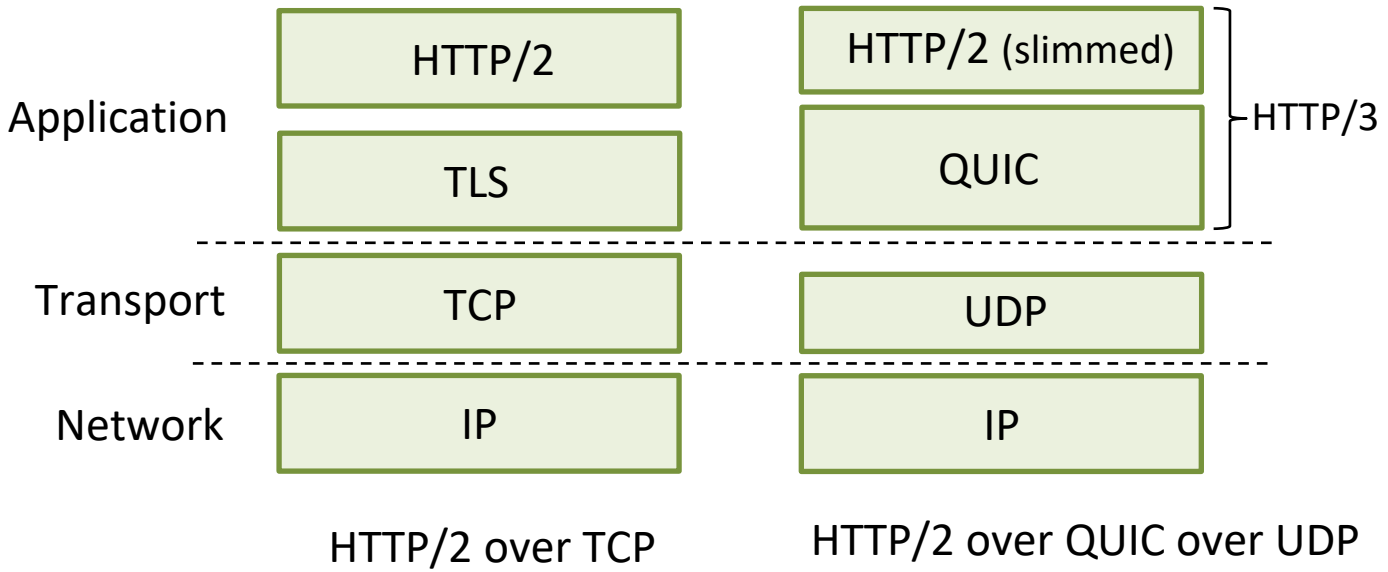
- The *only* common features of these protocols is that they use the TCP segment format
- moving transport–layer functions to application layer, on top of UDP
  - HTTP/3: QUIC

No single TCP variant can fit all scenarios



QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)



Relevant layers

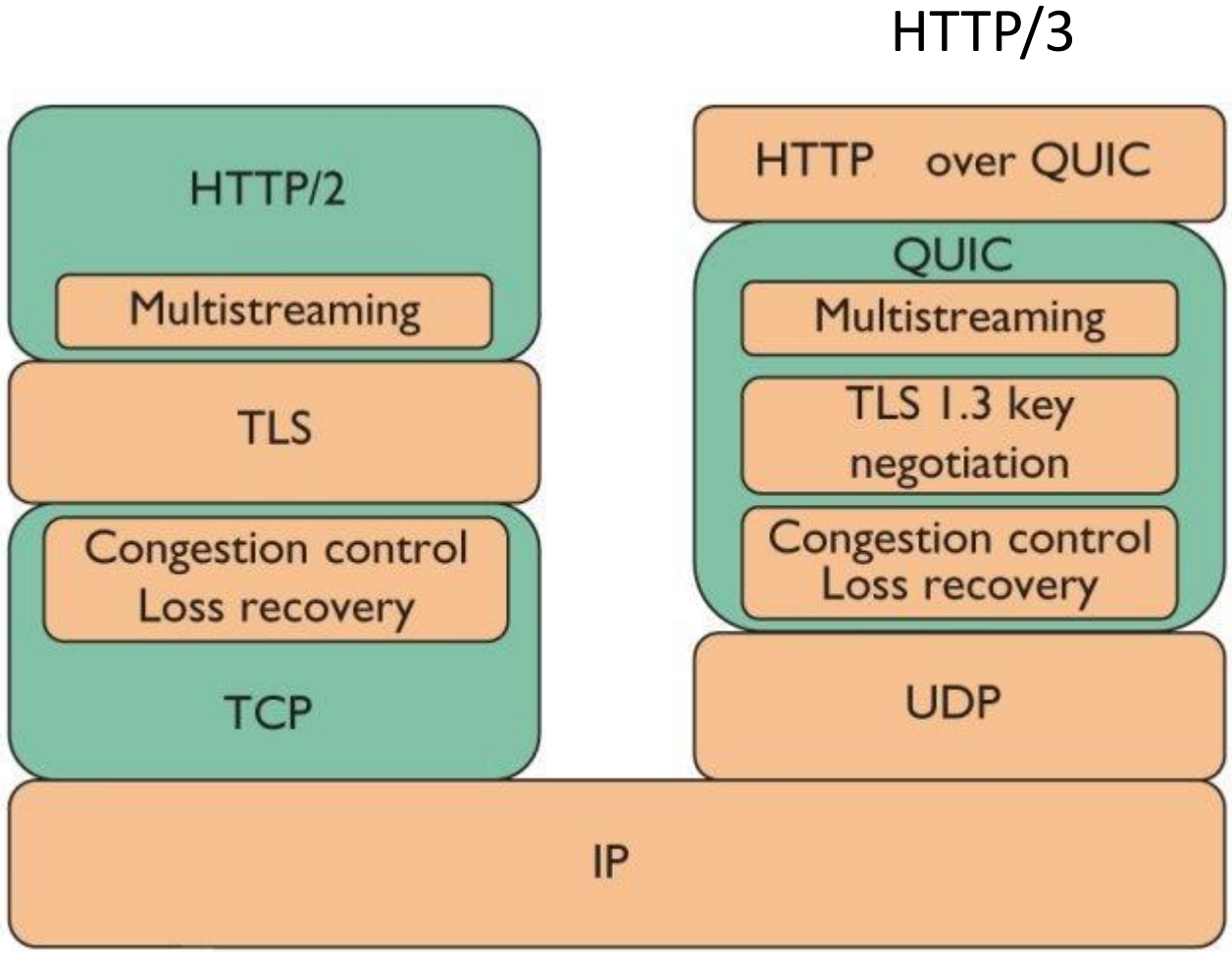
layers

application

security

Transport

Network



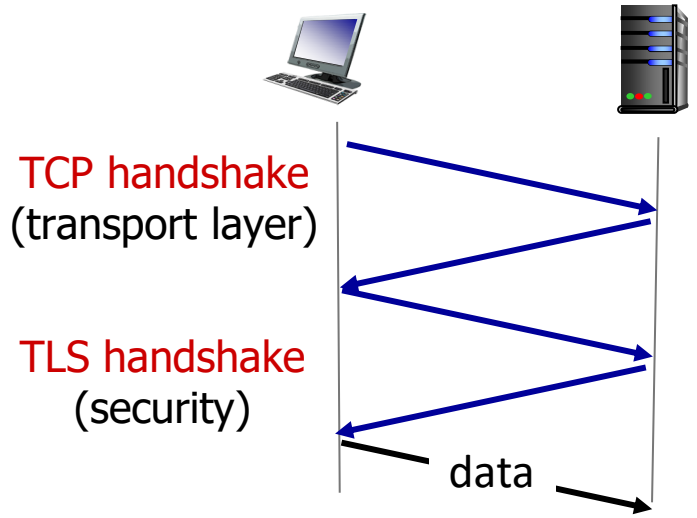
QUIC: Quick UDP Internet Connections

adopts approaches we've studied in this unit for connection establishment, error control, congestion control

- **error and congestion control:** "Readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones." [from QUIC specification]
- **connection establishment:** reliability, congestion control, authentication, encryption, state established in one RTT

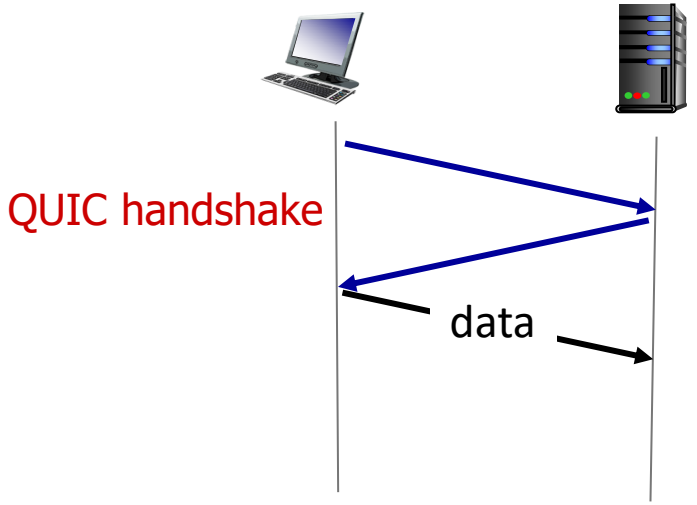
- multiple application-level "streams" multiplexed over single QUIC connection
  - separate reliable data transfer (rdt), security
  - common congestion control

QUIC: Connection establishment



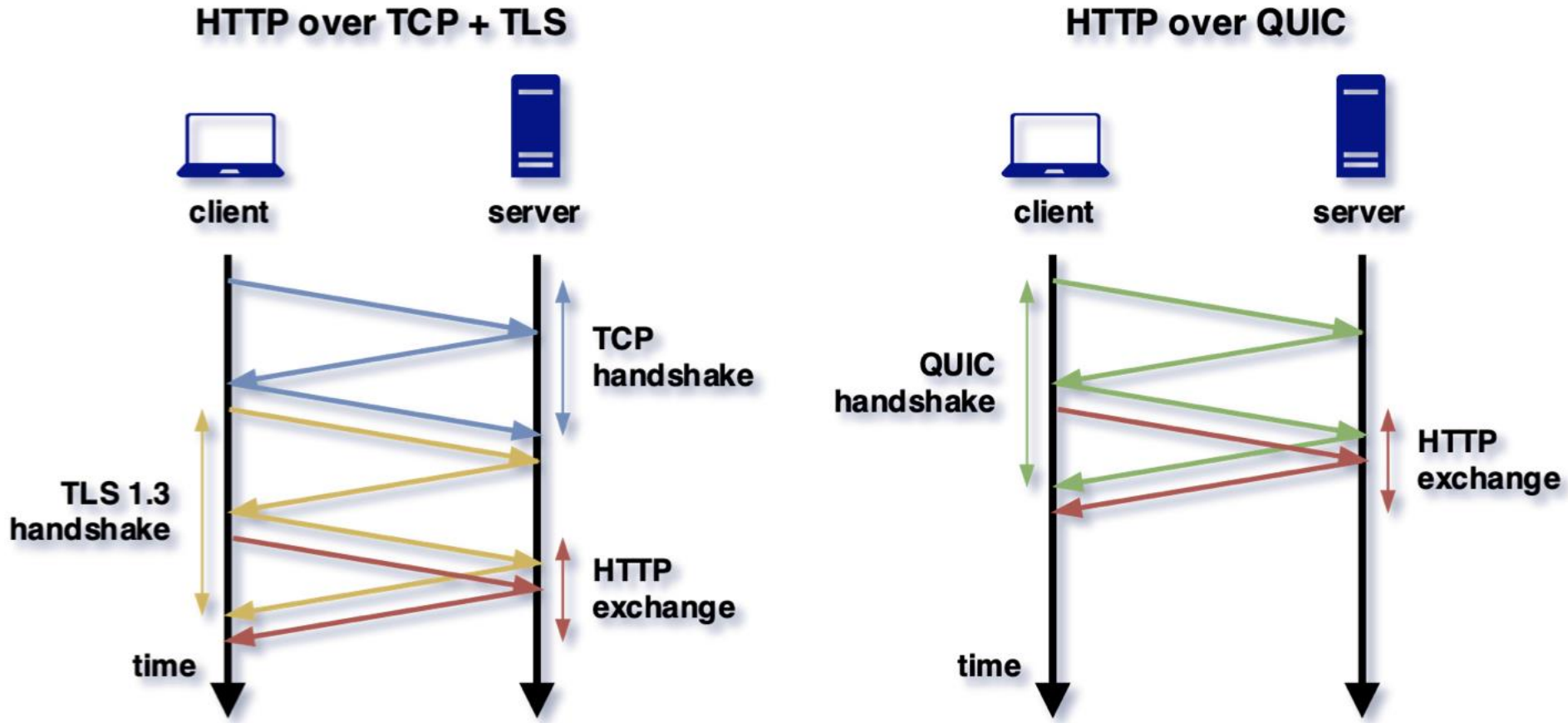
TCP (reliability, congestion control state) + TLS (authentication, crypto state)

- 2 serial handshakes



QUIC: reliability, congestion control, authentication, crypto state

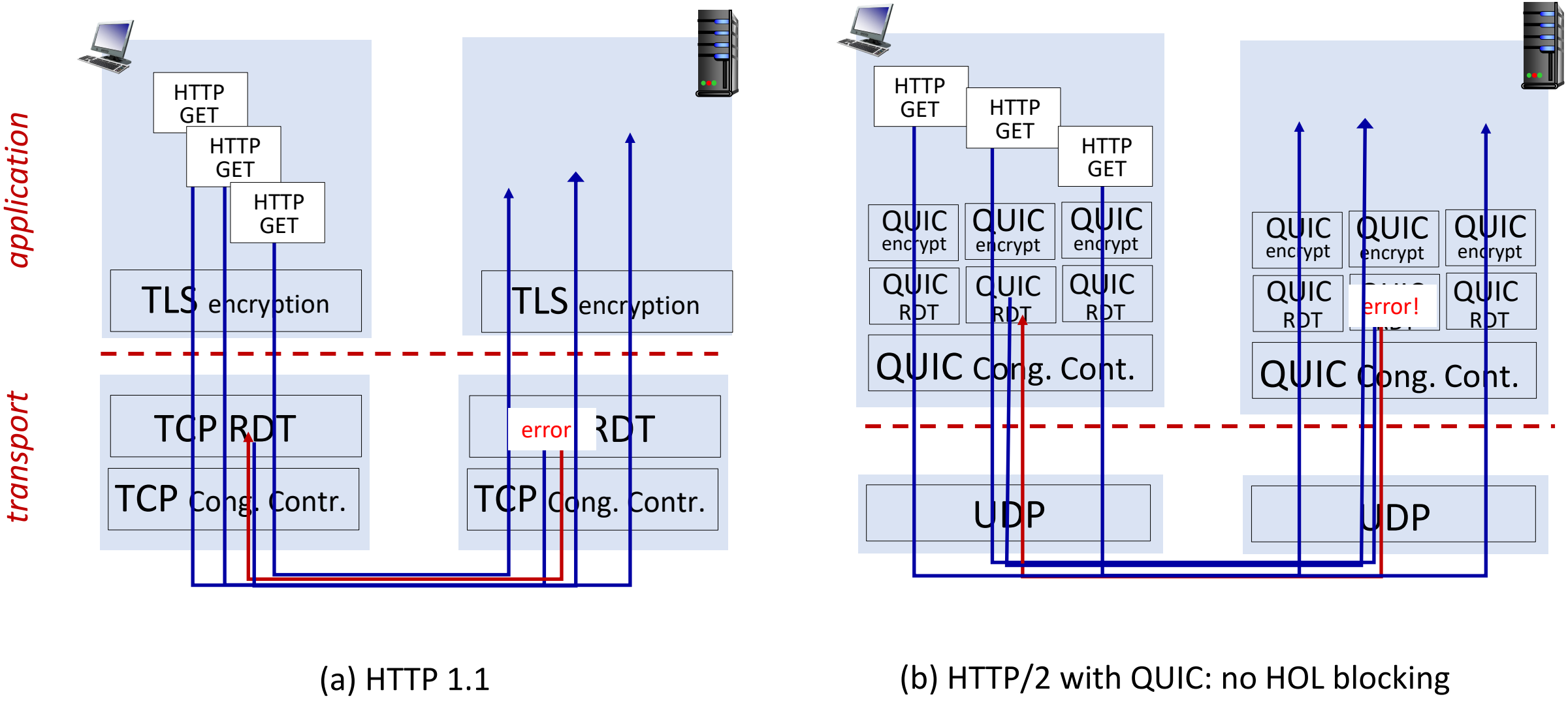
- 1 handshake



Connection handshake

1-RTT

QUIC: streams: parallelism, no HOL blocking



(a) HTTP 1.1

(b) HTTP/2 with QUIC: no HOL blocking

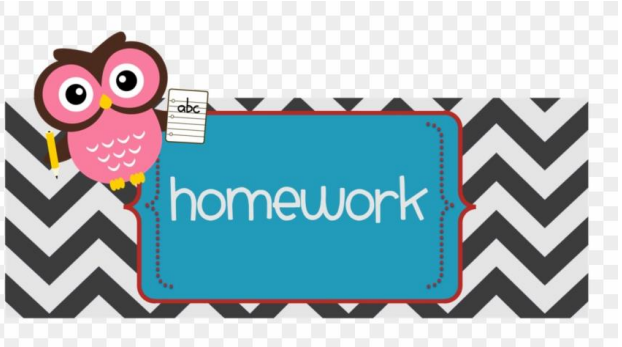
Chapter 3: summary

- principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- instantiation, implementation in the Internet
  - UDP
  - TCP

Up next:

- leaving the network "edge" (application, transport layers)
- into the network "core"
- two network-layer chapters:
  - data plane
  - control plane

Homework



1. Why does UDP provides a checksum in the first place, as many link-layer protocols (including the popular Ethernet protocol) also provide error checking. If an error has been found, what would UDP do?
2. Introduction of TCP CUBIC
3. Introduction of BBR
4. Compare the features of TCP Tahoe, TCP Reno, TCP CUBIC, BBR， TCP Vegas and their present deployment.
5. Evolving transport-layer functionality (HTTP/3)
6. What is SYN flooding? Discuss possible solution? Present situation.

Wireshark LAB

